

Standar Verifikasi Keamanan Aplikasi

Versi 5.0.0



Mei 2025

Contents

Halaman depan	7
Tentang Standar	7
Hak Cipta dan Lisensi	7
Pimpinan Proyek	7
Kelompok Kerja	7
Kontributor Utama Lainnya	7
Kontributor dan Peninjau Lainnya	8
Prakata	8
Pengantar	8
Prinsip-prinsip di Balik Versi 5.0	8
Pandangan ke Depan	9
Apa itu ASVS?	9
Cakupan ASVS	10
Aplikasi (Application)	10
Keamanan (Security)	10
Verifikasi (Verification)	11
Standar (Standard)	11
Persyaratan (Requirement)	11
Keputusan Keamanan yang Terdokumentasi	11
Tingkat Verifikasi Keamanan Aplikasi (Application Security Verification Levels)	13
Evaluasi Level	13
Level 1	13
Level 2	14
Level 3	14
Level Mana yang Harus Dicapai	14
Cara Menggunakan ASVS	15
Struktur ASVS	15
Strategi Rilis	15
Fleksibilitas dalam ASVS	15
Cara Merujuk Persyaratan ASVS	16
Melakukan Fork pada ASVS	16
Kasus Penggunaan (Use Cases) ASVS	17
Sebagai Panduan Arsitektur Keamanan Terperinci	17
Sebagai Referensi Secure Coding Khusus	17
Sebagai Panduan untuk Pengujian Otomatis (Unit & Integration Tests)	17
Untuk Pelatihan Pengembangan Aman (Secure Development Training)	18
Sebagai Kerangka Kerja Pengadaan Perangkat Lunak Aman	18
Menerapkan ASVS dalam Praktik	18

Penilaian dan Sertifikasi	18
Sikap OWASP Terhadap Sertifikasi ASVS dan Trust Mark	18
Cara Memverifikasi Kepatuhan ASVS	19
Pelaporan Verifikasi	19
Cakupan Verifikasi	19
Mekanisme Verifikasi	19
Perubahan Dibandingkan dengan v4.x	21
Pengantar	21
Filosofi Persyaratan	21
Cakupan dan Fokus	21
Penekanan pada Tujuan Keamanan Dibandingkan Mekanisme	21
Keputusan Keamanan yang Terdokumentasi	22
Perubahan Struktural dan Bab-Bab Baru	22
Penghapusan Pemetaan Langsung ke Standar Lain	23
Pengurangan Keterkaitan dengan NIST Digital Identity Guidelines	23
Beralih dari Common Weakness Enumeration (CWE)	23
Memikirkan Ulang Definisi Level	23
Level Masuk yang Lebih Mudah	24
Kekeliruan Pemikiran tentang Kemampuan Pengujian (Testability)	24
Tidak Hanya Tentang Risiko	24
V1 Encoding dan Sanitization	25
Tujuan Kontrol	25
V1.1 Arsitektur Encoding dan Sanitization	25
V1.2 Pencegahan Injection	25
V1.3 Sanitization	27
V1.4 Memory, String, dan Unmanaged Code	29
V1.5 Safe Deserialization	29
Referensi	30
V2 Validation dan Business Logic	31
Tujuan Kontrol	31
V2.1 Dokumentasi Validation dan Business Logic	31
V2.2 Input Validation	31
V2.3 Keamanan Business Logic	33
V2.4 Anti-automation	33
Referensi	34
V3 Web Frontend Security	34
Tujuan Kontrol	34
V3.1 Dokumentasi Web Frontend Security	34

V3.2 Interpretasi Konten yang Tidak Diinginkan	35
V3.3 Konfigurasi Cookie	35
V3.4 Header Mekanisme Keamanan Browser	36
V3.5 Pemisahan Origin Browser	38
V3.6 Integritas Resource Eksternal	40
V3.7 Pertimbangan Keamanan Browser Lainnya	40
Referensi	41
V4 API dan Web Service	41
Tujuan Kontrol	41
V4.1 Keamanan Web Service Umum	42
V4.2 Validasi Struktur Pesan HTTP	42
V4.3 GraphQL	43
V4.4 WebSocket	44
Referensi	44
V5 File Handling	45
Tujuan Kontrol	45
V5.1 Dokumentasi Penanganan File	45
V5.2 Upload File dan Konten	45
V5.3 Penyimpanan File	46
V5.4 Download File	47
Referensi	47
V6 Athentication	48
Objektif Kontrol	48
V6.1 Dokumentasi Athentication	48
V6.2 Keamanan Password	49
V6.3 Keamanan Athentication Umum	50
V6.4 Siklus Hidup dan Pemulihan Faktor Athentication	52
V6.5 Persyaratan Umum Multi-factor Authentication	53
V6.6 Mekanisme Athentication Out-of-Band	54
V6.7 Mekanisme Athentication Kriptografi	55
V6.8 Athentication dengan Identity Provider	56
Referensi	57
V7 Session Management	58
Tujuan Kontrol	58
V7.1 Dokumentasi Session Management	58
V7.2 Keamanan Dasar Session Management	59
V7.3 Session Timeout	59
V7.4 Terminasi Sesi	60

V7.5 Pertahanan Terhadap Penyalahgunaan Sesi	61
V7.6 Federated Re-authentication	61
Referensi	62
V8 Authorization	62
Tujuan Kontrol	62
V8.1 Dokumentasi Authorization	62
V8.2 Desain Authorization Umum	63
V8.3 Authorization Level Operasi	64
V8.4 Pertimbangan Authorization Lainnya	65
Referensi	65
V9 Self-contained Tokens	65
Tujuan Kontrol	65
V9.1 Sumber dan Integritas Token	66
V9.2 Konten Token	66
Referensi	67
V10 OAuth dan OIDC	67
Tujuan Kontrol	67
V10.1 Keamanan Umum OAuth dan OIDC	69
V10.2 OAuth Client	70
V10.3 OAuth Resource Server	71
V10.4 OAuth Authorization Server	72
V10.5 OIDC Client	74
V10.6 OpenID Provider	75
V10.7 Manajemen Consent	75
Referensi	76
V11 Cryptography	77
Tujuan Kontrol	77
V11.1 Inventaris dan Dokumentasi Kriptografi	77
V11.2 Implementasi Cryptography yang Aman	78
V11.3 Algoritma Enkripsi	79
V11.4 Hashing dan Fungsi Berbasis Hash	80
V11.5 Nilai Acak (Random Values)	81
V11.6 Public Key Cryptography	82
V11.7 Kriptografi Data yang Sedang Digunakan (In-Use)	82
Referensi	83
V12 Secure Communication	83
Tujuan Kontrol	83
V12.1 Panduan Umum Keamanan TLS	83

V12.2 Komunikasi HTTPS dengan Layanan yang Menghadap Eksternal	84
V12.3 Keamanan Komunikasi Umum Antar Layanan (Service to Service)	84
Referensi	85
V13 Configuration	86
Tujuan Kontrol	86
V13.1 Dokumentasi Konfigurasi	86
V13.2 Konfigurasi Komunikasi Backend	87
V13.3 Manajemen Secret	88
V13.4 Kebocoran Informasi yang Tidak Diinginkan	88
Referensi	89
V14 Data Protection	89
Tujuan Kontrol	89
V14.1 Dokumentasi Perlindungan Data	90
V14.2 Perlindungan Data Secara Umum	91
V14.3 Perlindungan Data di Sisi Client	92
Referensi	92
V15 Secure Coding dan Architecture	93
Tujuan Kontrol	93
V15.1 Dokumentasi Secure Coding dan Arsitektur	93
V15.2 Arsitektur Keamanan dan Dependensi	94
V15.3 Defensive Coding	95
V15.4 Concurrency yang Aman	96
Referensi	97
V16 Security Logging dan Error Handling	98
Tujuan Kontrol	98
V16.1 Dokumentasi Security Logging	98
V16.2 General Logging	98
V16.3 Security Events	99
V16.4 Log Protection	100
V16.5 Error Handling	100
Referensi	101
V17 WebRTC	101
Tujuan Kontrol	101
V17.1 TURN Server	102
V17.2 Media	103
V17.3 Signaling	104
Referensi	105

Lampiran A: Glosarium	105
Lampiran B: Referensi	111
Proyek Inti OWASP	112
Proyek Cheat Sheet OWASP	112
Proyek-proyek Terkait Keamanan Mobile	112
Proyek-proyek terkait Internet of Things OWASP	112
Proyek OWASP Serverless	112
Yang lain	112
Lampiran C: Standar Kriptografi	113
Inventarisasi dan Dokumentasi Kriptografi	113
Kekuatan Setara dari Parameter Kriptografi	114
Nilai Acak (Random Values)	114
Algoritma Cipher	115
Mode Cipher AES	116
Key Wrapping	117
Enkripsi Terotentikasi (Authenticated Encryption)	117
Fungsi Hash	118
Fungsi Hash untuk Kasus Penggunaan Umum	118
Fungsi Hash untuk Penyimpanan Kata Sandi	120
Key Derivation Functions (KDFs)	120
Key Derivation Functions Umum	120
Key Derivation Functions Berbasis Kata Sandi	121
Mekanisme Pertukaran Kunci (Key Exchange)	121
Skema KEX	122
Grup Diffie-Hellman	122
Message Authentication Codes (MAC)	123
Tanda Tangan Digital (Digital Signatures)	124
Standar Enkripsi Pasca-Kuantum (Post-Quantum Encryption)	124
Lampiran D: Rekomendasi	125
Pendahuluan	125
Mekanisme yang direkomendasikan dan termasuk dalam cakupan	125
Prinsip-prinsip Keamanan Perangkat Lunak	126
Proses Keamanan Perangkat Lunak	126
Lampiran E - Kontributor	127

Halaman depan

Tentang Standar

Standar Verifikasi Keamanan Aplikasi (ASVS) adalah daftar persyaratan keamanan aplikasi yang dapat digunakan oleh arsitek, pengembang, penguji, pakar keamanan, penyedia alat, dan pengguna untuk merancang, membangun, menguji, dan memverifikasi keamanan aplikasi.

Hak Cipta dan Lisensi

Versi 5.0.0, Mei 2025



Figure 1: license

Copyright © 2008-2025 The OWASP Foundation.

Dokumen ini diterbitkan di bawah lisensi Creative Commons Attribution-ShareAlike 4.0 International License.

Jika ingin menggunakan kembali atau mendistribusikan karya ini, Anda wajib menyampaikan ketentuan lisensi karya ini secara jelas kepada pihak lain.

Pimpinan Proyek

Elar Lang	Josh C Grossman
Jim Manico	Daniel Cuthbert

Kelompok Kerja

Tobias Ahnoff	Ralph Andalis	Ryan Armstrong	Gabriel Corona
Meghan Jacquot	Shanni Prutchi	Iman Sharafaldin	Eden Yardeni

Kontributor Utama Lainnya

Sjoerd Langkemper	Isaac Lewis
-------------------	-------------

Mark CarneySandro Gauci

Kontributor dan Peninjau Lainnya

Kami telah mencantumkan daftar kontributor lainnya di Lampiran E.

Jika ada kredit yang tidak tercantum dalam daftar kredit versi 5.x, silakan buat tiket di GitHub agar kredit tersebut dicantumkan dalam pembaruan versi 5.x mendatang.

Standar Verifikasi Keamanan Aplikasi ini dikembangkan berdasarkan hasil kerja para pihak yang terlibat dalam ASVS 1.0 (2008) hingga 4.0 (2019). Sebagian besar struktur dan banyak poin verifikasi yang masih terdapat dalam ASVS saat ini awalnya disusun oleh Andrew van der Stock, Mike Boberski, Jeff Williams, dan Dave Wichers, di antara banyak kontributor lainnya. Terima kasih kepada semua pihak yang telah berkontribusi di masa lalu. Untuk daftar lengkap kontributor sebelumnya, silakan merujuk ke masing-masing versi sebelumnya.

Prakata

Selamat datang di Standar Verifikasi Keamanan Aplikasi (ASVS) Versi 5.0.

Pengantar

Pertama kali diluncurkan pada tahun 2008 melalui kolaborasi komunitas global, ASVS mendefinisikan serangkaian persyaratan keamanan yang komprehensif untuk merancang, mengembangkan, dan menguji aplikasi serta layanan web modern.

Menyusul rilis ASVS 4.0 pada tahun 2019 dan pembaruan minornya (v4.0.3) pada tahun 2021, Versi 5.0 mewakili tonggak sejarah penting yang dimoderinisasi untuk mencerminkan kemajuan terbaru dalam keamanan perangkat lunak.

ASVS 5.0 adalah hasil dari kontribusi luas dari para pemimpin proyek, anggota kelompok kerja, dan komunitas OWASP yang lebih luas untuk memperbarui dan meningkatkan standar penting ini.

Prinsip-prinsip di Balik Versi 5.0

Revisi besar ini telah dikembangkan dengan mempertimbangkan beberapa prinsip utama:

- **Cakupan dan Fokus yang Disempurnakan:** Versi standar ini dirancang agar lebih selaras secara langsung dengan pilar-pilar dasar pada namanya: Application, Security, Verification, dan

Standard. Persyaratan telah ditulis ulang untuk menekankan pencegahan cacat keamanan (*security flaws*) daripada mewajibkan implementasi teknis tertentu. Teks persyaratan dimaksudkan untuk menjelaskan dirinya sendiri (*self-explanatory*), yaitu menjelaskan alasan mengapa persyaratan tersebut ada.

- **Dukungan untuk Keputusan Keamanan yang Terdokumentasi:** ASVS 5.0 memperkenalkan persyaratan untuk mendokumentasikan keputusan keamanan utama. Hal ini meningkatkan keterlacakan (*traceability*) dan mendukung implementasi yang peka terhadap konteks, memungkinkan organisasi untuk menyesuaikan postur keamanan mereka sesuai dengan kebutuhan dan risiko spesifik mereka.
- **Pembaruan Level:** Meskipun ASVS mempertahankan model tiga tingkatnya (*three-tier model*), definisi level telah berkembang untuk membuat ASVS lebih mudah diadopsi. Level 1 dirancang sebagai langkah awal untuk mengadopsi ASVS, yang menyediakan pertahanan lapisan pertama. Level 2 mewakili pandangan komprehensif tentang praktik keamanan standar, dan Level 3 menangani persyaratan tingkat lanjut (*high-assurance*).
- **Restrukturisasi dan Perluasan Konten:** ASVS 5.0 mencakup sekitar 350 persyaratan di 17 bab. Bab-bab telah diatur ulang demi kejelasan dan kemudahan penggunaan. Pemetaan dua arah antara v4.0 dan v5.0 disediakan untuk memfasilitasi migrasi.

Pandangan ke Depan

Sama halnya seperti mengamankan aplikasi yang tidak pernah benar-benar selesai, begitu pula dengan ASVS. Meskipun Versi 5.0 merupakan rilis besar, pengembangannya terus berlanjut. Rilis ini memungkinkan komunitas yang lebih luas untuk memanfaatkan peningkatan dan penambahan yang telah dikumpulkan, sekaligus meletakkan dasar untuk peningkatan di masa mendatang. Ini dapat mencakup upaya berbasis komunitas untuk membuat panduan implementasi dan verifikasi yang dibangun di atas kumpulan persyaratan utama.

ASVS 5.0 dirancang untuk berfungsi sebagai fondasi yang andal bagi pengembangan perangkat lunak yang aman. Komunitas diundang untuk mengadopsi, berkontribusi, dan membangun di atas standar ini untuk secara kolektif memajukan tingkat keamanan aplikasi.

Apa itu ASVS?

Application Security Verification Standard (ASVS) mendefinisikan persyaratan keamanan untuk aplikasi dan layanan web, serta merupakan sumber daya yang berharga bagi siapa saja yang bertujuan untuk merancang, mengembangkan, dan memelihara aplikasi yang aman atau mengevaluasi keamanannya.

Bab ini menguraikan aspek-aspek esensial dari penggunaan ASVS, termasuk cakupannya, struktur level berbasis prioritas, dan kasus penggunaan (*use cases*) utama untuk standar ini.

Cakupan ASVS

Cakupan (*scope*) dari ASVS didefinisikan oleh namanya: Application, Security, Verification, dan Standard. ASVS menetapkan persyaratan mana yang dimasukkan atau dikecualikan, dengan tujuan utama untuk mengidentifikasi prinsip-prinsip keamanan yang harus dicapai. Cakupan ini juga mempertimbangkan persyaratan dokumentasi, yang berfungsi sebagai fondasi bagi persyaratan implementasi.

Tidak ada yang namanya cakupan bagi penyerang (*attacker*). Oleh karena itu, persyaratan ASVS harus dievaluasi bersamaan dengan panduan untuk aspek-aspek lain dari siklus hidup aplikasi, termasuk proses CI/CD, *hosting*, dan aktivitas operasional.

Aplikasi (Application)

ASVS mendefinisikan “aplikasi” sebagai produk perangkat lunak yang sedang dikembangkan, di mana kontrol keamanan harus diintegrasikan ke dalamnya. ASVS tidak menetapkan aktivitas siklus hidup pengembangan atau mendikte bagaimana aplikasi harus dibangun melalui *pipeline* CI/CD; melainkan menentukan hasil keamanan yang harus dicapai dalam produk itu sendiri.

Komponen yang melayani, mengubah, atau memvalidasi lalu lintas HTTP, seperti Web Application Firewall (WAF), *load balancer*, atau *proxy*, dapat dianggap sebagai bagian dari aplikasi untuk tujuan spesifik tersebut, karena beberapa kontrol keamanan bergantung langsung padanya atau dapat diimplementasikan melaluinya. Komponen-komponen ini harus dipertimbangkan untuk persyaratan yang berkaitan dengan respons yang di-cache, *rate limiting*, atau pembatasan koneksi masuk dan keluar berdasarkan sumber dan tujuan.

Sebaliknya, ASVS umumnya mengecualikan persyaratan yang tidak relevan secara langsung dengan aplikasi atau di mana konfigurasinya berada di luar tanggung jawab aplikasi. Sebagai contoh, masalah DNS biasanya dikelola oleh tim atau fungsi yang terpisah.

Demikian pula, meskipun aplikasi bertanggung jawab atas cara mengonsumsi input dan menghasilkan output, jika proses eksternal berinteraksi dengan aplikasi atau datanya, proses tersebut dianggap di luar cakupan (*out of scope*) ASVS. Misalnya, pencadangan (*backup*) aplikasi atau datanya biasanya merupakan tanggung jawab proses eksternal dan tidak dikontrol oleh aplikasi atau pengembangnya.

Keamanan (Security)

Setiap persyaratan harus memiliki dampak yang terbukti terhadap keamanan. Absennya suatu persyaratan harus mengakibatkan aplikasi menjadi kurang aman, dan mengimplementasikan persyaratan tersebut harus mengurangi kemungkinan atau dampak dari risiko keamanan.

Semua pertimbangan lainnya, seperti aspek fungsional, gaya penulisan kode (*code style*), atau persyaratan kebijakan, berada di luar cakupan.

Verifikasi (Verification)

Persyaratan harus dapat diverifikasi (*verifiable*), dan verifikasi tersebut harus menghasilkan keputusan “gagal”(*fail*) atau “lulus”(*pass*).

Standar (Standard)

ASVS dirancang untuk menjadi kumpulan persyaratan keamanan yang diimplementasikan untuk mematuhi standar. Ini berarti bahwa persyaratan dibatasi untuk mendefinisikan tujuan keamanan yang ingin dicapai. Informasi relevan lainnya dapat dibangun di atas ASVS atau dihubungkan melalui pemetaan (*mappings*).

Secara khusus, OWASP memiliki banyak proyek, dan ASVS secara sengaja menghindari tumpang tindih dengan konten di proyek lain. Sebagai contoh, pengembang mungkin memiliki pertanyaan, “bagaimana cara mengimplementasikan persyaratan tertentu dalam teknologi atau lingkungan spesifik saya,” dan ini harus dicakup oleh proyek *Cheat Sheet Series*. Verifikator mungkin memiliki pertanyaan “bagaimana cara menguji persyaratan ini di lingkungan ini,” dan ini harus dicakup oleh proyek *Web Security Testing Guide*.

Meskipun ASVS tidak hanya ditujukan untuk digunakan oleh pakar keamanan, standar ini tetap mengharapkan pembaca memiliki pengetahuan teknis untuk memahami kontennya atau memiliki kemampuan untuk meneliti konsep-konsep tertentu.

Persyaratan (Requirement)

Kata *requirement* (persyaratan) digunakan secara khusus dalam ASVS karena menggambarkan apa yang harus dicapai untuk memenuhinya. ASVS hanya berisi persyaratan wajib (*must*) dan tidak berisi rekomendasi (*should*) sebagai kondisi utama.

Dengan kata lain, rekomendasi, baik yang hanya berupa salah satu dari banyak opsi untuk menyelesaikan masalah atau pertimbangan gaya penulisan kode, tidak memenuhi definisi untuk menjadi sebuah persyaratan.

Persyaratan ASVS dimaksudkan untuk menangani prinsip-prinsip keamanan tertentu tanpa terlalu spesifik pada implementasi atau teknologi, sekaligus memberikan penjelasan mandiri tentang mengapa persyaratan itu ada. Ini juga berarti persyaratan tidak dibangun di sekitar metode verifikasi atau implementasi tertentu.

Keputusan Keamanan yang Terdokumentasi

Dalam keamanan perangkat lunak, merencanakan desain keamanan dan mekanisme yang akan digunakan sejak awal akan mengarah pada implementasi yang lebih konsisten dan andal pada produk atau fitur yang selesai dibuat.

Selain itu, untuk persyaratan tertentu, implementasinya akan rumit dan sangat spesifik dengan kebutuhan aplikasi. Contoh umumnya meliputi hak akses (*permissions*), validasi input, dan kontrol perlindungan di sekitar berbagai tingkat data sensitif.

Untuk mengatasi hal ini, daripada membuat pernyataan umum seperti “semua data harus dienkripsi” atau mencoba mencakup setiap kemungkinan kasus penggunaan dalam satu persyaratan, persyaratan dokumentasi dimasukkan untuk mewajibkan agar pendekatan dan konfigurasi pengembang aplikasi terhadap jenis kontrol ini didokumentasikan. Dokumentasi ini kemudian dapat ditinjau kesesuaiannya dan implementasi aktualnya dapat dibandingkan dengan dokumentasi untuk menilai apakah implementasi sesuai dengan ekspektasi.

Persyaratan ini dimaksudkan untuk mendokumentasikan keputusan yang telah diambil oleh organisasi pengembang aplikasi mengenai cara mengimplementasikan persyaratan keamanan tertentu.

Persyaratan dokumentasi selalu berada di bagian pertama dari suatu bab (meskipun tidak setiap bab memilikinya) dan selalu memiliki persyaratan implementasi terkait di mana keputusan yang didokumentasikan tersebut harus benar-benar diterapkan. Poin di sini adalah bahwa memverifikasi keberadaan dokumentasi dan memverifikasi implementasi aktualnya adalah dua aktivitas yang terpisah.

Ada dua pendorong utama dimasukkannya persyaratan ini. Pendorong pertama adalah bahwa persyaratan keamanan sering kali melibatkan penegakan aturan, misal: jenis file apa yang diizinkan untuk diunggah, kontrol bisnis apa yang harus ditegakkan, karakter apa saja yang diizinkan untuk kolom tertentu. Aturan-aturan ini akan berbeda untuk setiap aplikasi, dan oleh karena itu, ASVS tidak dapat mendefinisikan secara preskriptif bagaimana aturan tersebut seharusnya, begitu pula *cheat sheet* atau respons yang lebih detail tidak dapat membantu dalam kasus ini. Demikian pula, tanpa keputusan ini didokumentasikan, verifikasi persyaratan yang mengimplementasikan keputusan tersebut tidak akan mungkin dilakukan.

Pendorong kedua adalah untuk persyaratan tertentu, penting untuk memberikan fleksibilitas kepada pengembang aplikasi mengenai cara mengatasi tantangan keamanan tertentu. Misalnya, dalam versi ASVS sebelumnya, aturan *session timeout* sangat preskriptif. Secara praktis, banyak aplikasi (terutama yang berhadapan langsung dengan konsumen) memiliki aturan yang jauh lebih longgar dan memilih untuk mengimplementasikan kontrol mitigasi lainnya. Oleh karena itu, persyaratan dokumentasi secara eksplisit memberikan fleksibilitas di seputar hal ini.

Jelas, tidak diharapkan bahwa masing-masing pengembang individu yang membuat dan mendokumentasikan keputusan ini, melainkan organisasi secara keseluruhan yang mengambil keputusan tersebut dan menyampaikannya kepada pengembang yang kemudian mematuinya.

Menyediakan spesifikasi dan desain fitur baru kepada pengembang adalah bagian standar dari pengembangan perangkat lunak. Demikian pula, pengembang diharapkan menggunakan komponen umum dan mekanisme *user interface* yang ada alih-alih membuat keputusan sendiri setiap saat. Dengan demikian, memperluas hal ini ke ranah keamanan seharusnya tidak dipandang sebagai hal yang mengejutkan atau kontroversial.

Ada juga fleksibilitas tentang cara mencapai hal ini. Keputusan keamanan dapat didokumentasikan dalam dokumen tertulis yang dirujuk oleh pengembang. Atau, keputusan keamanan dapat di-

dokumentasikan dan diimplementasikan dalam pustaka kode umum (*common code library*) yang diwajibkan untuk digunakan oleh semua pengembang. Dalam kedua kasus tersebut, hasil yang diinginkan tetap tercapai.

Tingkat Verifikasi Keamanan Aplikasi (Application Security Verification Levels)

ASVS mendefinisikan tiga tingkat verifikasi keamanan, di mana setiap tingkat meningkat dalam kedalaman dan kompleksitasnya. Tujuan umumnya adalah agar organisasi memulai dari tingkat pertama untuk mengatasi masalah keamanan yang paling krusial, lalu naik ke tingkat yang lebih tinggi sesuai dengan kebutuhan organisasi dan aplikasi. Tingkat ini dapat ditampilkan sebagai L1, L2, dan L3 dalam dokumen dan teks persyaratan.

Setiap level ASVS menunjukkan persyaratan keamanan yang wajib dicapai pada level tersebut, dengan sisa persyaratan di level yang lebih tinggi berfungsi sebagai rekomendasi.

Untuk menghindari persyaratan duplikat atau persyaratan yang tidak lagi relevan pada level yang lebih tinggi, beberapa persyaratan berlaku untuk level tertentu tetapi memiliki kondisi yang lebih ketat untuk level di atasnya.

Evaluasi Level

Level-level ini didefinisikan berdasarkan evaluasi prioritas dari setiap persyaratan yang berpatokan pada pengalaman implementasi dan pengujian keamanan. Fokus utamanya adalah membandingkan pengurangan risiko (*risk reduction*) dengan upaya untuk mengimplementasikan persyaratan tersebut (*effort to implement*). Faktor kunci lainnya adalah menjaga ambang batas masuk (*barrier to entry*) tetap rendah.

Pengurangan risiko mempertimbangkan sejauh mana persyaratan tersebut mengurangi tingkat risiko keamanan dalam aplikasi, dengan memperhitungkan faktor dampak klasik *Confidentiality*, *Integrity*, dan *Availability*, serta mempertimbangkan apakah ini merupakan lapisan pertahanan utama (*primary layer of defense*) atau pertahanan berlapis (*defense in depth*).

Diskusi ketat di sekitar kriteria dan keputusan tingkat telah menghasilkan alokasi yang seharusnya berlaku untuk sebagian besar kasus, sambil menerima bahwa hal ini mungkin tidak 100% cocok untuk setiap situasi. Ini berarti dalam kasus tertentu, organisasi mungkin ingin memprioritaskan persyaratan dari level yang lebih tinggi lebih awal berdasarkan pertimbangan risiko spesifik mereka sendiri.

Jenis persyaratan di setiap level dapat dikarakterisasikan sebagai berikut:

Level 1

Tingkat ini berisi persyaratan minimum yang harus dipertimbangkan saat mengamankan aplikasi dan mewakili titik awal yang sangat penting. Tingkat ini berisi sekitar 20% dari seluruh persyaratan

ASVS. Tujuan dari tingkat ini adalah memiliki persyaratan sesedikit mungkin untuk menurunkan ambang batas masuk.

Persyaratan ini umumnya kritis atau mendasar, sebagai pertahanan lapisan pertama untuk mencegah serangan umum yang tidak memerlukan kerentanan atau prakondisi lain agar dapat dieksploitasi.

Selain persyaratan pertahanan lapisan pertama, beberapa persyaratan memiliki dampak yang lebih kecil pada level yang lebih tinggi, seperti persyaratan yang berkaitan dengan kata sandi. Persyaratan tersebut lebih penting untuk Level 1, karena pada level yang lebih tinggi, persyaratan otentikasi multi-faktor (*multi-factor authentication*) menjadi lebih relevan.

Level 1 tidak selalu dapat diuji dengan *penetration test* oleh penguji eksternal tanpa akses internal ke dokumentasi atau kode (seperti pengujian *black box*), meskipun jumlah persyaratan yang lebih sedikit seharusnya membuatnya lebih mudah untuk diverifikasi.

Level 2

Sebagian besar aplikasi harus berusaha mencapai tingkat keamanan ini. Sekitar 50% persyaratan dalam ASVS berada di L2, yang berarti aplikasi perlu mengimplementasikan sekitar 70% dari seluruh persyaratan ASVS (semua persyaratan L1 dan L2) agar sesuai dengan L2.

Persyaratan ini umumnya berkaitan dengan serangan yang kurang umum atau perlindungan yang lebih rumit terhadap serangan umum. Persyaratan ini mungkin masih berupa pertahanan lapisan pertama, atau mungkin memerlukan prakondisi tertentu agar serangan berhasil.

Level 3

Tingkat ini harus menjadi target bagi aplikasi yang ingin menunjukkan tingkat keamanan tertinggi dan menyediakan sisa ~30% persyaratan terakhir yang harus dipenuhi.

Persyaratan pada bagian ini umumnya merupakan mekanisme pertahanan berlapis (*defense-in-depth*) atau kontrol lain yang berguna tetapi sulit diimplementasikan.

Level Mana yang Harus Dicapai

Level berbasis prioritas dimaksudkan untuk memberikan gambaran tentang kematangan keamanan aplikasi (*application security maturity*) dari organisasi dan aplikasi tersebut. Daripada ASVS mendikte secara kaku level apa yang harus dicapai oleh suatu aplikasi, organisasi harus menganalisis risikonya dan memutuskan level mana yang menurut mereka sesuai, tergantung pada sensitivitas aplikasi dan ekspektasi dari pengguna aplikasi.

Sebagai contoh, *startup* tahap awal yang hanya mengumpulkan data sensitif secara terbatas dapat memutuskan untuk berfokus pada Level 1 untuk target keamanan awalnya, tetapi sebuah bank

mungkin akan kesulitan membenarkan tingkat di bawah Level 3 kepada nasabahnya untuk aplikasi perbankan *online* mereka.

Cara Menggunakan ASVS

Struktur ASVS

ASVS terdiri dari total sekitar 350 persyaratan yang dibagi menjadi 17 bab, di mana masing-masing bab dibagi lagi menjadi beberapa bagian.

Tujuan dari pembagian bab dan bagian ini adalah untuk mempermudah pemeliharaan atau penyaringan bab dan bagian berdasarkan apa yang relevan untuk aplikasi. Misalnya, untuk API *machine-to-machine*, persyaratan dalam bab V3 yang terkait dengan *web frontend* tidak akan relevan. Jika tidak ada penggunaan OAuth atau WebRTC, bab-bab tersebut juga dapat diabaikan.

Strategi Rilis

Rilis ASVS mengikuti pola “Major.Minor.Patch” dan nomor-nomor tersebut memberikan informasi tentang apa yang telah berubah dalam rilis tersebut:

- **Major release** - Restrukturisasi penuh, hampir semuanya dapat berubah, termasuk nomor persyaratan. Reevaluasi kepatuhan akan diperlukan (misalnya, 4.0.3 -> 5.0.0).
- **Minor release** - Persyaratan dapat ditambahkan atau dihapus, tetapi penomoran secara keseluruhan akan tetap sama. Reevaluasi kepatuhan akan diperlukan, tetapi seharusnya lebih mudah (misalnya, 5.0.0 -> 5.1.0).
- **Patch release** - Persyaratan dapat dihapus (misalnya, jika duplikat atau usang) atau dibuat lebih longgar, tetapi aplikasi yang mematuhi rilis sebelumnya juga akan mematuhi rilis *patch* ini (misalnya, 5.0.0 -> 5.0.1).

Hal-hal di atas secara khusus berkaitan dengan persyaratan dalam ASVS. Perubahan pada teks sekutarnya dan konten lain seperti lampiran tidak dianggap sebagai perubahan besar (*breaking change*).

Fleksibilitas dalam ASVS

Beberapa poin yang dijelaskan di atas, seperti persyaratan dokumentasi dan mekanisme level, memberikan kemampuan untuk menggunakan ASVS secara lebih fleksibel dan spesifik sesuai kebutuhan organisasi.

Selain itu, organisasi sangat didorong untuk membuat *fork* khusus organisasi atau domain yang menyesuaikan persyaratan berdasarkan karakteristik spesifik dan tingkat risiko aplikasi mereka. Namun, penting untuk menjaga keterlacakan (*traceability*) sehingga memenuhi persyaratan 4.1.1 berarti hal yang sama di semua versi.

Ideally, setiap organisasi harus membuat ASVS tersendiri yang disesuaikan, dengan mengabaikan bagian yang tidak relevan (misalnya GraphQL, WebSockets, SOAP, jika tidak digunakan). Versi atau suplemen ASVS khusus organisasi juga merupakan tempat yang baik untuk menyediakan panduan implementasi internal, yang merinci pustaka (*libraries*) atau sumber daya yang digunakan saat mematuhi persyaratan.

Cara Merujuk Persyaratan ASVS

Setiap persyaratan memiliki pengenal (*identifier*) dalam format <bab>.<bagian>.<persyaratan>, di mana setiap elemen berupa angka. Contohnya, 1.11.3.

- Nilai <bab> sesuai dengan bab asal persyaratan tersebut; contohnya, semua persyaratan 1.#.# berasal dari bab ‘Encoding and Sanitization’.
- Nilai <bagian> sesuai dengan bagian dalam bab tersebut di mana persyaratan muncul; contohnya: semua persyaratan 1.2.# berada di bagian ‘Injection Prevention’ pada bab ‘Encoding and Sanitization’.
- Nilai <persyaratan> mengidentifikasi persyaratan spesifik dalam bab dan bagian tersebut, misalnya 1.2.5 yang pada versi 5.0.0 dari standar ini berbunyi:

Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.

Karena pengenal dapat berubah antar versi standar, lebih disukai jika dokumen, laporan, atau alat lain menggunakan format berikut: v<versi>-<bab>.<bagian>.<persyaratan>, di mana ‘versi’ adalah tag versi ASVS. Contohnya: v5.0.0-1.2.5 akan dipahami sebagai persyaratan ke-5 dalam bagian ‘Injection Prevention’ dari bab ‘Encoding and Sanitization’ pada versi 5.0.0. (Ini dapat diringkas sebagai v<versi>-<pengenal_persyaratan>.)

Catatan: Huruf v yang mendahului nomor versi dalam format ini harus selalu menggunakan huruf kecil.

Jika pengenal digunakan tanpa menyertakan elemen v<versi>, maka dianggap merujuk pada konten Application Security Verification Standard terbaru. Seiring berkembang dan berubahnya standar, hal ini dapat menimbulkan masalah, itulah sebabnya penulis atau pengembang harus menyertakan elemen versi.

Daftar persyaratan ASVS disediakan dalam format CSV, JSON, dan format lain yang dapat berguna untuk referensi atau penggunaan terprogram.

Melakukan Fork pada ASVS

Organisasi dapat memperoleh manfaat dari mengadopsi ASVS dengan memilih salah satu dari tiga level atau dengan membuat *fork* khusus domain yang menyesuaikan persyaratan per tingkat risiko aplikasi. Jenis *fork* ini sangat didorong, asalkan mempertahankan keterlacakan (*traceability*) sehingga memenuhi persyaratan 4.1.1 berarti hal yang sama di semua versi.

Ideally, setiap organisasi harus membuat ASVS tersendiri yang disesuaikan, dengan mengabaikan bagian yang tidak relevan (misalnya GraphQL, Websockets, SOAP, jika tidak digunakan). Proses *forking* harus dimulai dengan ASVS Level 1 sebagai tolok ukur (*baseline*), lalu meningkat ke Level 2 atau 3 berdasarkan risiko aplikasi.

Kasus Penggunaan (Use Cases) ASVS

ASVS dapat digunakan untuk menilai keamanan aplikasi dan ini dijelaskan lebih mendalam di bab berikutnya. Namun, beberapa potensi penggunaan lain untuk ASVS (atau versi *fork*) telah diidentifikasi:

Sebagai Panduan Arsitektur Keamanan Terperinci

Salah satu penggunaan ASVS yang paling umum adalah sebagai sumber daya bagi *security architect*. Sumber daya yang tersedia tentang cara membangun arsitektur aplikasi yang aman sangat terbatas, terutama untuk aplikasi modern. ASVS dapat digunakan untuk mengisi celah tersebut dengan memungkinkan *security architect* memilih kontrol yang lebih baik untuk masalah umum, seperti pola perlindungan data dan strategi validasi input. Persyaratan arsitektur dan dokumentasi akan sangat berguna untuk tujuan ini.

Sebagai Referensi Secure Coding Khusus

ASVS dapat digunakan sebagai dasar untuk menyiapkan referensi pengodean aman (*secure coding reference*) selama pengembangan aplikasi, membantu pengembang memastikan mereka tetap mempertimbangkan keamanan saat membuat perangkat lunak. Meskipun ASVS dapat menjadi basisnya, organisasi harus menyiapkan panduan spesifik mereka sendiri yang jelas dan terpadu, dan idealnya disiapkan berdasarkan arahan dari *security engineer* atau *security architect*. Sebagai ekstensi dari hal ini, organisasi didorong untuk menyiapkan mekanisme keamanan dan pustaka (*libraries*) yang disetujui yang dapat dirujuk dalam panduan dan digunakan oleh pengembang.

Sebagai Panduan untuk Pengujian Otomatis (Unit & Integration Tests)

ASVS dirancang agar sangat dapat diuji (*testable*). Beberapa verifikasi bersifat teknis sedangkan persyaratan lain (seperti persyaratan arsitektur dan dokumentasi) mungkin memerlukan peninjauan dokumentasi atau arsitektur. Dengan membangun *unit test* dan *integration test* yang menguji serta melakukan *fuzzing* untuk kasus-kasus penyalahgunaan (*abuse cases*) yang spesifik dan relevan terkait dengan persyaratan teknis, akan lebih mudah untuk memastikan bahwa kontrol tersebut beroperasi dengan benar pada setiap *build*. Misalnya, pengujian tambahan dapat dibuat untuk *test suite* kontroler *login*, menguji parameter *username* untuk *default username* umum, *account enumeration*, *brute*

force, *LDAP/SQL injection*, dan *XSS*. Demikian pula, pengujian pada parameter *password* harus mencakup kata sandi umum, panjang kata sandi, *null byte injection*, penghapusan parameter, *XSS*, dan lainnya.

Untuk Pelatihan Pengembangan Aman (Secure Development Training)

ASVS juga dapat digunakan untuk mendefinisikan karakteristik perangkat lunak yang aman. Banyak kursus “secure coding” hanya berupa kursus *ethical hacking* yang diberi sedikit tip pengodean. Ini mungkin tidak membantu pengembang untuk menulis kode yang lebih aman. Sebaliknya, kursus pengembangan aman dapat menggunakan ASVS dengan fokus kuat pada mekanisme positif yang ditemukan dalam ASVS, daripada Top 10 hal negatif yang tidak boleh dilakukan. Struktur ASVS juga memberikan alur logika yang teratur untuk mempelajari berbagai topik saat mengamankan aplikasi.

Sebagai Kerangka Kerja Pengadaan Perangkat Lunak Aman

ASVS adalah kerangka kerja yang sangat baik untuk membantu pengadaan perangkat lunak aman atau pengadaan layanan pengembangan kustom. Pembeli hanya perlu menetapkan persyaratan bahwa perangkat lunak yang ingin mereka beli harus dikembangkan pada ASVS Level X, dan meminta penjual membuktikan bahwa perangkat lunak tersebut memenuhi ASVS Level X.

Menerapkan ASVS dalam Praktik

Ancaman yang berbeda memiliki motivasi yang berbeda pula. Beberapa industri memiliki aset informasi/teknologi yang unik serta persyaratan kepatuhan regulasi khusus domain.

Organisasi sangat didorong untuk melihat secara mendalam karakteristik risiko unik mereka berdasarkan sifat bisnis mereka, dan berdasarkan risiko serta persyaratan bisnis tersebut, tentukan level ASVS yang sesuai.

Penilaian dan Sertifikasi

Sikap OWASP Terhadap Sertifikasi ASVS dan Trust Mark

OWASP, sebagai organisasi nirlaba yang netral vendor, tidak menyertifikasi vendor, verifikator, atau perangkat lunak apa pun. Jaminan (*assurance*), *trust mark*, atau sertifikasi apa pun yang mengklaim kepatuhan ASVS tidak didukung secara resmi oleh OWASP, sehingga organisasi harus berhati-hati terhadap klaim pihak ketiga mengenai sertifikasi ASVS.

Organisasi dapat menawarkan layanan jaminan (*assurance services*), asalkan mereka tidak mengklaim sertifikasi resmi dari OWASP.

Cara Memverifikasi Kepatuhan ASVS

ASVS sengaja dibuat tidak preskriptif mengenai cara tepat memverifikasi kepatuhan pada tingkat panduan pengujian. Namun, penting untuk menyoroti beberapa poin kunci.

Pelaporan Verifikasi

Laporan *penetration testing* tradisional biasanya melaporkan masalah “berdasarkan pengecualian” (*by exception*), yang hanya mencantumkan kegagalan. Namun, laporan sertifikasi ASVS harus mencakup cakupan (*scope*), ringkasan dari semua persyaratan yang diperiksa, persyaratan di mana pengecualian dicatat, serta panduan tentang penyelesaian masalah. Beberapa persyaratan mungkin tidak berlaku (misalnya, manajemen sesi pada API *stateless*), dan hal ini harus dicatat dalam laporan.

Cakupan Verifikasi

Organisasi yang mengembangkan aplikasi umumnya tidak akan mengimplementasikan semua persyaratan, karena beberapa mungkin tidak relevan atau kurang signifikan berdasarkan fungsionalitas aplikasi tersebut. Verifikator harus memperjelas cakupan verifikasi, termasuk Level mana yang ingin dicapai oleh organisasi dan persyaratan mana saja yang dilingkupi. Hal ini harus dilihat dari perspektif apa saja yang dimasukkan, alih-alih apa yang tidak dimasukkan. Mereka juga harus memberikan pandangan/pendapat mengenai alasan mengecualikan persyaratan yang belum diimplementasikan.

Hal ini memungkinkan konsumen laporan verifikasi untuk memahami konteks verifikasi dan membuat keputusan yang tepat mengenai tingkat kepercayaan yang dapat mereka berikan pada aplikasi tersebut.

Organisasi penilai dapat memilih metode pengujian mereka sendiri, tetapi harus mengungkapkannya dalam laporan dan metode ini idealnya dapat diulang (*repeatable*). Berbagai metode, seperti *manual penetration test* atau analisis *source code*, dapat digunakan untuk memverifikasi aspek-aspek seperti validasi input, tergantung pada aplikasi dan persyaratannya.

Mekanisme Verifikasi

Terdapat sejumlah teknik berbeda yang mungkin diperlukan untuk memverifikasi persyaratan ASVS tertentu. Selain *penetration testing* (menggunakan kredensial yang valid untuk mendapatkan cakupan aplikasi secara penuh), memverifikasi persyaratan ASVS dapat memerlukan akses ke dokumentasi, *source code*, konfigurasi, dan orang-orang yang terlibat dalam proses pengembangan. Terutama untuk memverifikasi persyaratan Level 2 dan Level 3. Sudah menjadi praktik standar untuk menyediakan bukti temuan yang kuat dengan dokumentasi terperinci, yang dapat mencakup lembar kerja (*work papers*), tangkapan layar, skrip, dan log pengujian. Hanya menjalankan alat otomatis tanpa

pengujian menyeluruh tidaklah cukup untuk sertifikasi, karena setiap persyaratan harus diuji secara terverifikasi.

Penggunaan otomatisasi untuk memverifikasi persyaratan ASVS adalah topik yang terus-menerus menarik perhatian. Oleh karena itu, penting untuk memperjelas beberapa poin terkait pengujian otomatis dan *black box*.

Peran Alat Pengujian Keamanan Otomatis (Automated Security Testing Tools)

Ketika alat pengujian keamanan otomatis seperti *Dynamic and Static Application Security Testing* (DAST dan SAST) diimplementasikan dengan benar dalam *build pipeline*, alat tersebut mungkin dapat mengidentifikasi beberapa masalah keamanan yang seharusnya tidak pernah ada. Namun, tanpa konfigurasi dan penyesuaian (*tuning*) yang cermat, alat-alat ini tidak akan memberikan cakupan yang diperlukan, dan tingkat *noise* akan mencegah masalah keamanan yang sebenarnya untuk diidentifikasi dan dimitigasi.

Meskipun hal ini dapat memberikan cakupan untuk beberapa persyaratan teknis yang lebih dasar dan lugas seperti yang berkaitan dengan *output encoding* atau sanitasi, sangat penting untuk dicatat bahwa alat-alat ini tidak akan mampu sepenuhnya memverifikasi banyak persyaratan ASVS yang lebih rumit atau yang berkaitan dengan logika bisnis dan kontrol akses.

Untuk persyaratan yang kurang sederhana, kemungkinan otomatisasi masih dapat dimanfaatkan, tetapi verifikasi khusus aplikasi perlu ditulis untuk mencapainya. Ini mungkin mirip dengan *unit test* dan *integration test* yang mungkin sudah digunakan oleh organisasi. Oleh karena itu, infrastruktur otomatisasi pengujian yang ada dapat digunakan untuk menulis pengujian khusus ASVS ini. Meskipun hal ini memerlukan investasi jangka pendek, manfaat jangka panjang dari kemampuan memverifikasi persyaratan ASVS ini secara terus-menerus akan sangat signifikan.

Singkatnya, dapat diuji menggunakan otomatisasi != menjalankan alat siap pakai (*off the shelf tool*).

Peran Penetration Testing

Meskipun L1 dalam versi 4.0 dioptimalkan untuk pengujian *black box* (tanpa dokumentasi dan tanpa *source code*), bahkan saat itu standar sudah menegaskan bahwa hal tersebut bukanlah aktivitas jaminan (*assurance*) yang efektif dan sangat tidak dianjurkan.

Pengujian tanpa akses ke informasi tambahan yang diperlukan adalah mekanisme yang tidak efisien dan tidak efektif untuk verifikasi keamanan, karena kehilangan kesempatan untuk meninjau *source code*, mengidentifikasi ancaman dan kontrol yang hilang, serta melakukan pengujian yang jauh lebih menyeluruh dalam jangka waktu yang lebih singkat.

Sangat disarankan untuk melakukan *penetration testing* berbasis dokumentasi atau *source code (hybrid)*, yang memiliki akses penuh ke pengembang aplikasi dan dokumentasi aplikasi, daripada *penetration testing* tradisional. Hal ini tentu sangat diperlukan untuk memverifikasi banyak persyaratan ASVS.

Perubahan Dibandingkan dengan v4.x

Pengantar

Pengguna yang sudah terbiasa dengan versi 4.x dari standar ini mungkin merasa terbantu untuk meninjau perubahan kunci yang diperkenalkan pada versi 5.0, termasuk pembaruan dalam konten, cakupan (*scope*), dan filosofi dasar.

Dari 286 persyaratan dalam versi 4.0.3, hanya 11 yang tetap tidak berubah, sementara 15 telah mengalami penyesuaian tata bahasa minor tanpa mengubah maknanya. Secara total, 109 persyaratan (38%) tidak lagi menjadi persyaratan terpisah di versi 5.0, dengan rincian: 50 persyaratan dihapus, 28 dihapus karena duplikat, dan 31 digabungkan ke dalam persyaratan lain. Sisanya telah direvisi sedemikian rupa. Bahkan persyaratan yang tidak diubah secara substansial memiliki pengenalan (*identifier*) yang berbeda karena adanya pengurutan ulang atau restrukturisasi.

Untuk memfasilitasi adopsi versi 5.0, dokumen pemetaan (*mapping documents*) telah disediakan untuk membantu pengguna menelusuri bagaimana persyaratan dari versi 4.x sesuai dengan persyaratan di versi 5.0. Pemetaan ini tidak terikat pada pemetaan versi rilis dan dapat diperbarui atau diperjelas sesuai kebutuhan.

Filosofi Persyaratan

Cakupan dan Fokus

Versi 4.x mencakup persyaratan yang tidak selaras dengan cakupan yang dimaksudkan dari standar ini; persyaratan tersebut telah dihapus. Persyaratan yang tidak memenuhi kriteria cakupan untuk 5.0 atau tidak dapat diverifikasi (*verifiable*) juga telah dikeluarkannya.

Penekanan pada Tujuan Keamanan Dibandingkan Mekanisme

Dalam versi 4.x, banyak persyaratan berfokus pada mekanisme tertentu alih-alih tujuan keamanan yang mendasarinya. Dalam versi 5.0, persyaratan berpusat pada tujuan keamanan (*security goals*), merujuk pada mekanisme tertentu hanya jika mekanisme tersebut merupakan satu-satunya solusi praktis, atau menyediakannya sebagai contoh atau panduan tambahan.

Pendekatan ini menyadari bahwa beberapa metode mungkin ada untuk mencapai tujuan keamanan tertentu, dan menghindari sifat preskriptif yang tidak perlu yang dapat membatasi fleksibilitas organisasi.

Selain itu, persyaratan yang menangani masalah keamanan yang sama telah dikonsolidasikan jika sesuai.

Keputusan Keamanan yang Terdokumentasi

Meskipun konsep keputusan keamanan yang terdokumentasi mungkin tampak baru di versi 5.0, konsep ini merupakan evolusi dari persyaratan terdahulu terkait penerapan kebijakan dan pemodelan ancaman (*threat modeling*) di versi 4.0. Sebelumnya, beberapa persyaratan secara tersirat menuntut analisis untuk menginformasikan implementasi kontrol keamanan, seperti menentukan koneksi jaringan yang diizinkan.

Untuk memastikan bahwa informasi yang diperlukan tersedia untuk implementasi dan verifikasi, ekspektasi ini sekarang didefinisikan secara eksplisit sebagai persyaratan dokumentasi, menjadikannya jelas, dapat ditindaklanjuti, dan dapat diverifikasi.

Perubahan Struktural dan Bab-Bab Baru

Beberapa bab dalam versi 5.0 memperkenalkan konten yang sama sekali baru:

- OAuth dan OIDC –Meningat adopsi yang luas dari protokol-protokol ini untuk delegasi akses dan *single sign-on*, persyaratan khusus telah ditambahkan untuk menangani berbagai skenario yang mungkin ditemui oleh pengembang. Area ini pada akhirnya mungkin berkembang menjadi standar berdiri sendiri, mirip dengan penanganan persyaratan *Mobile* dan *IoT* pada versi sebelumnya.
- WebRTC –Seiring meningkatnya popularitas teknologi ini, pertimbangan dan tantangan keamanan uniknya kini ditangani dalam bagian khusus.

Upaya juga telah dilakukan untuk memastikan bahwa bab dan bagian diatur di sekitar kumpulan persyaratan terkait yang kohesif.

Restrukturisasi ini telah menyebabkan terciptanya bab-bab tambahan:

- Self-contained Tokens –Sebelumnya dikelompokkan di bawah manajemen sesi (*session management*), *self-contained tokens* kini diakui sebagai mekanisme terpisah dan elemen mendasar untuk komunikasi *stateless* (seperti dalam OAuth dan OIDC). Karena implikasi keamanannya yang unik, topik ini ditangani dalam bab khusus, dengan beberapa persyaratan baru diperkenalkan pada versi 5.x.
- Web Frontend Security –Dengan meningkatnya kompleksitas aplikasi berbasis peramban dan berkembangnya arsitektur *API-only*, persyaratan keamanan *frontend* telah dipisahkan ke dalam bab mereka sendiri.
- Secure Coding and Architecture –Persyaratan baru yang menangani praktik keamanan umum yang tidak cocok di dalam bab-bab yang ada telah dikelompokkan di sini.

Perubahan organisasional lainnya di versi 5.0 dibuat untuk memperjelas maksud. Sebagai contoh, persyaratan validasi input dipindahkan berdampingan dengan logika bisnis, mencerminkan peran mereka dalam menegakkan aturan bisnis, alih-alih dikelompokkan dengan sanitasi dan *encoding*.

Bab V1 Architecture sebelumnya telah dihapus. Bagian awalnya berisi persyaratan yang berada di luar cakupan (*out of scope*), sementara bagian selanjutnya telah didistribusikan ulang ke bab-bab yang relevan, dengan persyaratan yang terduplikasi dan diperjelas seperlunya.

Penghapusan Pemetaan Langsung ke Standar Lain

Pemetaan langsung ke standar lain telah dihapus dari bagian utama standar ini. Tujuannya adalah menyiapkan pemetaan dengan proyek OWASP Common Requirement Enumeration (CRE), yang pada gilirannya akan menghubungkan ASVS ke berbagai proyek OWASP dan standar eksternal.

Pemetaan langsung ke CWE dan NIST tidak lagi dipertahankan, seperti yang dijelaskan di bawah ini.

Pengurangan Keterkaitan dengan NIST Digital Identity Guidelines

NIST Digital Identity Guidelines (SP 800-63) telah lama berfungsi sebagai referensi untuk kontrol otentikasi dan otorisasi. Dalam versi 4.x, bab-bab tertentu sangat selaras dengan struktur dan terminologi NIST.

Meskipun panduan ini tetap menjadi referensi penting, keselarasan yang ketat menimbulkan tantangan, termasuk terminologi yang kurang dikenal secara luas, duplikasi persyaratan serupa, dan pemetaan yang tidak lengkap. Versi 5.0 beralih dari pendekatan ini untuk meningkatkan kejelasan dan relevansi.

Beralih dari Common Weakness Enumeration (CWE)

Common Weakness Enumeration (CWE) menyediakan taksonomi yang berguna tentang kelemahan keamanan perangkat lunak. Namun, tantangan seperti CWE yang hanya berupa kategori, kesulitan dalam memetakan persyaratan ke satu CWE, dan adanya pemetaan yang tidak presisi pada versi 4.x telah menyebabkan keputusan untuk menghentikan pemetaan CWE langsung pada versi 5.0.

Memikirkan Ulang Definisi Level

Versi 4.x menggambarkan tingkat/level sebagai L1 (“Minimum”), L2 (“Standar”), dan L3 (“Lanjutan”), dengan implikasi bahwa semua aplikasi yang menangani data sensitif harus memenuhi setidaknya L2.

Versi 5.0 menangani beberapa masalah dengan pendekatan ini yang dijelaskan dalam paragraf berikut.

Secara praktis, di mana versi 4.x menggunakan tanda centang (*tick marks*) sebagai indikator level, versi 5.x menggunakan angka sederhana pada semua format standar termasuk Markdown, PDF, DOCX, CSV, JSON, dan XML. Untuk kompatibilitas mundur (*backwards compatibility*), output CSV, JSON, dan XML versi *legacy* yang masih menggunakan tanda centang juga tetap dihasilkan.

Level Masuk yang Lebih Mudah

Umpan balik menunjukkan bahwa sejumlah besar persyaratan Level 1 (~120), dikombinasikan dengan sebutan sebagai level “minimum” yang dianggap tidak cukup baik untuk sebagian besar aplikasi, menyurutkan niat untuk melakukan adopsi. Versi 5.0 bertujuan untuk menurunkan hambatan ini dengan mendefinisikan Level 1 utamanya di sekitar persyaratan pertahanan lapisan pertama (*first-layer defense*), yang menghasilkan persyaratan yang lebih jelas dan lebih sedikit pada level tersebut. Untuk menunjukkannya secara numerik, pada v4.0.3 terdapat 128 persyaratan L1 dari total 278 persyaratan, mewakili 46%. Pada 5.0.0 terdapat 70 persyaratan L1 dari total 345 persyaratan, mewakili 20%.

Kekeliruan Pemikiran tentang Kemampuan Pengujian (Testability)

Faktor kunci dalam memilih kontrol untuk Level 1 pada versi 4.x adalah kesesuaian mereka untuk penilaian melalui pengujian penetrasi eksternal *black box*. Namun, pendekatan ini tidak sepenuhnya selaras dengan maksud Level 1 sebagai kumpulan kontrol keamanan minimum. Beberapa pengguna berpendapat bahwa Level 1 tidak cukup untuk mengamankan aplikasi, sementara yang lain menganggapnya terlalu sulit untuk diuji.

Mengandalkan kemampuan pengujian (*testability*) sebagai kriteria adalah hal yang relatif dan terkadang menyesatkan. Fakta bahwa suatu persyaratan dapat diuji tidak menjamin bahwa persyaratan tersebut dapat diuji secara otomatis atau dengan cara yang sederhana. Selain itu, persyaratan yang paling mudah diuji tidak selalu merupakan persyaratan dengan dampak keamanan terbesar atau yang paling mudah diimplementasikan.

Dengan demikian, dalam versi 5.0, keputusan level dibuat terutama berdasarkan pengurangan risiko dan juga mengingat upaya yang diperlukan untuk mengimplementasikannya.

Tidak Hanya Tentang Risiko

Penggunaan level berbasis risiko yang preskriptif yang mewajibkan level tertentu untuk aplikasi tertentu telah terbukti terlalu kaku. Dalam praktiknya, prioritas dan implementasi kontrol keamanan bergantung pada banyak faktor, termasuk pengurangan risiko dan upaya yang diperlukan untuk implementasi.

Oleh karena itu, organisasi didorong untuk mencapai level yang mereka merasa harus dicapai berdasarkan kematangan (*maturity*) mereka dan pesan yang ingin mereka sampaikan kepada pengguna mereka.

V1 Encoding dan Sanitization

Tujuan Kontrol

Bab ini membahas kelemahan keamanan aplikasi web yang paling umum terkait dengan pemrosesan data tidak tepercaya (untrusted data) secara tidak aman. Kelemahan semacam ini dapat mengakibatkan berbagai kerentanan teknis, di mana data tidak tepercaya diinterpretasikan sesuai dengan aturan sintaks dari interpreter yang relevan.

Untuk aplikasi web modern, selalu lebih baik menggunakan API yang lebih aman, seperti parameterized queries, auto-escaping, atau templating frameworks. Jika tidak, output encoding, escaping, atau sanitization yang dilakukan dengan cermat menjadi krusial bagi keamanan aplikasi.

Input validation berfungsi sebagai mekanisme defense-in-depth untuk melindungi dari konten yang tidak terduga atau berbahaya. Namun, karena tujuan utamanya adalah memastikan bahwa konten yang masuk sesuai dengan ekspektasi fungsional dan bisnis, persyaratan terkait hal ini dapat ditemukan pada bab “Validation and Business Logic”.

V1.1 Arsitektur Encoding dan Sanitization

Pada bagian di bawah ini, disediakan persyaratan yang spesifik terhadap sintaks atau interpreter untuk memproses konten tidak aman secara aman guna menghindari kerentanan keamanan. Persyaratan dalam bagian ini mencakup urutan pemrosesan yang harus dilakukan dan di mana pemrosesan tersebut harus terjadi. Persyaratan ini juga bertujuan untuk memastikan bahwa setiap kali data disimpan, data tersebut tetap dalam bentuk aslinya dan tidak disimpan dalam bentuk yang sudah di-encode atau di-escape (misalnya, HTML encoding), untuk mencegah masalah double encoding.

#	Deskripsi	Level
1.1.1	Verifikasi bahwa input di-decode atau di-unescape ke dalam bentuk kanonik hanya satu kali, hanya di-decode ketika data yang di-encode dalam bentuk tersebut memang diharapkan, dan hal ini dilakukan sebelum pemrosesan input lebih lanjut, misalnya tidak dilakukan setelah input validation atau sanitization.	2
1.1.2	Verifikasi bahwa aplikasi melakukan output encoding dan escaping baik sebagai langkah terakhir sebelum digunakan oleh interpreter yang dituju, atau dilakukan oleh interpreter itu sendiri.	2

V1.2 Pencegahan Injection

Output encoding atau escaping, yang dilakukan dekat atau berdekatan dengan konteks yang berpotensi berbahaya, sangat krusial bagi keamanan aplikasi apa pun. Biasanya, output encoding

dan escaping tidak disimpan (persisted), melainkan digunakan untuk membuat output aman untuk digunakan segera pada interpreter yang sesuai. Melakukan hal ini terlalu dini dapat mengakibatkan konten menjadi malformed atau membuat encoding maupun escaping menjadi tidak efektif.

Dalam banyak kasus, pustaka perangkat lunak menyertakan fungsi yang aman atau lebih aman yang melakukan hal ini secara otomatis, meskipun tetap perlu dipastikan bahwa fungsi tersebut sesuai untuk konteks yang digunakan.

#	Deskripsi	Level
1.2.1	Verifikasi bahwa output encoding untuk HTTP response, dokumen HTML, atau dokumen XML relevan dengan konteks yang dibutuhkan, seperti melakukan encoding pada karakter yang relevan untuk elemen HTML, atribut HTML, komentar HTML, CSS, atau HTTP header fields, guna menghindari perubahan pada struktur pesan atau dokumen.	1
1.2.2	Verifikasi bahwa ketika membangun URL secara dinamis, data tidak tepercaya di-encode sesuai dengan konteksnya (misalnya, URL encoding atau base64url encoding untuk parameter query atau path). Pastikan hanya protokol URL yang aman yang diizinkan (misalnya, larang javascript: atau data:).	1
1.2.3	Verifikasi bahwa output encoding atau escaping digunakan saat membangun konten JavaScript secara dinamis (termasuk JSON), guna menghindari perubahan pada struktur pesan atau dokumen (untuk menghindari JavaScript dan JSON injection).	1
1.2.4	Verifikasi bahwa pemilihan data atau query database (misalnya, SQL, HQL, NoSQL, Cypher) menggunakan parameterized queries, ORM, entity frameworks, atau dilindungi dengan cara lain dari SQL Injection dan serangan database injection lainnya. Hal ini juga relevan saat menulis stored procedures.	1
1.2.5	Verifikasi bahwa aplikasi terlindungi dari OS command injection dan bahwa pemanggilan sistem operasi menggunakan parameterized OS queries atau menggunakan contextual command line output encoding.	1
1.2.6	Verifikasi bahwa aplikasi terlindungi dari kerentanan LDAP injection, atau bahwa kontrol keamanan khusus untuk mencegah LDAP injection telah diterapkan.	2
1.2.7	Verifikasi bahwa aplikasi terlindungi dari serangan XPath injection dengan menggunakan query parameterization atau precompiled queries.	2
1.2.8	Verifikasi bahwa LaTeX processors dikonfigurasi secara aman (seperti tidak menggunakan flag “-shell-escape”) dan sebuah allowlist perintah digunakan untuk mencegah serangan LaTeX injection.	2

#	Deskripsi	Level
1.2.9	Verifikasi bahwa aplikasi melakukan escape terhadap karakter khusus dalam regular expressions (biasanya menggunakan backslash) untuk mencegah karakter tersebut disalahartikan sebagai metacharacters.	2
1.2.10	Verifikasi bahwa aplikasi terlindungi dari CSV and Formula Injection. Aplikasi harus mengikuti aturan escaping yang didefinisikan dalam RFC 4180 bagian 2.6 dan 2.7 saat mengeksport konten CSV. Selain itu, saat mengeksport ke CSV atau format spreadsheet lain (seperti XLS, XLSX, atau ODF), karakter khusus (termasuk '=', '+', '-', '@', '\t'(tab), dan '\0'(null character)) harus di-escape dengan tanda kutip tunggal jika muncul sebagai karakter pertama dalam nilai suatu field.	3

Catatan: Menggunakan parameterized queries atau melakukan escaping SQL tidak selalu cukup. Bagian query seperti nama tabel dan nama kolom (termasuk nama kolom pada "ORDER BY") tidak dapat di-escape. Menyertakan data yang berasal dari pengguna dan sudah di-escape pada bagian-bagian ini dapat mengakibatkan query gagal atau terjadinya SQL injection.

V1.3 Sanitization

Perlindungan ideal terhadap penggunaan konten tidak tepercaya dalam konteks yang tidak aman adalah dengan menggunakan context-specific encoding atau escaping, yang mempertahankan makna semantik yang sama dari konten tidak aman tersebut namun membuatnya aman untuk digunakan pada konteks tertentu, sebagaimana dibahas lebih rinci pada bagian sebelumnya.

Jika hal ini tidak memungkinkan, sanitization menjadi diperlukan, yaitu dengan menghapus karakter atau konten yang berpotensi berbahaya. Dalam beberapa kasus, hal ini dapat mengubah makna semantik dari input, namun demi alasan keamanan, mungkin tidak ada alternatif lain.

#	Deskripsi	Level
1.3.1	Verifikasi bahwa semua input HTML tidak tepercaya dari WYSIWYG editor atau sejenisnya di-sanitize menggunakan pustaka atau fitur framework HTML sanitization yang terkenal dan aman.	1
1.3.2	Verifikasi bahwa aplikasi menghindari penggunaan eval() atau fitur dynamic code execution lainnya seperti Spring Expression Language (SpEL). Jika tidak ada alternatif lain, input pengguna yang disertakan harus di-sanitize sebelum dieksekusi.	1

#	Deskripsi	Level
1.3.3	Verifikasi bahwa data yang diteruskan ke konteks yang berpotensi berbahaya di-sanitize terlebih dahulu untuk menerapkan langkah-langkah keamanan, seperti hanya mengizinkan karakter yang aman untuk konteks tersebut dan memotong (trimming) input yang terlalu panjang.	2
1.3.4	Verifikasi bahwa konten Scalable Vector Graphics (SVG) scriptable yang disediakan pengguna divalidasi atau di-sanitize agar hanya berisi tag dan atribut (seperti untuk menggambar grafik) yang aman bagi aplikasi, misalnya, tidak mengandung script dan foreignObject.	2
1.3.5	Verifikasi bahwa aplikasi melakukan sanitize atau menonaktifkan konten scriptable atau expression template language yang disediakan pengguna, seperti Markdown, CSS atau XSL stylesheets, BBCode, atau sejenisnya.	2
1.3.6	Verifikasi bahwa aplikasi terlindungi dari serangan Server-side Request Forgery (SSRF), dengan memvalidasi data tidak tepercaya terhadap allowlist protokol, domain, path, dan port, serta melakukan sanitize terhadap karakter yang berpotensi berbahaya sebelum menggunakan data tersebut untuk memanggil layanan lain.	2
1.3.7	Verifikasi bahwa aplikasi terlindungi dari serangan template injection dengan tidak mengizinkan template dibangun berdasarkan input tidak tepercaya. Jika tidak ada alternatif lain, input tidak tepercaya yang disertakan secara dinamis selama pembuatan template harus di-sanitize atau divalidasi secara ketat.	2
1.3.8	Verifikasi bahwa aplikasi melakukan sanitize secara tepat terhadap input tidak tepercaya sebelum digunakan dalam query Java Naming and Directory Interface (JNDI) dan bahwa JNDI dikonfigurasi secara aman untuk mencegah serangan JNDI injection.	2
1.3.9	Verifikasi bahwa aplikasi melakukan sanitize terhadap konten sebelum dikirim ke memcache untuk mencegah serangan injection.	2
1.3.10	Verifikasi bahwa format strings yang berpotensi diinterpretasikan dengan cara yang tidak terduga atau berbahaya saat digunakan telah di-sanitize sebelum diproses.	2
1.3.11	Verifikasi bahwa aplikasi melakukan sanitize terhadap input pengguna sebelum diteruskan ke sistem mail untuk melindungi dari SMTP atau IMAP injection.	2
1.3.12	Verifikasi bahwa regular expressions bebas dari elemen yang menyebabkan exponential backtracking, dan pastikan input tidak tepercaya di-sanitize untuk memitigasi serangan ReDoS atau Runaway Regex.	3

V1.4 Memory, String, dan Unmanaged Code

Persyaratan berikut membahas risiko yang terkait dengan penggunaan memory yang tidak aman, yang umumnya berlaku ketika aplikasi menggunakan bahasa sistem (systems language) atau unmanaged code.

Dalam beberapa kasus, hal ini dapat dicapai dengan mengatur compiler flags yang mengaktifkan perlindungan dan peringatan terhadap buffer overflow, termasuk stack randomization dan data execution prevention, serta menghentikan proses build jika ditemukan operasi pointer, memory, format string, integer, atau string yang tidak aman.

#	Deskripsi	Level
1.4.1	Verifikasi bahwa aplikasi menggunakan string yang memory-safe, memory copy, dan pointer arithmetic yang lebih aman untuk mendeteksi atau mencegah stack, buffer, atau heap overflow.	2
1.4.2	Verifikasi bahwa teknik validasi sign, range, dan input digunakan untuk mencegah integer overflow.	2
1.4.3	Verifikasi bahwa memory dan resource yang dialokasikan secara dinamis dibebaskan (released), dan bahwa referensi atau pointer ke memory yang telah dibebaskan dihapus atau diatur ke null untuk mencegah dangling pointers dan kerentanan use-after-free.	2

V1.5 Safe Deserialization

Konversi data dari representasi yang disimpan atau ditransmisikan menjadi objek aplikasi yang sesungguhnya (deserialization) secara historis telah menjadi penyebab berbagai kerentanan code injection. Penting untuk melakukan proses ini dengan hati-hati dan aman guna menghindari jenis masalah tersebut.

Secara khusus, metode deserialization tertentu telah diidentifikasi oleh dokumentasi bahasa pemrograman atau framework sebagai tidak aman dan tidak dapat dibuat aman jika digunakan dengan data tidak tepercaya. Untuk setiap mekanisme yang digunakan, uji tuntas (due diligence) yang cermat harus dilakukan.

#	Deskripsi	Level
1.5.1	Verifikasi bahwa aplikasi mengonfigurasi XML parser untuk menggunakan konfigurasi yang restriktif dan bahwa fitur tidak aman seperti resolving external entities dinonaktifkan untuk mencegah serangan XML eXternal Entity (XXE).	1

#	Deskripsi	Level
1.5.2	Verifikasi bahwa deserialization terhadap data tidak tepercaya menerapkan penanganan input yang aman, seperti menggunakan allowlist tipe objek atau membatasi tipe objek yang didefinisikan oleh client, untuk mencegah serangan deserialization. Mekanisme deserialization yang secara eksplisit didefinisikan sebagai tidak aman tidak boleh digunakan dengan input tidak tepercaya.	2
1.5.3	Verifikasi bahwa parser yang berbeda-beda digunakan dalam aplikasi untuk tipe data yang sama (misalnya, JSON parser, XML parser, URL parser), melakukan parsing dengan cara yang konsisten dan menggunakan mekanisme character encoding yang sama untuk menghindari masalah seperti kerentanan JSON Interoperability atau perbedaan perilaku URI atau file parsing yang dapat dieksploitasi dalam serangan Remote File Inclusion (RFI) atau Server-side Request Forgery (SSRF).	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify - Client-side HTML Sanitization Library
- RFC4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files

Untuk informasi lebih lanjut, khususnya mengenai masalah deserialization atau parsing, silakan lihat:

- OWASP Deserialization Cheat Sheet
- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai - A New Era of SSRF Exploiting URL Parser In Trending Programming Languages

V2 Validation dan Business Logic

Tujuan Kontrol

Bab ini bertujuan untuk memastikan bahwa aplikasi yang diverifikasi memenuhi tujuan tingkat tinggi berikut:

- Input yang diterima oleh aplikasi sesuai dengan ekspektasi bisnis atau fungsional.
- Alur business logic bersifat sekuensial, diproses secara berurutan, dan tidak dapat dilewati (bypassed).
- Business logic mencakup batasan dan kontrol untuk mendeteksi serta mencegah serangan otomatis, seperti transfer dana kecil yang dilakukan terus-menerus atau menambahkan satu juta teman satu per satu.
- Alur business logic yang bernilai tinggi telah mempertimbangkan abuse case dan pelaku jahat (malicious actors), serta memiliki perlindungan terhadap serangan spoofing, tampering, information disclosure, dan elevation of privilege.

V2.1 Dokumentasi Validation dan Business Logic

Dokumentasi validation dan business logic harus secara jelas mendefinisikan batasan business logic, aturan validasi, dan konsistensi kontekstual dari kombinasi item data, sehingga jelas apa yang perlu diimplementasikan dalam aplikasi.

#	Deskripsi	Level
2.1.1	Verifikasi bahwa dokumentasi aplikasi mendefinisikan aturan input validation untuk cara memeriksa validitas item data terhadap struktur yang diharapkan. Ini dapat berupa format data umum seperti nomor kartu kredit, alamat email, nomor telepon, atau dapat berupa format data internal.	1
2.1.2	Verifikasi bahwa dokumentasi aplikasi mendefinisikan cara memvalidasi konsistensi logis dan kontekstual dari kombinasi item data, seperti memeriksa bahwa nama kelurahan/kecamatan dan kode pos saling cocok.	2
2.1.3	Verifikasi bahwa ekspektasi untuk batasan dan validasi business logic didokumentasikan, baik per pengguna maupun secara global di seluruh aplikasi.	2

V2.2 Input Validation

Kontrol input validation yang efektif menegakkan ekspektasi bisnis atau fungsional terkait jenis data yang diharapkan diterima oleh aplikasi. Hal ini memastikan kualitas data yang baik dan mengurangi

attack surface. Namun, hal ini tidak menghilangkan atau menggantikan kebutuhan untuk menggunakan encoding, parameterization, atau sanitization yang tepat saat data tersebut digunakan pada komponen lain atau saat disajikan sebagai output.

Dalam konteks ini, “input” dapat berasal dari berbagai sumber, termasuk HTML form fields, REST requests, parameter URL, HTTP header fields, cookies, file pada disk, database, dan API eksternal.

Sebuah kontrol business logic dapat memeriksa bahwa input tertentu adalah angka kurang dari 100. Sebuah ekspektasi fungsional dapat memeriksa bahwa suatu angka berada di bawah ambang batas tertentu, karena angka tersebut mengontrol berapa kali sebuah loop tertentu akan dijalankan, dan angka yang tinggi dapat mengakibatkan pemrosesan berlebihan serta potensi kondisi denial of service.

Meskipun schema validation tidak diwajibkan secara eksplisit, ini bisa menjadi mekanisme paling efektif untuk cakupan validasi penuh terhadap HTTP API atau interface lain yang menggunakan JSON atau XML.

Perhatikan poin-poin berikut mengenai Schema Validation:

- “Published version” dari spesifikasi JSON Schema validation dianggap sudah siap untuk produksi (production-ready), namun secara ketat belum bisa dikatakan “stabil”. Saat menggunakan JSON Schema validation, pastikan tidak ada celah dengan panduan pada persyaratan di bawah ini.
- Pustaka JSON Schema validation apa pun yang digunakan juga harus dipantau dan diperbarui jika diperlukan setelah standar tersebut diresmikan.
- DTD validation tidak boleh digunakan, dan evaluasi DTD pada framework harus dinonaktifkan, guna menghindari masalah serangan XXE terhadap DTD.

#	Deskripsi	Level
2.2.1	Verifikasi bahwa input divalidasi untuk menegakkan ekspektasi bisnis atau fungsional terhadap input tersebut. Hal ini harus menggunakan positive validation terhadap allow list nilai, pola, dan rentang, atau berdasarkan perbandingan input dengan struktur yang diharapkan dan batasan logis sesuai aturan yang telah ditentukan sebelumnya. Untuk L1, hal ini dapat difokuskan pada input yang digunakan untuk membuat keputusan bisnis atau keamanan tertentu. Untuk L2 ke atas, hal ini harus berlaku untuk semua input.	1
2.2.2	Verifikasi bahwa aplikasi dirancang untuk menegakkan input validation pada trusted service layer. Meskipun client-side validation meningkatkan usability dan harus didorong penggunaannya, hal ini tidak boleh diandalkan sebagai kontrol keamanan.	1

#	Deskripsi	Level
2.2.3	Verifikasi bahwa aplikasi memastikan kombinasi item data yang saling terkait masuk akal (reasonable) sesuai dengan aturan yang telah ditentukan sebelumnya.	2

V2.3 Keamanan Business Logic

Bagian ini membahas persyaratan utama untuk memastikan bahwa aplikasi menegakkan proses business logic dengan cara yang benar dan tidak rentan terhadap serangan yang mengeksploitasi logika dan alur aplikasi.

#	Deskripsi	Level
2.3.1	Verifikasi bahwa aplikasi hanya akan memproses alur business logic untuk pengguna yang sama sesuai urutan langkah sekuensial yang diharapkan dan tanpa melewati langkah apa pun.	1
2.3.2	Verifikasi bahwa batasan business logic diimplementasikan sesuai dokumentasi aplikasi untuk menghindari kelemahan business logic dieksploitasi.	2
2.3.3	Verifikasi bahwa transactions digunakan pada level business logic sedemikian rupa sehingga sebuah operasi business logic berhasil dilakukan secara keseluruhan atau dikembalikan (rolled back) ke keadaan benar sebelumnya.	2
2.3.4	Verifikasi bahwa mekanisme locking pada level business logic digunakan untuk memastikan resource dengan kuantitas terbatas (seperti kursi teater atau slot pengiriman) tidak dapat dipesan ganda (double-booked) dengan memanipulasi logika aplikasi.	2
2.3.5	Verifikasi bahwa alur business logic yang bernilai tinggi memerlukan persetujuan multi-user (multi-user approval) untuk mencegah tindakan yang tidak sah atau tidak disengaja. Hal ini dapat mencakup, namun tidak terbatas pada, transfer uang dalam jumlah besar, persetujuan kontrak, akses ke informasi rahasia, atau safety override dalam proses manufaktur.	3

V2.4 Anti-automation

Bagian ini mencakup kontrol anti-automation untuk memastikan bahwa interaksi yang menyerupai interaksi manusia diperlukan dan permintaan otomatis yang berlebihan dapat dicegah.

#	Deskripsi	Level
2.4.1	Verifikasi bahwa kontrol anti-automation diterapkan untuk melindungi dari pemanggilan fungsi aplikasi secara berlebihan yang dapat menyebabkan data exfiltration, pembuatan data sampah (garbage-data), habisnya quota, pelanggaran rate-limit, denial-of-service, atau penggunaan berlebihan terhadap resource yang mahal.	2
2.4.2	Verifikasi bahwa alur business logic memerlukan timing yang realistis layaknya manusia, guna mencegah pengiriman transaksi yang berlangsung terlalu cepat.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing
- Anti-automation dapat dicapai dengan berbagai cara, termasuk penggunaan OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 Web Frontend Security

Tujuan Kontrol

Kategori ini berfokus pada persyaratan yang dirancang untuk melindungi dari serangan yang dilakukan melalui web frontend. Persyaratan ini tidak berlaku untuk solusi machine-to-machine.

V3.1 Dokumentasi Web Frontend Security

Bagian ini menjelaskan fitur keamanan browser yang harus ditentukan dalam dokumentasi aplikasi.

#	Deskripsi	Level
3.1.1	Verifikasi bahwa dokumentasi aplikasi menyatakan fitur keamanan yang diharapkan didukung oleh browser yang digunakan untuk mengakses aplikasi (seperti HTTPS, HTTP Strict Transport Security (HSTS), Content Security Policy (CSP), dan mekanisme keamanan HTTP relevan lainnya). Dokumentasi tersebut juga harus mendefinisikan bagaimana aplikasi harus berperilaku jika beberapa fitur ini tidak tersedia (seperti memberi peringatan kepada pengguna atau memblokir akses).	3

V3.2 Interpretasi Konten yang Tidak Diinginkan

Me-render konten atau fungsionalitas dalam konteks yang salah dapat mengakibatkan konten berbahaya dieksekusi atau ditampilkan.

#	Deskripsi	Level
3.2.1	Verifikasi bahwa kontrol keamanan diterapkan untuk mencegah browser me-render konten atau fungsionalitas dalam HTTP responses pada konteks yang salah (misalnya, ketika sebuah API, file yang diunggah pengguna, atau resource lain diminta secara langsung). Kontrol yang mungkin dapat mencakup: tidak menyajikan konten kecuali HTTP request header fields (seperti Sec-Fetch-*) menunjukkan bahwa itu adalah konteks yang benar, menggunakan sandbox directive pada Content-Security-Policy header field, atau menggunakan attachment disposition type pada Content-Disposition header field.	1
3.2.2	Verifikasi bahwa konten yang dimaksudkan untuk ditampilkan sebagai teks, bukan di-render sebagai HTML, ditangani menggunakan fungsi rendering yang aman (seperti <code>createTextNode</code> atau <code>textContent</code>) untuk mencegah eksekusi konten yang tidak diinginkan seperti HTML atau JavaScript.	1
3.2.3	Verifikasi bahwa aplikasi menghindari DOM clobbering ketika menggunakan client-side JavaScript dengan menerapkan deklarasi variabel secara eksplisit, melakukan strict type checking, menghindari penyimpanan variabel global pada objek document, dan menerapkan namespace isolation.	3

V3.3 Konfigurasi Cookie

Bagian ini menjelaskan persyaratan untuk mengonfigurasi cookie yang sensitif secara aman guna memberikan tingkat jaminan yang lebih tinggi bahwa cookie tersebut dibuat oleh aplikasi itu sendiri,

serta mencegah isinya bocor atau dimodifikasi secara tidak sah.

#	Deskripsi	Level
3.3.1	Verifikasi bahwa cookie memiliki atribut 'Secure' yang diatur, dan jika prefix '___Host-' tidak digunakan untuk nama cookie, maka prefix '___Secure-' harus digunakan untuk nama cookie tersebut.	1
3.3.2	Verifikasi bahwa nilai atribut 'SameSite' pada setiap cookie diatur sesuai dengan tujuan cookie tersebut, untuk membatasi paparan terhadap user interface redress attacks dan browser-based request forgery attacks, yang umum dikenal sebagai cross-site request forgery (CSRF).	2
3.3.3	Verifikasi bahwa cookie memiliki prefix '___Host-' untuk nama cookie, kecuali jika cookie tersebut memang dirancang secara eksplisit untuk dibagikan dengan host lain.	2
3.3.4	Verifikasi bahwa jika nilai suatu cookie tidak dimaksudkan untuk dapat diakses oleh client-side scripts (seperti session token), cookie tersebut harus memiliki atribut 'HttpOnly' yang diatur dan nilai yang sama (misalnya session token) hanya boleh dikirimkan ke client melalui 'Set-Cookie' header field.	2
3.3.5	Verifikasi bahwa saat aplikasi menulis sebuah cookie, panjang gabungan nama dan nilai cookie tidak melebihi 4096 byte. Cookie yang terlalu besar tidak akan disimpan oleh browser dan karenanya tidak akan dikirimkan bersama request, sehingga mencegah pengguna menggunakan fungsionalitas aplikasi yang bergantung pada cookie tersebut.	3

V3.4 Header Mekanisme Keamanan Browser

Bagian ini menjelaskan header keamanan mana yang harus diatur pada HTTP responses untuk mengaktifkan fitur keamanan dan pembatasan browser saat menangani response dari aplikasi.

#	Deskripsi	Level
3.4.1	Verifikasi bahwa Strict-Transport-Security header field disertakan pada semua response untuk menegakkan kebijakan HTTP Strict Transport Security (HSTS). Sebuah maximum age minimal 1 tahun harus didefinisikan, dan untuk L2 ke atas, kebijakan tersebut juga harus berlaku untuk semua subdomain.	1

#	Deskripsi	Level
3.4.2	Verifikasi bahwa Cross-Origin Resource Sharing (CORS) Access-Control-Allow-Origin header field bernilai tetap (fixed value) yang ditentukan oleh aplikasi, atau jika nilai Origin HTTP request header field digunakan, nilai tersebut divalidasi terhadap allowlist origin tepercaya. Ketika 'Access-Control-Allow-Origin: *' perlu digunakan, verifikasi bahwa response tersebut tidak menyertakan informasi sensitif apa pun.	1
3.4.3	Verifikasi bahwa HTTP responses menyertakan Content-Security-Policy response header field yang mendefinisikan directives untuk memastikan browser hanya memuat dan menjalankan konten atau resource yang tepercaya, guna membatasi eksekusi JavaScript berbahaya. Minimal, sebuah kebijakan global harus digunakan yang mencakup directives object-src 'none' dan base-uri 'none' serta mendefinisikan allowlist atau menggunakan nonces atau hashes. Untuk aplikasi L3, sebuah kebijakan per-response dengan nonces atau hashes harus didefinisikan.	2
3.4.4	Verifikasi bahwa semua HTTP responses mengandung 'X-Content-Type-Options: nosniff' header field. Hal ini menginstruksikan browser untuk tidak menggunakan content sniffing dan MIME type guessing terhadap response tertentu, dan mewajibkan nilai Content-Type header field pada response tersebut sesuai dengan resource tujuan. Misalnya, response terhadap sebuah permintaan style hanya diterima jika Content-Type pada response tersebut adalah 'text/css'. Hal ini juga mengaktifkan penggunaan fungsionalitas Cross-Origin Read Blocking (CORB) oleh browser.	2
3.4.5	Verifikasi bahwa aplikasi mengatur sebuah referrer policy untuk mencegah kebocoran data teknis yang sensitif ke layanan pihak ketiga melalui 'Referer' HTTP request header field. Hal ini dapat dilakukan dengan menggunakan Referrer-Policy HTTP response header field atau melalui atribut elemen HTML. Data sensitif dapat mencakup data path dan query pada URL, dan untuk aplikasi internal non-publik juga termasuk hostname.	2
3.4.6	Verifikasi bahwa aplikasi web menggunakan frame-ancestors directive pada Content-Security-Policy header field untuk setiap HTTP response guna memastikan aplikasi tidak dapat di-embed secara default dan bahwa embedding resource tertentu hanya diizinkan jika diperlukan. Perlu dicatat bahwa X-Frame-Options header field, meskipun masih didukung oleh browser, sudah usang (obsolete) dan tidak boleh diandalkan.	2
3.4.7	Verifikasi bahwa Content-Security-Policy header field menentukan lokasi untuk melaporkan pelanggaran (violations).	3

#	Deskripsi	Level
3.4.8	Verifikasi bahwa semua HTTP responses yang memicu rendering dokumen (seperti response dengan Content-Type text/html), menyertakan Cross-Origin-Opener-Policy header field dengan directive same-origin atau directive same-origin-allow-popups sesuai kebutuhan. Hal ini mencegah serangan yang menyalahgunakan shared access terhadap Window objects, seperti tabnabbing dan frame counting.	3

V3.5 Pemisahan Origin Browser

Saat menerima permintaan terhadap fungsionalitas sensitif pada sisi server, aplikasi perlu memastikan bahwa permintaan tersebut diinisiasi oleh aplikasi itu sendiri atau oleh pihak tepercaya dan bukan dipalsukan (forged) oleh penyerang.

Fungsionalitas sensitif dalam konteks ini dapat mencakup penerimaan form posts untuk pengguna yang telah maupun belum terautentikasi (seperti permintaan autentikasi), operasi yang mengubah state, atau fungsionalitas yang memerlukan resource besar (seperti data export).

Perlindungan utama di sini adalah kebijakan keamanan browser seperti Same Origin Policy untuk JavaScript dan juga logika SameSite untuk cookie. Perlindungan umum lainnya adalah mekanisme CORS preflight. Mekanisme ini sangat penting untuk endpoint yang dirancang untuk dipanggil dari origin yang berbeda, namun juga dapat menjadi mekanisme pencegahan request forgery yang berguna untuk endpoint yang tidak dirancang untuk dipanggil dari origin yang berbeda.

#	Deskripsi	Level
3.5.1	Verifikasi bahwa, jika aplikasi tidak mengandalkan mekanisme CORS preflight untuk mencegah cross-origin requests yang tidak diizinkan menggunakan fungsionalitas sensitif, request tersebut divalidasi untuk memastikan bahwa request tersebut berasal dari aplikasi itu sendiri. Hal ini dapat dilakukan dengan menggunakan dan memvalidasi anti-forgery tokens atau mewajibkan HTTP header fields tambahan yang bukan merupakan CORS-safelisted request-header fields. Hal ini bertujuan untuk melindungi dari browser-based request forgery attacks, yang umum dikenal sebagai cross-site request forgery (CSRF).	1

#	Deskripsi	Level
3.5.2	Verifikasi bahwa, jika aplikasi mengandalkan mekanisme CORS preflight untuk mencegah penggunaan cross-origin yang tidak diizinkan terhadap fungsionalitas sensitif, tidak mungkin untuk memanggil fungsionalitas tersebut dengan request yang tidak memicu CORS-preflight request. Hal ini mungkin memerlukan pemeriksaan nilai 'Origin' dan 'Content-Type' request header fields atau menggunakan header field tambahan yang bukan merupakan CORS-safelisted header-field.	1
3.5.3	Verifikasi bahwa HTTP requests terhadap fungsionalitas sensitif menggunakan metode HTTP yang sesuai seperti POST, PUT, PATCH, atau DELETE, dan bukan metode yang didefinisikan oleh spesifikasi HTTP sebagai "safe" seperti HEAD, OPTIONS, atau GET. Sebagai alternatif, validasi ketat terhadap Sec-Fetch-* request header fields dapat digunakan untuk memastikan bahwa request tidak berasal dari pemanggilan cross-origin yang tidak sesuai, navigation request, atau resource load (seperti image source) yang seharusnya tidak diharapkan.	1
3.5.4	Verifikasi bahwa aplikasi yang terpisah di-hosting pada hostname yang berbeda untuk memanfaatkan pembatasan yang disediakan oleh same-origin policy, termasuk bagaimana dokumen atau script yang dimuat oleh satu origin dapat berinteraksi dengan resource dari origin dan hostname lain serta pembatasan cookie berbasis hostname.	2
3.5.5	Verifikasi bahwa pesan yang diterima oleh interface postMessage dibuang (discarded) jika origin dari pesan tersebut tidak tepercaya, atau jika sintaks pesan tersebut tidak valid.	2
3.5.6	Verifikasi bahwa fungsionalitas JSONP tidak diaktifkan di bagian mana pun dalam aplikasi untuk menghindari serangan Cross-Site Script Inclusion (XSSI).	3
3.5.7	Verifikasi bahwa data yang memerlukan otorisasi tidak disertakan dalam response resource script, seperti file JavaScript, untuk mencegah serangan Cross-Site Script Inclusion (XSSI).	3

#	Deskripsi	Level
3.5.8	Verifikasi bahwa resource yang memerlukan autentikasi (seperti gambar, video, script, dan dokumen lainnya) hanya dapat dimuat atau di-embed atas nama pengguna ketika memang dimaksudkan demikian. Hal ini dapat dicapai dengan validasi ketat terhadap Sec-Fetch-* HTTP request header fields untuk memastikan bahwa request tidak berasal dari pemanggilan cross-origin yang tidak sesuai, atau dengan mengatur Cross-Origin-Resource-Policy HTTP response header field yang bersifat restriktif untuk menginstruksikan browser memblokir konten yang dikembalikan.	3

V3.6 Integritas Resource Eksternal

Bagian ini memberikan panduan untuk hosting konten yang aman pada situs pihak ketiga.

#	Deskripsi	Level
3.6.1	Verifikasi bahwa client-side assets, seperti pustaka JavaScript, CSS, atau web fonts, hanya di-hosting secara eksternal (misalnya, pada sebuah Content Delivery Network) jika resource tersebut bersifat statis dan memiliki versi (versioned) serta Subresource Integrity (SRI) digunakan untuk memvalidasi integritas asset tersebut. Jika hal ini tidak memungkinkan, harus ada keputusan keamanan yang terdokumentasi untuk menjustifikasi hal tersebut pada setiap resource.	3

V3.7 Pertimbangan Keamanan Browser Lainnya

Bagian ini mencakup berbagai kontrol keamanan lain dan fitur keamanan browser modern yang diperlukan untuk keamanan browser sisi client.

#	Deskripsi	Level
3.7.1	Verifikasi bahwa aplikasi hanya menggunakan teknologi client-side yang masih didukung dan dianggap aman. Contoh teknologi yang tidak memenuhi persyaratan ini antara lain NSAPI plugins, Flash, Shockwave, ActiveX, Silverlight, NACL, atau client-side Java applets.	2

#	Deskripsi	Level
3.7.2	Verifikasi bahwa aplikasi hanya akan secara otomatis mengarahkan (redirect) pengguna ke hostname atau domain yang berbeda (yang tidak dikendalikan oleh aplikasi) apabila tujuan tersebut terdapat pada sebuah allowlist.	2
3.7.3	Verifikasi bahwa aplikasi menampilkan notifikasi ketika pengguna diarahkan (redirect) ke URL di luar kendali aplikasi, dengan opsi untuk membatalkan navigasi tersebut.	3
3.7.4	Verifikasi bahwa top-level domain aplikasi (misalnya, site.tld) ditambahkan ke public preload list untuk HTTP Strict Transport Security (HSTS). Hal ini memastikan bahwa penggunaan TLS untuk aplikasi tertanam langsung pada browser-browser utama, alih-alih hanya mengandalkan Strict-Transport-Security response header field.	3
3.7.5	Verifikasi bahwa aplikasi berperilaku sesuai dengan yang didokumentasikan (seperti memberi peringatan kepada pengguna atau memblokir akses) jika browser yang digunakan untuk mengakses aplikasi tidak mendukung fitur keamanan yang diharapkan.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- Set-Cookie __Host- prefix details
- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API dan Web Service

Tujuan Kontrol

Beberapa pertimbangan berlaku secara khusus untuk aplikasi yang mengekspos API untuk digunakan oleh web browser atau consumer lainnya (umumnya menggunakan JSON, XML, atau GraphQL). Bab ini membahas konfigurasi dan mekanisme keamanan relevan yang harus diterapkan.

Perlu dicatat bahwa hal-hal terkait authentication, session management, dan input validation dari bab lain juga berlaku untuk API, sehingga bab ini tidak dapat dipisahkan dari konteksnya atau diuji secara terpisah.

V4.1 Keamanan Web Service Umum

Bagian ini membahas pertimbangan keamanan web service secara umum dan, oleh karena itu, praktik-praktik dasar kebersihan (hygiene) web service.

#	Deskripsi	Level
4.1.1	Verifikasi bahwa setiap HTTP response dengan message body memiliki Content-Type header field yang sesuai dengan konten sebenarnya dari response tersebut, termasuk parameter charset untuk menentukan character encoding yang aman (misalnya, UTF-8, ISO-8859-1) sesuai dengan IANA Media Types, seperti “text/”, “/+xml” dan “/xml”.	1
4.1.2	Verifikasi bahwa hanya endpoint yang menghadap pengguna (user-facing, dimaksudkan untuk akses manual melalui web browser) yang secara otomatis melakukan redirect dari HTTP ke HTTPS, sedangkan layanan atau endpoint lainnya tidak menerapkan transparent redirects. Hal ini untuk menghindari situasi di mana client secara keliru mengirimkan HTTP request yang tidak terenkripsi, namun karena request tersebut secara otomatis diarahkan ke HTTPS, kebocoran data sensitif menjadi tidak terdeteksi.	2
4.1.3	Verifikasi bahwa HTTP header field apa pun yang digunakan oleh aplikasi dan diatur oleh lapisan perantara (intermediary layer), seperti load balancer, web proxy, atau layanan backend-for-frontend, tidak dapat di-override oleh pengguna akhir. Contoh header dapat mencakup X-Real-IP, X-Forwarded-*, atau X-User-ID.	2
4.1.4	Verifikasi bahwa hanya metode HTTP yang secara eksplisit didukung oleh aplikasi atau API-nya (termasuk OPTIONS selama preflight requests) yang dapat digunakan dan metode yang tidak digunakan diblokir.	3
4.1.5	Verifikasi bahwa per-message digital signatures digunakan untuk memberikan jaminan tambahan di atas perlindungan transport untuk request atau transaksi yang sangat sensitif atau yang melintasi sejumlah sistem.	3

V4.2 Validasi Struktur Pesan HTTP

Bagian ini menjelaskan bagaimana struktur dan header fields dari sebuah pesan HTTP harus divalidasi untuk mencegah serangan seperti request smuggling, response splitting, header injection, dan

denial of service melalui pesan HTTP yang terlalu panjang.

Persyaratan ini relevan untuk pemrosesan dan pembuatan pesan HTTP secara umum, namun menjadi sangat penting terutama saat mengonversi pesan HTTP antar versi HTTP yang berbeda.

#	Deskripsi	Level
4.2.1	Verifikasi bahwa semua komponen aplikasi (termasuk load balancer, firewall, dan application server) menentukan batasan pesan HTTP yang masuk menggunakan mekanisme yang sesuai untuk versi HTTP tersebut guna mencegah HTTP request smuggling. Pada HTTP/1.x, jika sebuah Transfer-Encoding header field ada, Content-Length header harus diabaikan sesuai RFC 2616. Saat menggunakan HTTP/2 atau HTTP/3, jika sebuah Content-Length header field ada, penerima harus memastikan bahwa nilai tersebut konsisten dengan panjang DATA frames.	2
4.2.2	Verifikasi bahwa saat membuat pesan HTTP, Content-Length header field tidak bertentangan dengan panjang konten sebagaimana ditentukan oleh framing protokol HTTP, guna mencegah serangan request smuggling.	3
4.2.3	Verifikasi bahwa aplikasi tidak mengirim maupun menerima pesan HTTP/2 atau HTTP/3 dengan connection-specific header fields seperti Transfer-Encoding guna mencegah serangan response splitting dan header injection.	3
4.2.4	Verifikasi bahwa aplikasi hanya menerima request HTTP/2 dan HTTP/3 yang header fields dan nilainya tidak mengandung urutan CR (\r), LF (\n), atau CRLF (\r\n), guna mencegah serangan header injection.	3
4.2.5	Verifikasi bahwa, jika aplikasi (backend atau frontend) membangun dan mengirim request, aplikasi tersebut menggunakan validation, sanitization, atau mekanisme lain untuk menghindari pembuatan URI (seperti untuk pemanggilan API) atau HTTP request header fields (seperti Authorization atau Cookie) yang terlalu panjang untuk diterima oleh komponen penerima. Hal ini dapat menyebabkan denial of service, seperti saat mengirimkan request yang terlalu panjang (misalnya, cookie header field yang panjang), yang mengakibatkan server selalu merespons dengan status error.	3

V4.3 GraphQL

GraphQL semakin umum digunakan sebagai cara untuk membuat client yang kaya data (data-rich) yang tidak terikat erat dengan berbagai layanan backend. Bagian ini membahas pertimbangan keamanan untuk GraphQL.

#	Deskripsi	Level
4.3.1	Verifikasi bahwa query allowlist, depth limiting, amount limiting, atau query cost analysis digunakan untuk mencegah GraphQL atau data layer expression Denial of Service (DoS) akibat query yang mahal (expensive) dan bersarang (nested).	2
4.3.2	Verifikasi bahwa GraphQL introspection queries dinonaktifkan pada environment produksi, kecuali jika GraphQL API tersebut memang dimaksudkan untuk digunakan oleh pihak lain.	2

V4.4 WebSocket

WebSocket adalah protokol komunikasi yang menyediakan saluran komunikasi dua arah secara simultan melalui satu koneksi TCP tunggal. Protokol ini distandarkan oleh IETF sebagai RFC 6455 pada tahun 2011 dan berbeda dari HTTP, meskipun dirancang untuk bekerja melalui port HTTP 443 dan 80.

Bagian ini menyediakan persyaratan keamanan utama untuk mencegah serangan yang terkait dengan keamanan komunikasi dan session management yang secara khusus mengeksploitasi saluran komunikasi real-time ini.

#	Deskripsi	Level
4.4.1	Verifikasi bahwa WebSocket over TLS (WSS) digunakan untuk semua koneksi WebSocket.	1
4.4.2	Verifikasi bahwa, selama initial HTTP WebSocket handshake, Origin header field diperiksa terhadap daftar origin yang diizinkan untuk aplikasi tersebut.	2
4.4.3	Verifikasi bahwa, jika session management standar aplikasi tidak dapat digunakan, token khusus (dedicated tokens) digunakan untuk hal ini, yang sesuai dengan persyaratan keamanan Session Management yang relevan.	2
4.4.4	Verifikasi bahwa token session management WebSocket khusus pada awalnya diperoleh atau divalidasi melalui sesi HTTPS yang telah terautentikasi sebelumnya, saat mengalihkan sesi HTTPS yang ada ke saluran WebSocket.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP REST Security Cheat Sheet

- Sumber daya mengenai GraphQL Authorization dari graphql.org dan Apollo.
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 File Handling

Tujuan Kontrol

Penggunaan file dapat menghadirkan berbagai risiko bagi aplikasi, termasuk denial of service, akses tidak sah, dan habisnya kapasitas penyimpanan (storage exhaustion). Bab ini mencakup persyaratan untuk mengatasi risiko-risiko tersebut.

V5.1 Dokumentasi Penanganan File

Bagian ini mencakup persyaratan untuk mendokumentasikan karakteristik file yang diharapkan diterima oleh aplikasi, sebagai prasyarat yang diperlukan untuk mengembangkan dan memverifikasi pemeriksaan keamanan yang relevan.

#	Deskripsi	Level
5.1.1	Verifikasi bahwa dokumentasi mendefinisikan jenis file yang diizinkan, ekstensi file yang diharapkan, dan ukuran maksimum (termasuk ukuran setelah unpacked) untuk setiap fitur upload. Selain itu, pastikan dokumentasi tersebut menentukan bagaimana file dibuat aman bagi pengguna akhir untuk diunduh dan diproses, seperti bagaimana perilaku aplikasi ketika sebuah file berbahaya terdeteksi.	2

V5.2 Upload File dan Konten

Fungsionalitas upload file merupakan sumber utama file yang tidak tepercaya. Bagian ini menjelaskan persyaratan untuk memastikan bahwa keberadaan, volume, atau konten dari file-file tersebut tidak dapat membahayakan aplikasi.

#	Deskripsi	Level
5.2.1	Verifikasi bahwa aplikasi hanya akan menerima file dengan ukuran yang dapat diproses tanpa menyebabkan penurunan performa atau serangan denial of service.	1

#	Deskripsi	Level
5.2.2	Verifikasi bahwa saat aplikasi menerima sebuah file, baik secara mandiri maupun di dalam sebuah archive seperti file zip, aplikasi memeriksa apakah ekstensi file sesuai dengan ekstensi file yang diharapkan dan memvalidasi bahwa isi file tersebut sesuai dengan tipe yang direpresentasikan oleh ekstensi tersebut. Hal ini mencakup, namun tidak terbatas pada, pemeriksaan 'magic bytes' awal, melakukan image re-writing, dan menggunakan pustaka khusus untuk validasi konten file. Untuk L1, hal ini dapat difokuskan hanya pada file yang digunakan untuk membuat keputusan bisnis atau keamanan tertentu. Untuk L2 ke atas, hal ini harus berlaku untuk semua file yang diterima.	1
5.2.3	Verifikasi bahwa aplikasi memeriksa file terkompresi (misalnya, zip, gz, docx, odt) terhadap ukuran maksimum yang diizinkan setelah dekompresi (uncompressed) dan terhadap jumlah maksimum file sebelum melakukan dekompresi file tersebut.	2
5.2.4	Verifikasi bahwa kuota ukuran file dan jumlah maksimum file per pengguna diterapkan untuk memastikan bahwa satu pengguna tidak dapat memenuhi storage dengan terlalu banyak file, atau file yang berukuran sangat besar.	3
5.2.5	Verifikasi bahwa aplikasi tidak mengizinkan pengunggahan file terkompresi yang berisi symlink kecuali hal ini memang secara khusus diperlukan (dalam hal ini perlu diterapkan allowlist untuk file-file yang dapat di-symlink).	3
5.2.6	Verifikasi bahwa aplikasi menolak gambar yang diunggah dengan ukuran pixel lebih besar dari maksimum yang diizinkan, untuk mencegah serangan pixel flood.	3

V5.3 Penyimpanan File

Bagian ini mencakup persyaratan untuk mencegah file dieksekusi secara tidak sah setelah diunggah, untuk mendeteksi konten berbahaya, dan untuk menghindari data tidak tepercaya digunakan untuk mengontrol di mana file disimpan.

#	Deskripsi	Level
5.3.1	Verifikasi bahwa file yang diunggah atau dihasilkan dari input tidak tepercaya dan disimpan pada folder publik, tidak dieksekusi sebagai kode program server-side ketika diakses secara langsung melalui HTTP request.	1

#	Deskripsi	Level
5.3.2	Verifikasi bahwa saat aplikasi membuat file paths untuk operasi file, aplikasi menggunakan data yang dihasilkan secara internal atau data tepercaya, bukan nama file yang dikirimkan oleh pengguna; atau jika nama file atau metadata file yang dikirimkan pengguna harus digunakan, validasi dan sanitization yang ketat harus diterapkan. Hal ini bertujuan untuk melindungi dari serangan path traversal, local atau remote file inclusion (LFI, RFI), dan server-side request forgery (SSRF).	1
5.3.3	Verifikasi bahwa pemrosesan file di sisi server, seperti dekompresi file, mengabaikan informasi path yang disediakan oleh pengguna untuk mencegah kerentanan seperti zip slip.	3

V5.4 Download File

Bagian ini berisi persyaratan untuk memitigasi risiko saat menyajikan file untuk diunduh, termasuk serangan path traversal dan injection. Hal ini juga mencakup memastikan bahwa file tersebut tidak mengandung konten berbahaya.

#	Deskripsi	Level
5.4.1	Verifikasi bahwa aplikasi memvalidasi atau mengabaikan nama file yang dikirimkan oleh pengguna, termasuk pada JSON, JSONP, atau parameter URL, dan menentukan sebuah nama file pada Content-Disposition header field dalam response.	2
5.4.2	Verifikasi bahwa nama file yang disajikan (misalnya, pada HTTP response header fields atau lampiran email) di-encode atau di-sanitize (misalnya, mengikuti RFC 6266) untuk menjaga struktur dokumen dan mencegah serangan injection.	2
5.4.3	Verifikasi bahwa file yang diperoleh dari sumber tidak tepercaya dipindai oleh antivirus scanner untuk mencegah penyajian konten berbahaya yang telah dikenal.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP File Upload Cheat Sheet
- Contoh penggunaan symlink untuk pembacaan file secara sewenang-wenang (arbitrary file read)

- Penjelasan mengenai “Magic Bytes” dari Wikipedia

V6 Authentication

Objektif Kontrol

Authentication adalah proses menetapkan atau mengonfirmasi keaslian suatu individu atau perangkat [cite: 1]. Ini melibatkan verifikasi klaim yang dibuat oleh seseorang atau tentang sebuah perangkat, memastikan ketahanan terhadap peniruan (impersonation), dan mencegah pemulihan atau pencetakan sandi (password) [cite: 1].

NIST SP 800-63 adalah standar modern berbasis bukti yang berharga bagi organisasi di seluruh dunia, tetapi secara khusus relevan bagi agensi AS dan mereka yang berinteraksi dengan agensi AS [cite: 1].

Meskipun banyak dari persyaratan dalam bab ini didasarkan pada bagian kedua dari standar tersebut (dikenal sebagai NIST SP 800-63B “Digital Identity Guidelines - Authentication and Lifecycle Management”), bab ini berfokus pada ancaman umum dan kelemahan Authentication yang sering dieksploitasi [cite: 1]. Bab ini tidak mencoba untuk secara komprehensif mencakup setiap poin dalam standar [cite: 1]. Untuk kasus di mana kepatuhan penuh terhadap NIST SP 800-63 diperlukan, silakan merujuk ke NIST SP 800-63 [cite: 1].

Selain itu, terminologi NIST SP 800-63 terkadang dapat berbeda, dan bab ini sering menggunakan terminologi yang lebih umum dipahami untuk meningkatkan kejelasan [cite: 1].

Fitur umum dari aplikasi yang lebih canggih adalah kemampuan untuk mengadaptasi tahapan Authentication yang diperlukan berdasarkan berbagai faktor risiko [cite: 1]. Fitur ini dibahas dalam bab “Otorisasi”(Authorization), karena mekanisme ini juga perlu dipertimbangkan untuk keputusan otorisasi [cite: 1].

V6.1 Dokumentasi Authentication

Bagian ini berisi persyaratan yang merinci dokumentasi Authentication yang harus dikelola untuk sebuah aplikasi [cite: 1]. Hal ini sangat penting untuk mengimplementasikan dan menilai bagaimana kontrol Authentication yang relevan harus dikonfigurasi [cite: 1].

#	Deskripsi	Level
6.1.1	Pastikan bahwa dokumentasi aplikasi mendefinisikan bagaimana kontrol seperti rate limiting, anti-automation, dan adaptive response, digunakan untuk bertahan dari serangan seperti credential stuffing dan password brute force [cite: 1]. Dokumentasi harus memperjelas bagaimana kontrol ini dikonfigurasi dan mencegah penguncian akun yang berbahaya (malicious account lockout) [cite: 1].	1
6.1.2	Pastikan bahwa daftar kata-kata spesifik konteks didokumentasikan untuk mencegah penggunaannya dalam password [cite: 1]. Daftar tersebut dapat mencakup permutasi nama organisasi, nama produk, pengenalan sistem (system identifiers), nama sandi proyek, nama departemen atau peran, dan sejenisnya [cite: 1].	2
6.1.3	Pastikan bahwa, jika aplikasi mencakup beberapa jalur Authentication, semuanya didokumentasikan bersama dengan kontrol keamanan dan kekuatan Authentication yang harus diterapkan secara konsisten di seluruh jalur tersebut [cite: 1].	2

V6.2 Keamanan Password

Password, yang disebut “Memorized Secrets” oleh NIST SP 800-63, mencakup password, passphrase, PIN, pola buka kunci (unlock patterns), dan memilih gambar anak kucing yang tepat atau elemen gambar lainnya [cite: 1]. Mereka umumnya dianggap sebagai “sesuatu yang Anda ketahui” (something you know) dan sering digunakan sebagai mekanisme single-factor authentication [cite: 1].

Oleh karena itu, bagian ini berisi persyaratan untuk memastikan bahwa password dibuat dan ditangani dengan aman [cite: 1]. Sebagian besar persyaratan adalah L1 karena mereka paling penting pada level tersebut [cite: 1]. Dari L2 dan seterusnya, mekanisme multi-factor authentication (MFA) diwajibkan, di mana password mungkin menjadi salah satu dari faktor-faktor tersebut [cite: 1].

Persyaratan di bagian ini sebagian besar terkait dengan § 5.1.1.2 dari Panduan NIST [cite: 1].

#	Deskripsi	Level
6.2.1	Pastikan bahwa password yang diatur pengguna memiliki panjang setidaknya 8 karakter, meskipun minimal 15 karakter sangat direkomendasikan [cite: 1].	1
6.2.2	Pastikan bahwa pengguna dapat mengubah password mereka [cite: 1].	1
6.2.3	Pastikan bahwa fungsionalitas perubahan password memerlukan password pengguna saat ini dan yang baru [cite: 1].	1

#	Deskripsi	Level
6.2.4	Pastikan bahwa password yang dikirimkan selama pendaftaran akun atau perubahan password diperiksa terhadap sekumpulan, setidaknya, 3000 password teratas yang cocok dengan kebijakan password aplikasi, mis. panjang minimum [cite: 1].	1
6.2.5	Pastikan bahwa password dari komposisi apa pun dapat digunakan, tanpa aturan yang membatasi jenis karakter yang diizinkan [cite: 1]. Tidak boleh ada persyaratan untuk jumlah minimum karakter huruf besar atau kecil, angka, atau karakter khusus [cite: 1].	1
6.2.6	Pastikan bahwa kolom input password menggunakan type=password untuk menutupi (mask) entri [cite: 1]. Aplikasi dapat mengizinkan pengguna untuk sementara melihat seluruh password yang ditutupi, atau karakter terakhir yang diketik dari password [cite: 1].	1
6.2.7	Pastikan bahwa fungsi “paste”, browser password helpers, dan external password managers diizinkan [cite: 1].	1
6.2.8	Pastikan bahwa aplikasi memverifikasi password pengguna persis seperti yang diterima dari pengguna, tanpa modifikasi apa pun seperti pemotongan (truncation) atau transformasi huruf (case transformation) [cite: 1].	1
6.2.9	Pastikan bahwa password dengan panjang setidaknya 64 karakter diizinkan [cite: 1].	2
6.2.10	Pastikan bahwa password pengguna tetap valid sampai diketahui telah disusupi (compromised) atau pengguna memutarkannya (rotates) [cite: 1]. Aplikasi tidak boleh mensyaratkan rotasi kredensial berkala [cite: 1].	2
6.2.11	Pastikan bahwa daftar kata-kata spesifik konteks yang didokumentasikan digunakan untuk mencegah pembuatan password yang mudah ditebak [cite: 1].	2
6.2.12	Pastikan bahwa password yang dikirimkan selama pendaftaran akun atau perubahan password diperiksa terhadap sekumpulan password yang bocor (breached passwords) [cite: 1].	2

V6.3 Keamanan Authentication Umum

Bagian ini berisi persyaratan umum untuk keamanan mekanisme Authentication serta menetapkan ekspektasi yang berbeda untuk berbagai level [cite: 1]. Aplikasi L2 harus memaksa penggunaan multi-factor authentication (MFA) [cite: 1]. Aplikasi L3 harus menggunakan hardware-based authentication, yang dilakukan di lingkungan eksekusi yang dibuktikan dan dipercaya (Trusted Execution Environment - TEE) [cite: 1]. Ini bisa termasuk device-bound passkeys, autentikator eIDAS Level of

Assurance (LoA) High, autentikator dengan jaminan NIST Authenticator Assurance Level 3 (AAL3), atau mekanisme yang setara [cite: 1].

Meskipun ini adalah sikap yang relatif agresif terhadap MFA, sangat penting untuk meningkatkan standar di seputar ini untuk melindungi pengguna, dan setiap upaya untuk melonggarkan persyaratan ini harus disertai dengan rencana yang jelas tentang bagaimana risiko seputar Authentication akan dimitigasi, dengan mempertimbangkan panduan dan penelitian NIST mengenai topik ini [cite: 1].

Perhatikan bahwa pada saat rilis, NIST SP 800-63 menganggap email tidak dapat diterima sebagai mekanisme Authentication (salinan arsip) [cite: 1].

Persyaratan di bagian ini berkaitan dengan berbagai bagian dari Panduan NIST, termasuk: § 4.2.1, § 4.3.1, § 5.2.2, dan § 6.1.2 [cite: 1].

#	Deskripsi	Level
6.3.1	Pastikan bahwa kontrol untuk mencegah serangan seperti credential stuffing dan password brute force diimplementasikan sesuai dengan dokumentasi keamanan aplikasi [cite: 1].	1
6.3.2	Pastikan bahwa akun pengguna default (misalnya, “root”, “admin”, atau “sa”) tidak ada di dalam aplikasi atau dinonaktifkan [cite: 1].	1
6.3.3	Pastikan bahwa mekanisme multi-factor authentication (MFA) atau kombinasi mekanisme single-factor authentication harus digunakan untuk mengakses aplikasi [cite: 1]. Untuk L3, salah satu faktor harus berupa mekanisme hardware-based authentication yang memberikan ketahanan terhadap kompromi dan peniruan (impersonation resistance) terhadap serangan phishing sekaligus memverifikasi niat untuk mengAuthentication dengan mewajibkan tindakan yang dimulai oleh pengguna (seperti menekan tombol pada FIDO hardware key atau ponsel) [cite: 1]. Melonggarkan salah satu pertimbangan dalam persyaratan ini memerlukan alasan yang didokumentasikan sepenuhnya dan serangkaian kontrol mitigasi yang komprehensif [cite: 1].	2
6.3.4	Pastikan bahwa, jika aplikasi mencakup beberapa jalur Authentication, tidak ada jalur yang tidak didokumentasikan dan kontrol keamanan serta kekuatan Authentication diterapkan secara konsisten [cite: 1].	2
6.3.5	Pastikan bahwa pengguna diberi tahu tentang upaya Authentication yang mencurigakan (berhasil atau tidak berhasil) [cite: 1]. Ini dapat mencakup upaya Authentication dari lokasi atau klien yang tidak biasa, Authentication yang berhasil sebagian (hanya satu dari beberapa faktor), upaya Authentication setelah periode tidak aktif yang lama, atau Authentication yang berhasil setelah beberapa upaya gagal [cite: 1].	3

#	Deskripsi	Level
6.3.6	Pastikan bahwa email tidak digunakan sebagai mekanisme single-factor atau multi-factor authentication [cite: 1].	3
6.3.7	Pastikan bahwa pengguna diberi tahu setelah pembaruan detail Authentication, seperti pengaturan ulang kredensial (credential resets) atau modifikasi username atau alamat email [cite: 1].	3
6.3.8	Pastikan bahwa pengguna yang valid tidak dapat disimpulkan dari tantangan Authentication yang gagal, seperti dengan berdasarkan pada pesan kesalahan, kode respons HTTP, atau waktu respons yang berbeda [cite: 1]. Fungsionalitas pendaftaran dan lupa password (forgot password) juga harus memiliki perlindungan ini [cite: 1].	3

V6.4 Siklus Hidup dan Pemulihan Faktor Authentication

Faktor Authentication dapat mencakup password, soft tokens, hardware tokens, dan perangkat biometrik [cite: 1]. Menangani siklus hidup mekanisme ini dengan aman sangat penting untuk keamanan aplikasi, dan bagian ini mencakup persyaratan yang terkait dengan hal tersebut [cite: 1].

Persyaratan di bagian ini sebagian besar terkait dengan § 5.1.1.2 atau § 6.1.2.3 dari Panduan NIST [cite: 1].

#	Deskripsi	Level
6.4.1	Pastikan bahwa password awal atau kode aktivasi yang dihasilkan sistem dibuat secara acak dengan aman (securely randomly generated), mengikuti kebijakan password yang ada, dan kedaluwarsa setelah periode waktu yang singkat atau setelah pertama kali digunakan [cite: 1]. Rahasia awal ini tidak boleh diizinkan menjadi password jangka panjang [cite: 1].	1
6.4.2	Pastikan bahwa petunjuk password (password hints) atau knowledge-based authentication (disebut “pertanyaan rahasia”) tidak digunakan [cite: 1].	1
6.4.3	Pastikan bahwa proses yang aman untuk mengatur ulang password yang terlupakan diimplementasikan, yang tidak mengabaikan (bypass) mekanisme multi-factor authentication yang diaktifkan [cite: 1].	2
6.4.4	Pastikan bahwa jika suatu faktor multi-factor authentication hilang, bukti pembuktian identitas (identity proofing) dilakukan pada tingkat yang sama seperti saat pendaftaran (enrollment) [cite: 1].	2

#	Deskripsi	Level
6.4.5	Pastikan bahwa instruksi perpanjangan untuk mekanisme Authentication yang kedaluwarsa dikirim dengan waktu yang cukup untuk dilakukan sebelum mekanisme Authentication yang lama kedaluwarsa, mengatur pengingat otomatis jika perlu [cite: 1].	3
6.4.6	Pastikan bahwa pengguna administratif dapat memulai proses pengaturan ulang password untuk pengguna, tetapi ini tidak mengizinkan mereka untuk mengubah atau memilih password pengguna [cite: 1]. Hal ini mencegah situasi di mana mereka mengetahui password pengguna [cite: 1].	3

V6.5 Persyaratan Umum Multi-factor Authentication

Bagian ini memberikan panduan umum yang relevan untuk berbagai metode multi-factor authentication (MFA) yang berbeda [cite: 1].

Mekanismenya meliputi:

- Lookup Secrets [cite: 1]
- Time based One-time Passwords (TOTPs) [cite: 1]
- Mekanisme Out-of-Band [cite: 1]

Lookup secrets adalah daftar kode rahasia yang dibuat sebelumnya, mirip dengan Transaction Authorization Numbers (TAN), kode pemulihan media sosial, atau kisi (grid) yang berisi sekumpulan nilai acak [cite: 1]. Jenis mekanisme Authentication ini dianggap sebagai “sesuatu yang Anda miliki” (something you have) karena kode tersebut sengaja dibuat agar tidak mudah diingat sehingga perlu disimpan di suatu tempat [cite: 1].

Time based One-time Passwords (TOTPs) adalah token fisik atau perangkat lunak (soft tokens) yang menampilkan tantangan pseudo-acak (pseudo-random one-time challenge) yang terus berubah [cite: 1]. Jenis mekanisme Authentication ini dianggap sebagai “sesuatu yang Anda miliki” (something you have) [cite: 1]. TOTP multi-factor mirip dengan TOTP single-factor, namun memerlukan kode PIN yang valid, pembukaan kunci biometrik (biometric unlocking), penyisipan USB atau pemasangan NFC, atau beberapa nilai tambahan (seperti kalkulator penandatanganan transaksi) untuk dimasukkan guna membuat One-time Password (OTP) akhir [cite: 1].

Detail mengenai mekanisme out-of-band akan diberikan pada bagian berikutnya [cite: 1].

Persyaratan di bagian ini sebagian besar terkait dengan § 5.1.2, § 5.1.3, § 5.1.4.2, § 5.1.5.2, § 5.2.1, dan § 5.2.3 dari Panduan NIST [cite: 1].

#	Deskripsi	Level
6.5.1	Pastikan bahwa lookup secrets, permintaan atau kode Authentication out-of-band, dan time-based one-time passwords (TOTPs) hanya dapat berhasil digunakan sekali [cite: 1].	2
6.5.2	Pastikan bahwa, saat disimpan di backend aplikasi, lookup secrets dengan entropi kurang dari 112 bit (19 karakter alfanumerik acak atau 34 digit acak) di-hash dengan algoritma hashing penyimpanan password yang disetujui yang menggabungkan salt acak 32-bit [cite: 1]. Fungsi hash standar dapat digunakan jika rahasia tersebut memiliki entropi 112 bit atau lebih [cite: 1].	2
6.5.3	Pastikan bahwa lookup secrets, kode Authentication out-of-band, dan seed time-based one-time password (TOTP), dihasilkan menggunakan Cryptographically Secure Pseudorandom Number Generator (CSPRNG) untuk menghindari nilai yang dapat ditebak [cite: 1].	2
6.5.4	Pastikan bahwa lookup secrets dan kode Authentication out-of-band memiliki entropi minimal 20 bit (biasanya 4 karakter alfanumerik acak atau 6 digit acak sudah cukup) [cite: 1].	2
6.5.5	Pastikan bahwa permintaan, kode, atau token Authentication out-of-band, serta time-based one-time passwords (TOTPs) memiliki masa pakai yang ditentukan [cite: 1]. Permintaan out-of-band harus memiliki masa pakai maksimal 10 menit dan untuk TOTP masa pakai maksimal 30 detik [cite: 1].	2
6.5.6	Pastikan bahwa setiap faktor Authentication (termasuk physical devices) dapat dicabut jika terjadi pencurian atau kehilangan lainnya [cite: 1].	3
6.5.7	Pastikan bahwa mekanisme Authentication biometrik hanya digunakan sebagai faktor sekunder (secondary factors) bersama dengan sesuatu yang Anda miliki (something you have) atau sesuatu yang Anda ketahui (something you know) [cite: 1].	3
6.5.8	Pastikan bahwa time-based one-time passwords (TOTPs) diperiksa berdasarkan sumber waktu dari layanan tepercaya (trusted service) dan bukan dari waktu yang tidak tepercaya atau yang disediakan klien [cite: 1].	3

V6.6 Mekanisme Authentication Out-of-Band

Hal ini biasanya melibatkan server Authentication yang berkomunikasi dengan perangkat fisik melalui saluran sekunder yang aman [cite: 1]. Misalnya, mengirimkan push notifications ke perangkat seluler [cite: 1]. Jenis mekanisme Authentication ini dianggap sebagai “sesuatu yang Anda miliki”(something you have) [cite: 1].

Mekanisme Authentication out-of-band yang tidak aman seperti e-mail dan VOIP tidak diizinkan [cite:

1]. Authentication PSTN dan SMS saat ini dianggap sebagai “restricted” authentication mechanisms oleh NIST dan harus dihentikan penggunaannya demi Time based One-time Passwords (TOTPs), mekanisme kriptografi, atau sejenisnya [cite: 1]. NIST SP 800-63B § 5.1.3.3 menyarankan untuk menangani risiko device swap, penggantian SIM (SIM change), porting nomor, atau perilaku abnormal lainnya, jika Authentication out-of-band telepon atau SMS mutlak harus didukung [cite: 1]. Meskipun bagian ASVS ini tidak mewajibkan hal ini sebagai persyaratan, tidak mengambil tindakan pencegahan ini untuk aplikasi L2 yang sensitif atau aplikasi L3 harus dilihat sebagai bendera merah (red flag) yang signifikan [cite: 1].

Perhatikan bahwa NIST baru-baru ini juga memberikan panduan yang tidak menyarankan penggunaan push notifications [cite: 1]. Walaupun bagian ASVS ini tidak melakukannya, penting untuk menyadari risiko “push bombing” [cite: 1].

#	Deskripsi	Level
6.6.1	Pastikan bahwa mekanisme Authentication menggunakan Public Switched Telephone Network (PSTN) untuk mengirimkan One-time Passwords (OTPs) melalui telepon atau SMS hanya ditawarkan ketika nomor telepon telah divalidasi sebelumnya, metode alternatif yang lebih kuat (seperti Time based One-time Passwords) juga ditawarkan, dan layanan tersebut memberikan informasi tentang risiko keamanannya kepada pengguna [cite: 1]. Untuk aplikasi L3, telepon dan SMS tidak boleh tersedia sebagai opsi [cite: 1].	2
6.6.2	Pastikan bahwa permintaan, kode, atau token Authentication out-of-band terikat pada permintaan Authentication asli yang menghasilkannya dan tidak dapat digunakan untuk permintaan sebelumnya atau sesudahnya [cite: 1].	2
6.6.3	Pastikan bahwa mekanisme Authentication out-of-band berbasis kode dilindungi dari serangan brute force dengan menggunakan rate limiting [cite: 1]. Pertimbangkan juga untuk menggunakan kode dengan entropi setidaknya 64 bit [cite: 1].	2
6.6.4	Pastikan bahwa, jika push notifications digunakan untuk multi-factor authentication, rate limiting digunakan untuk mencegah serangan push bombing [cite: 1]. Pencocokan nomor (number matching) juga dapat memitigasi risiko ini [cite: 1].	3

V6.7 Mekanisme Authentication Kriptografi

Mekanisme Authentication kriptografi mencakup smart cards atau FIDO keys, di mana pengguna harus menyambungkan atau memasang perangkat kriptografi ke komputer untuk menyelesaikan Authentication [cite: 1]. Server Authentication akan mengirimkan tantangan (challenge nonce) ke perangkat atau perangkat lunak kriptografi, dan perangkat atau perangkat lunak tersebut

menghitung respons berdasarkan kunci kriptografi yang disimpan dengan aman [cite: 1]. Persyaratan di bagian ini memberikan panduan khusus implementasi untuk mekanisme ini, dengan panduan tentang algoritma kriptografi yang tercakup dalam bab “Kriptografi”[cite: 1].

Di mana shared atau secret keys digunakan untuk Authentication kriptografi, ini harus disimpan menggunakan mekanisme yang sama seperti rahasia sistem lainnya, sebagaimana didokumentasikan dalam bagian “Manajemen Rahasia”(Secret Management) di bab “Konfigurasi”[cite: 1].

Persyaratan di bagian ini sebagian besar terkait dengan § 5.1.7.2 dari Panduan NIST [cite: 1].

#	Deskripsi	Level
6.7.1	Pastikan bahwa sertifikat yang digunakan untuk memverifikasi asersi Authentication kriptografi disimpan dengan cara yang melindunginya dari modifikasi [cite: 1].	3
6.7.2	Pastikan bahwa challenge nonce memiliki panjang setidaknya 64 bit, dan secara statistik unik atau unik selama masa pakai perangkat kriptografi tersebut [cite: 1].	3

V6.8 Authentication dengan Identity Provider

Identity Providers (IdPs) menyediakan identitas federasi (federated identity) untuk pengguna [cite: 1]. Pengguna sering kali akan memiliki lebih dari satu identitas di beberapa IdP, seperti identitas perusahaan yang menggunakan Azure AD, Okta, Ping Identity, atau Google, atau identitas konsumen yang menggunakan Facebook, Twitter, Google, atau WeChat, untuk menyebutkan beberapa alternatif umum [cite: 1]. Daftar ini bukan dukungan terhadap perusahaan atau layanan tersebut, melainkan sekadar dorongan bagi pengembang untuk mempertimbangkan kenyataan bahwa banyak pengguna memiliki banyak identitas yang telah mapan [cite: 1]. Organisasi harus mempertimbangkan untuk mengintegrasikan identitas pengguna yang ada, sesuai dengan profil risiko dari kekuatan pembuktian identitas (identity proofing) IdP [cite: 1]. Misalnya, kecil kemungkinannya organisasi pemerintah akan menerima identitas media sosial sebagai login untuk sistem yang sensitif, karena mudah untuk membuat identitas palsu atau sekali pakai, sedangkan perusahaan game seluler mungkin perlu berintegrasi dengan platform media sosial besar untuk menumbuhkan basis pemain aktif mereka [cite: 1].

Penggunaan external identity providers yang aman membutuhkan konfigurasi dan verifikasi yang hati-hati untuk mencegah identity spoofing atau forged assertions [cite: 1]. Bagian ini menyediakan persyaratan untuk mengatasi risiko tersebut [cite: 1].

#	Deskripsi	Level
6.8.1	Pastikan bahwa, jika aplikasi mendukung beberapa identity providers (IdPs), identitas pengguna tidak dapat dipalsukan (spoofed) melalui identity provider lain yang didukung (mis. dengan menggunakan user identifier yang sama) [cite: 1]. Mitigasi standar adalah aplikasi mendaftarkan dan mengidentifikasi pengguna menggunakan kombinasi IdP ID (berfungsi sebagai namespace) dan ID pengguna di IdP [cite: 1].	2
6.8.2	Pastikan bahwa keberadaan dan integritas tanda tangan digital pada asersi Authentication (misalnya pada JWTs atau SAML assertions) selalu divalidasi, menolak setiap asersi yang tidak ditandatangani atau memiliki tanda tangan tidak valid [cite: 1].	2
6.8.3	Pastikan bahwa SAML assertions diproses secara unik dan digunakan hanya sekali dalam periode validitas untuk mencegah replay attacks [cite: 1].	2
6.8.4	Pastikan bahwa, jika sebuah aplikasi menggunakan Identity Provider (IdP) terpisah dan mengharapkan kekuatan Authentication, metode, atau kebaruan spesifik untuk fungsi tertentu, aplikasi memverifikasi ini menggunakan informasi yang dikembalikan oleh IdP [cite: 1]. Misalnya, jika OIDC digunakan, ini mungkin dicapai dengan memvalidasi klaim ID Token seperti 'acr', 'amr', dan 'auth_time'(jika ada) [cite: 1]. Jika IdP tidak memberikan informasi ini, aplikasi harus memiliki pendekatan fallback yang didokumentasikan yang mengasumsikan bahwa mekanisme Authentication kekuatan minimum telah digunakan (misalnya, single-factor authentication menggunakan username dan password) [cite: 1].	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- NIST SP 800-63 - Digital Identity Guidelines [cite: 1]
- NIST SP 800-63B - Authentication and Lifecycle Management [cite: 1]
- NIST SP 800-63 FAQ [cite: 1]
- OWASP Web Security Testing Guide: Testing for Authentication [cite: 1]
- OWASP Password Storage Cheat Sheet [cite: 1]
- OWASP Forgot Password Cheat Sheet [cite: 1]
- OWASP Choosing and Using Security Questions Cheat Sheet [cite: 1]
- CISA Guidance on "Number Matching" [cite: 1]
- Details on the FIDO Alliance [cite: 1] v6_Authentication_terjemahan.md Menampilkan v6_Authentication_terjemahan.md.

V7 Session Management

Tujuan Kontrol

Mekanisme session management memungkinkan aplikasi untuk mengaitkan interaksi pengguna dan perangkat dari waktu ke waktu, bahkan ketika menggunakan protokol komunikasi yang stateless (seperti HTTP). Aplikasi modern dapat menggunakan beberapa session token dengan karakteristik dan tujuan yang berbeda. Sistem session management yang aman adalah sistem yang mencegah penyerang memperoleh, memanfaatkan, atau menyalahgunakan sesi milik korban. Aplikasi yang mengelola sesi harus memastikan bahwa persyaratan tingkat tinggi berikut mengenai session management terpenuhi:

- Sesi bersifat unik untuk setiap individu dan tidak dapat ditebak atau dibagikan.
- Sesi menjadi tidak valid ketika sudah tidak diperlukan lagi dan mengalami timeout selama periode tidak aktif.

Banyak persyaratan dalam bab ini terkait dengan kontrol terpilih dari NIST SP 800-63 Digital Identity Guidelines, yang berfokus pada ancaman umum dan kelemahan autentikasi yang sering dieksploitasi.

Perlu dicatat bahwa persyaratan untuk detail implementasi spesifik dari mekanisme session management tertentu dapat ditemukan di tempat lain:

- HTTP Cookies merupakan mekanisme umum untuk mengamankan session token. Persyaratan keamanan spesifik untuk cookie dapat ditemukan pada bab “Web Frontend Security”.
- Self-contained tokens sering digunakan sebagai cara untuk mempertahankan sesi. Persyaratan keamanan spesifik dapat ditemukan pada bab “Self-contained Tokens”.

V7.1 Dokumentasi Session Management

Tidak ada satu pola yang cocok untuk semua aplikasi. Oleh karena itu, tidak memungkinkan untuk mendefinisikan batasan dan limit universal yang cocok untuk semua kasus. Sebuah analisis risiko dengan keputusan keamanan yang terdokumentasi terkait penanganan sesi harus dilakukan sebagai prasyarat sebelum implementasi dan pengujian. Hal ini memastikan bahwa sistem session management disesuaikan dengan kebutuhan spesifik aplikasi.

Terlepas dari apakah mekanisme sesi yang dipilih bersifat stateful atau “stateless”, analisis tersebut harus lengkap dan terdokumentasi untuk menunjukkan bahwa solusi yang dipilih mampu memenuhi semua persyaratan keamanan yang relevan. Interaksi dengan mekanisme Single Sign-on (SSO) yang digunakan juga harus dipertimbangkan.

#	Deskripsi	Level
7.1.1	Verifikasi bahwa inactivity timeout sesi pengguna dan absolute maximum session lifetime terdokumentasi, sesuai jika dikombinasikan dengan kontrol lainnya, dan bahwa dokumentasi tersebut mencakup justifikasi untuk setiap penyimpangan dari persyaratan re-authentication pada NIST SP 800-63B.	2
7.1.2	Verifikasi bahwa dokumentasi mendefinisikan berapa banyak sesi paralel (concurrent) yang diizinkan untuk satu akun serta perilaku dan tindakan yang dimaksudkan ketika jumlah maksimum sesi aktif tercapai.	2
7.1.3	Verifikasi bahwa semua sistem yang membuat dan mengelola sesi pengguna sebagai bagian dari ekosistem federated identity management (seperti sistem SSO) terdokumentasi beserta kontrol untuk mengoordinasikan session lifetime, terminasi, dan kondisi lain apa pun yang memerlukan re-authentication.	2

V7.2 Keamanan Dasar Session Management

Bagian ini memenuhi persyaratan mendasar dari sesi yang aman dengan memverifikasi bahwa session token dihasilkan dan divalidasi secara aman.

#	Deskripsi	Level
7.2.1	Verifikasi bahwa aplikasi melakukan seluruh verifikasi session token menggunakan layanan backend yang tepercaya.	1
7.2.2	Verifikasi bahwa aplikasi menggunakan self-contained token atau reference token yang dihasilkan secara dinamis untuk session management, yaitu tidak menggunakan API secrets dan keys yang statis.	1
7.2.3	Verifikasi bahwa jika reference token digunakan untuk merepresentasikan sesi pengguna, token tersebut bersifat unik dan dihasilkan menggunakan cryptographically secure pseudo-random number generator (CSPRNG) serta memiliki entropi minimal 128 bit.	1
7.2.4	Verifikasi bahwa aplikasi menghasilkan session token baru pada saat autentikasi pengguna, termasuk re-authentication, dan menghentikan session token yang sedang berlaku (current).	1

V7.3 Session Timeout

Mekanisme session timeout berfungsi untuk meminimalkan jendela peluang (window of opportunity) terjadinya session hijacking dan bentuk penyalahgunaan sesi lainnya. Timeout harus

memenuhi keputusan keamanan yang terdokumentasi.

#	Deskripsi	Level
7.3.1	Verifikasi bahwa terdapat inactivity timeout sedemikian rupa sehingga re-authentication diberlakukan sesuai dengan analisis risiko dan keputusan keamanan yang terdokumentasi.	2
7.3.2	Verifikasi bahwa terdapat absolute maximum session lifetime sedemikian rupa sehingga re-authentication diberlakukan sesuai dengan analisis risiko dan keputusan keamanan yang terdokumentasi.	2

V7.4 Terminasi Sesi

Terminasi sesi dapat ditangani baik oleh aplikasi itu sendiri maupun oleh penyedia SSO jika penyedia SSO yang menangani session management, bukan aplikasi. Mungkin perlu ditentukan apakah penyedia SSO termasuk dalam scope saat mempertimbangkan persyaratan pada bagian ini, karena beberapa hal mungkin dikendalikan oleh penyedia tersebut.

Terminasi sesi harus mengakibatkan diperlukannya re-authentication dan efektif di seluruh aplikasi, federated login (jika ada), dan pihak-pihak terkait (relying parties) lainnya.

Untuk mekanisme sesi stateful, terminasi umumnya melibatkan invalidasi sesi pada backend. Dalam kasus self-contained token, langkah tambahan diperlukan untuk mencabut (revoke) atau memblokir token tersebut, karena jika tidak, token tersebut dapat tetap valid hingga masa berlakunya habis (expiration).

#	Deskripsi	Level
7.4.1	Verifikasi bahwa ketika terminasi sesi dipicu (seperti logout atau expiration), aplikasi melarang penggunaan sesi tersebut lebih lanjut. Untuk reference token atau sesi stateful, hal ini berarti melakukan invalidasi terhadap data sesi pada backend aplikasi. Aplikasi yang menggunakan self-contained token memerlukan solusi seperti mempertahankan daftar token yang telah dihentikan, melarang token yang dibuat sebelum tanggal dan waktu tertentu per pengguna, atau melakukan rotasi signing key per pengguna.	1
7.4.2	Verifikasi bahwa aplikasi menghentikan semua sesi aktif ketika sebuah akun pengguna dinonaktifkan atau dihapus (seperti seorang karyawan yang keluar dari perusahaan).	1

#	Deskripsi	Level
7.4.3	Verifikasi bahwa aplikasi memberikan opsi untuk menghentikan semua sesi aktif lainnya setelah keberhasilan perubahan atau penghapusan faktor autentikasi apa pun (termasuk perubahan password melalui reset atau recovery dan, jika ada, pembaruan pengaturan MFA).	2
7.4.4	Verifikasi bahwa semua halaman yang memerlukan autentikasi memiliki akses yang mudah dan terlihat jelas ke fungsionalitas logout.	2
7.4.5	Verifikasi bahwa administrator aplikasi mampu menghentikan sesi aktif untuk pengguna tertentu atau untuk semua pengguna.	2

V7.5 Pertahanan Terhadap Penyalahgunaan Sesi

Bagian ini menyediakan persyaratan untuk memitigasi risiko yang ditimbulkan oleh sesi aktif yang dibajak (hijacked) atau disalahgunakan melalui vektor yang bergantung pada keberadaan dan kemampuan sesi pengguna aktif. Misalnya, menggunakan eksekusi konten berbahaya untuk memaksa browser korban yang telah terautentikasi melakukan suatu tindakan menggunakan sesi milik korban.

Perlu dicatat bahwa panduan spesifik per level pada bab “Authentication” harus dipertimbangkan saat menerapkan persyaratan pada bagian ini.

#	Deskripsi	Level
7.5.1	Verifikasi bahwa aplikasi mewajibkan re-authentication penuh sebelum mengizinkan modifikasi terhadap atribut akun yang sensitif yang dapat memengaruhi autentikasi, seperti alamat email, nomor telepon, konfigurasi MFA, atau informasi lain yang digunakan dalam account recovery.	2
7.5.2	Verifikasi bahwa pengguna dapat melihat dan (setelah melakukan autentikasi ulang dengan setidaknya satu faktor) menghentikan sebagian atau seluruh sesi yang sedang aktif saat ini.	2
7.5.3	Verifikasi bahwa aplikasi mewajibkan autentikasi lebih lanjut dengan setidaknya satu faktor atau verifikasi sekunder sebelum melakukan transaksi atau operasi yang sangat sensitif.	3

V7.6 Federated Re-authentication

Bagian ini terkait dengan pihak yang menulis kode Relying Party (RP) atau Identity Provider (IdP). Persyaratan ini diturunkan dari NIST SP 800-63C untuk Federation & Assertions.

#	Deskripsi	Level
7.6.1	Verifikasi bahwa session lifetime dan terminasi antara Relying Parties (RP) dan Identity Providers (IdP) berperilaku sebagaimana yang terdokumentasi, mewajibkan re-authentication sesuai kebutuhan seperti ketika waktu maksimum antar peristiwa autentikasi IdP tercapai.	2
7.6.2	Verifikasi bahwa pembuatan sesi memerlukan persetujuan pengguna atau tindakan eksplisit, mencegah pembuatan sesi aplikasi baru tanpa interaksi pengguna.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 Authorization

Tujuan Kontrol

Authorization memastikan bahwa akses hanya diberikan kepada consumer yang diizinkan (pengguna, server, dan client lainnya). Untuk menegakkan Principle of Least Privilege (POLP), aplikasi yang diverifikasi harus memenuhi persyaratan tingkat tinggi berikut:

- Mendokumentasikan aturan authorization, termasuk faktor pengambilan keputusan dan konteks lingkungan (environmental contexts).
- Consumer hanya boleh memiliki akses terhadap resource yang diizinkan oleh entitlement yang telah ditentukan bagi mereka.

V8.1 Dokumentasi Authorization

Dokumentasi authorization yang komprehensif sangat penting untuk memastikan bahwa keputusan keamanan diterapkan secara konsisten, dapat diaudit, dan selaras dengan kebijakan organisasi. Hal ini mengurangi risiko akses tidak sah dengan membuat persyaratan keamanan menjadi jelas dan dapat ditindaklanjuti bagi developer, administrator, dan tester.

#	Deskripsi	Level
8.1.1	Verifikasi bahwa dokumentasi authorization mendefinisikan aturan untuk membatasi akses pada level fungsi (function-level) dan akses spesifik terhadap data berdasarkan permission consumer dan atribut resource.	1
8.1.2	Verifikasi bahwa dokumentasi authorization mendefinisikan aturan untuk pembatasan akses pada level field (baik untuk read maupun write) berdasarkan permission consumer dan atribut resource. Perlu dicatat bahwa aturan ini mungkin bergantung pada nilai atribut lain dari objek data terkait, seperti state atau status.	2
8.1.3	Verifikasi bahwa dokumentasi aplikasi mendefinisikan atribut lingkungan dan kontekstual (termasuk namun tidak terbatas pada, waktu dalam sehari, lokasi pengguna, alamat IP, atau perangkat) yang digunakan dalam aplikasi untuk membuat keputusan keamanan, termasuk yang berkaitan dengan authentication dan authorization.	3
8.1.4	Verifikasi bahwa dokumentasi authentication dan authorization mendefinisikan bagaimana faktor lingkungan dan kontekstual digunakan dalam pengambilan keputusan, selain authorization pada level fungsi, data spesifik, dan level field. Hal ini harus mencakup atribut yang dievaluasi, ambang batas (thresholds) risiko, dan tindakan yang diambil (misalnya, allow, challenge, deny, step-up authentication).	3

V8.2 Desain Authorization Umum

Menerapkan kontrol authorization yang terperinci pada level fungsi, data, dan field memastikan bahwa consumer hanya dapat mengakses apa yang secara eksplisit telah diberikan kepada mereka.

#	Deskripsi	Level
8.2.1	Verifikasi bahwa aplikasi memastikan akses pada level fungsi dibatasi hanya untuk consumer dengan permission eksplisit.	1
8.2.2	Verifikasi bahwa aplikasi memastikan akses spesifik terhadap data dibatasi hanya untuk consumer dengan permission eksplisit terhadap item data tertentu guna memitigasi insecure direct object reference (IDOR) dan broken object level authorization (BOLA).	1
8.2.3	Verifikasi bahwa aplikasi memastikan akses pada level field dibatasi hanya untuk consumer dengan permission eksplisit terhadap field tertentu guna memitigasi broken object property level authorization (BOPLA).	2

#	Deskripsi	Level
8.2.4	Verifikasi bahwa kontrol keamanan adaptif berdasarkan atribut lingkungan dan kontekstual consumer (seperti waktu dalam sehari, lokasi, alamat IP, atau perangkat) diterapkan untuk keputusan authentication dan authorization, sebagaimana didefinisikan dalam dokumentasi aplikasi. Kontrol ini harus diterapkan baik ketika consumer mencoba memulai sesi baru maupun selama sesi yang sedang berlangsung.	3

V8.3 Authorization Level Operasi

Penerapan segera (immediate) terhadap perubahan authorization pada tier arsitektur aplikasi yang sesuai sangat penting untuk mencegah tindakan tidak sah, terutama pada lingkungan yang dinamis.

#	Deskripsi	Level
8.3.1	Verifikasi bahwa aplikasi menegakkan aturan authorization pada trusted service layer dan tidak mengandalkan kontrol yang dapat dimanipulasi oleh consumer yang tidak tepercaya, seperti client-side JavaScript.	1
8.3.2	Verifikasi bahwa perubahan pada nilai yang menjadi dasar keputusan authorization diterapkan secara langsung (immediate). Jika perubahan tidak dapat diterapkan secara langsung (seperti ketika mengandalkan data pada self-contained token), harus ada kontrol mitigasi untuk memberikan peringatan ketika seorang consumer melakukan suatu tindakan pada saat mereka sudah tidak lagi berwenang untuk melakukannya, serta mengembalikan (revert) perubahan tersebut. Perlu dicatat bahwa alternatif ini tidak memitigasi kebocoran informasi (information leakage).	3
8.3.3	Verifikasi bahwa akses terhadap sebuah objek didasarkan pada permission dari subjek asal (originating subject, misalnya consumer), bukan pada permission dari perantara atau layanan mana pun yang bertindak atas nama mereka. Misalnya, jika seorang consumer memanggil sebuah web service menggunakan self-contained token untuk autentikasi, dan layanan tersebut kemudian meminta data dari layanan lain, layanan kedua tersebut harus menggunakan token milik consumer, bukan token machine-to-machine dari layanan pertama, untuk membuat keputusan permission.	3

V8.4 Pertimbangan Authorization Lainnya

Pertimbangan tambahan untuk authorization, khususnya untuk interface administratif dan lingkungan multi-tenant, membantu mencegah akses tidak sah.

#	Deskripsi	Level
8.4.1	Verifikasi bahwa aplikasi multi-tenant menggunakan kontrol cross-tenant untuk memastikan operasi consumer tidak akan pernah memengaruhi tenant lain yang tidak memiliki permission untuk berinteraksi dengan mereka.	2
8.4.2	Verifikasi bahwa akses ke interface administratif menerapkan beberapa lapisan keamanan, termasuk verifikasi identitas consumer secara berkelanjutan, penilaian postur keamanan perangkat (device security posture assessment), dan analisis risiko kontekstual, guna memastikan bahwa lokasi jaringan atau endpoint tepercaya bukan satu-satunya faktor untuk authorization, meskipun faktor-faktor tersebut dapat mengurangi kemungkinan akses tidak sah.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 Self-contained Tokens

Tujuan Kontrol

Konsep self-contained token disebutkan dalam RFC 6749 OAuth 2.0 asli dari tahun 2012. Istilah ini merujuk pada sebuah token yang berisi data atau claims yang akan diandalkan oleh layanan penerima untuk membuat keputusan keamanan. Hal ini perlu dibedakan dari token sederhana yang hanya berisi sebuah identifier, yang digunakan oleh layanan penerima untuk mencari data secara lokal. Contoh paling umum dari self-contained token adalah JSON Web Tokens (JWT) dan SAML assertions.

Penggunaan self-contained token telah menjadi sangat luas, bahkan di luar OAuth dan OIDC. Pada saat yang sama, keamanan mekanisme ini bergantung pada kemampuan untuk memvalidasi integritas token dan memastikan bahwa token tersebut valid untuk konteks tertentu. Terdapat banyak jelek (pitfalls) dalam proses ini, dan bab ini menyediakan detail spesifik mengenai mekanisme yang harus dimiliki oleh aplikasi untuk mencegahnya.

V9.1 Sumber dan Integritas Token

Bagian ini mencakup persyaratan untuk memastikan bahwa token telah dihasilkan oleh pihak tepercaya dan tidak telah dimanipulasi (tampered with).

#	Deskripsi	Level
9.1.1	Verifikasi bahwa self-contained token divalidasi menggunakan digital signature atau MAC-nya untuk melindungi dari manipulasi (tampering) sebelum menerima isi token tersebut.	1
9.1.2	Verifikasi bahwa hanya algoritma yang terdapat pada allowlist yang dapat digunakan untuk membuat dan memverifikasi self-contained token, untuk konteks tertentu. Allowlist tersebut harus mencakup algoritma yang diizinkan, idealnya hanya algoritma simetris atau asimetris saja, dan tidak boleh mencakup algoritma 'None'. Jika baik algoritma simetris maupun asimetris harus didukung, kontrol tambahan diperlukan untuk mencegah key confusion.	1
9.1.3	Verifikasi bahwa key material yang digunakan untuk memvalidasi self-contained token berasal dari sumber tepercaya yang telah dikonfigurasi sebelumnya (pre-configured) untuk token issuer tersebut, guna mencegah penyerang menentukan sumber dan key yang tidak tepercaya. Untuk JWT dan struktur JWS lainnya, header seperti 'jku', 'x5u', dan 'jwk' harus divalidasi terhadap allowlist sumber tepercaya.	1

V9.2 Konten Token

Sebelum membuat keputusan keamanan berdasarkan konten dari sebuah self-contained token, perlu dilakukan validasi bahwa token tersebut telah dipresentasikan dalam masa berlakunya (validity period) dan bahwa token tersebut memang dimaksudkan untuk digunakan oleh layanan penerima serta untuk tujuan token tersebut dipresentasikan. Hal ini membantu menghindari penggunaan silang (cross-usage) yang tidak aman antar layanan yang berbeda atau dengan jenis token yang berbeda dari issuer yang sama.

Persyaratan spesifik untuk OAuth dan OIDC dibahas pada bab tersendiri.

#	Deskripsi	Level
9.2.1	Verifikasi bahwa, jika sebuah rentang waktu validitas (validity time span) terdapat pada data token, token dan isinya hanya diterima jika waktu verifikasi berada dalam rentang waktu validitas tersebut. Misalnya, untuk JWT, claims 'nbf' dan 'exp' harus diverifikasi.	1
9.2.2	Verifikasi bahwa layanan yang menerima sebuah token memvalidasi bahwa token tersebut memiliki tipe yang benar dan dimaksudkan untuk tujuan yang sesuai sebelum menerima isi token tersebut. Misalnya, hanya access token yang dapat diterima untuk keputusan authorization dan hanya ID Token yang dapat digunakan untuk membuktikan autentikasi pengguna.	2
9.2.3	Verifikasi bahwa layanan hanya menerima token yang dimaksudkan untuk digunakan dengan layanan tersebut (audience). Untuk JWT, hal ini dapat dicapai dengan memvalidasi claim 'aud' terhadap allowlist yang didefinisikan pada layanan tersebut.	2
9.2.4	Verifikasi bahwa, jika sebuah token issuer menggunakan private key yang sama untuk menerbitkan token bagi audience yang berbeda-beda, token yang diterbitkan tersebut mengandung sebuah audience restriction yang secara unik mengidentifikasi audience yang dituju. Hal ini akan mencegah sebuah token digunakan kembali dengan audience yang tidak dimaksudkan. Jika audience identifier disediakan secara dinamis (dynamically provisioned), token issuer harus memvalidasi audience tersebut guna memastikan bahwa hal ini tidak mengakibatkan audience impersonation.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP JSON Web Token Cheat Sheet

V10 OAuth dan OIDC

Tujuan Kontrol

OAuth2 (disebut sebagai OAuth dalam bab ini) adalah framework standar industri untuk delegated authorization. Misalnya, dengan menggunakan OAuth, sebuah aplikasi client dapat memperoleh akses ke API (server resources) atas nama pengguna, asalkan pengguna tersebut telah mengizinkan aplikasi client untuk melakukannya.

Dengan sendirinya, OAuth tidak dirancang untuk autentikasi pengguna. Framework OpenID Connect (OIDC) memperluas OAuth dengan menambahkan sebuah lapisan identitas pengguna di atas OAuth. OIDC menyediakan dukungan untuk fitur-fitur termasuk informasi pengguna yang terstandardisasi, Single Sign-On (SSO), dan session management. Karena OIDC merupakan perluasan dari OAuth, persyaratan OAuth pada bab ini juga berlaku untuk OIDC.

Peran-peran berikut didefinisikan dalam OAuth:

- OAuth client adalah aplikasi yang berupaya memperoleh akses ke server resources (misalnya, dengan memanggil sebuah API menggunakan access token yang diterbitkan). OAuth client sering kali berupa aplikasi server-side.
 - Confidential client adalah client yang mampu menjaga kerahasiaan kredensial yang digunakannya untuk mengautentikasi dirinya sendiri dengan authorization server.
 - Public client tidak mampu menjaga kerahasiaan kredensial untuk mengautentikasi dengan authorization server. Oleh karena itu, alih-alih mengautentikasi dirinya sendiri (misalnya, menggunakan parameter 'client_id' dan 'client_secret'), client tersebut hanya mengidentifikasi dirinya (menggunakan parameter 'client_id').
- OAuth resource server (RS) adalah server API yang mengekspos resource kepada OAuth client.
- OAuth authorization server (AS) adalah aplikasi server yang menerbitkan access token kepada OAuth client. Access token ini memungkinkan OAuth client mengakses resource RS, baik atas nama pengguna akhir (end-user) maupun atas nama OAuth client itu sendiri. AS sering kali merupakan aplikasi terpisah, namun (jika sesuai) dapat diintegrasikan ke dalam RS yang sesuai.
- Resource owner (RO) adalah pengguna akhir yang mengizinkan OAuth client untuk memperoleh akses terbatas ke resource yang di-hosting pada resource server atas nama mereka. Resource owner menyetujui delegated authorization ini dengan berinteraksi dengan authorization server.

Peran-peran berikut didefinisikan dalam OIDC:

- Relying party (RP) adalah aplikasi client yang meminta autentikasi pengguna akhir melalui OpenID Provider. RP berperan sebagai OAuth client.
- OpenID Provider (OP) adalah sebuah OAuth AS yang mampu mengautentikasi pengguna akhir dan menyediakan OIDC claims kepada RP. OP dapat merupakan identity provider (IdP), namun dalam skenario federated, OP dan identity provider (tempat pengguna akhir melakukan autentikasi) dapat merupakan aplikasi server yang berbeda.

OAuth dan OIDC awalnya dirancang untuk aplikasi pihak ketiga (third-party). Saat ini, keduanya juga sering digunakan oleh aplikasi pihak pertama (first-party). Namun, ketika digunakan dalam skenario pihak pertama, seperti authentication dan session management, protokol tersebut menambahkan kompleksitas tertentu, yang dapat menimbulkan tantangan keamanan baru.

OAuth dan OIDC dapat digunakan untuk berbagai jenis aplikasi, namun fokus untuk ASVS dan persyaratan pada bab ini adalah pada aplikasi web dan API.

Karena OAuth dan OIDC dapat dianggap sebagai logika di atas teknologi web, persyaratan umum dari bab-bab lain selalu berlaku, dan bab ini tidak dapat dipisahkan dari konteksnya.

Bab ini membahas praktik terbaik terkini (best current practices) untuk OAuth2 dan OIDC yang selaras dengan spesifikasi yang terdapat pada <https://oauth.net/2/> dan <https://openid.net/developers/specs/>. Meskipun RFC dianggap sudah matang (mature), RFC tersebut sering diperbarui. Oleh karena itu, penting untuk menyelaraskan dengan versi terbaru saat menerapkan persyaratan pada bab ini. Lihat bagian referensi untuk detail lebih lanjut.

Mengingat kompleksitas area ini, sangat penting bagi solusi OAuth atau OIDC yang aman untuk menggunakan authorization server yang sudah dikenal dan menjadi standar industri serta menerapkan konfigurasi keamanan yang direkomendasikan.

Terminologi yang digunakan dalam bab ini selaras dengan RFC OAuth dan spesifikasi OIDC, namun perlu dicatat bahwa terminologi OIDC hanya digunakan untuk persyaratan yang spesifik terhadap OIDC; selain itu, terminologi OAuth yang digunakan.

Dalam konteks OAuth dan OIDC, istilah “token” pada bab ini merujuk pada:

- Access token, yang hanya boleh dikonsumsi oleh RS dan dapat berupa reference token yang divalidasi menggunakan introspection atau self-contained token yang divalidasi menggunakan key material tertentu.
- Refresh token, yang hanya boleh dikonsumsi oleh authorization server yang menerbitkan token tersebut.
- OIDC ID Token, yang hanya boleh dikonsumsi oleh client yang memicu authorization flow.

Level risiko untuk beberapa persyaratan pada bab ini bergantung pada apakah client tersebut merupakan confidential client atau dianggap sebagai public client. Karena penggunaan client authentication yang kuat dapat memitigasi banyak attack vector, beberapa persyaratan mungkin dilonggarkan saat menggunakan confidential client untuk aplikasi L1.

V10.1 Keamanan Umum OAuth dan OIDC

Bagian ini membahas persyaratan arsitektur umum yang berlaku untuk semua aplikasi yang menggunakan OAuth atau OIDC.

#	Deskripsi	Level
10.1.1	Verifikasi bahwa token hanya dikirimkan ke komponen yang benar-benar memerlukannya. Misalnya, saat menggunakan pola backend-for-frontend untuk aplikasi JavaScript berbasis browser, access token dan refresh token hanya boleh dapat diakses oleh backend.	2

#	Deskripsi	Level
10.1.2	Verifikasi bahwa client hanya menerima nilai dari authorization server (seperti authorization code atau ID Token) jika nilai tersebut berasal dari sebuah authorization flow yang diinisiasi oleh user agent session dan transaksi yang sama. Hal ini mensyaratkan bahwa secret yang dihasilkan oleh client, seperti proof key for code exchange (PKCE) 'code_verifier', 'state', atau OIDC 'nonce', tidak dapat ditebak, bersifat spesifik terhadap transaksi tersebut, dan terikat secara aman baik pada client maupun pada user agent session tempat transaksi tersebut dimulai.	2

V10.2 OAuth Client

Persyaratan ini merinci tanggung jawab untuk aplikasi OAuth client. Client dapat berupa, misalnya, sebuah backend web server (sering bertindak sebagai Backend For Frontend, BFF), sebuah integrasi backend service, atau sebuah frontend Single Page Application (SPA, dikenal juga sebagai aplikasi berbasis browser).

Secara umum, backend client dianggap sebagai confidential client dan frontend client dianggap sebagai public client. Namun, aplikasi native yang berjalan pada perangkat pengguna akhir dapat dianggap sebagai confidential ketika menggunakan OAuth dynamic client registration.

#	Deskripsi	Level
10.2.1	Verifikasi bahwa, jika code flow digunakan, OAuth client memiliki perlindungan terhadap serangan browser-based request forgery, yang umum dikenal sebagai cross-site request forgery (CSRF), yang memicu token requests, baik dengan menggunakan fungsionalitas proof key for code exchange (PKCE) atau dengan memeriksa parameter 'state' yang dikirimkan pada authorization request.	2
10.2.2	Verifikasi bahwa, jika OAuth client dapat berinteraksi dengan lebih dari satu authorization server, client tersebut memiliki pertahanan terhadap mix-up attacks. Misalnya, dapat dilakukan dengan mewajibkan authorization server mengembalikan nilai parameter 'iss' dan memvalidasinya pada authorization response dan token response.	2
10.2.3	Verifikasi bahwa OAuth client hanya meminta scopes (atau parameter authorization lainnya) yang diperlukan pada request ke authorization server.	3

V10.3 OAuth Resource Server

Dalam konteks ASVS dan bab ini, resource server adalah sebuah API. Untuk menyediakan akses yang aman, resource server harus:

- Memvalidasi access token, sesuai dengan format token dan spesifikasi protokol yang relevan, misalnya, JWT-validation atau OAuth token introspection.
- Jika valid, menegakkan keputusan authorization berdasarkan informasi dari access token dan permission yang telah diberikan. Misalnya, resource server perlu memverifikasi bahwa client (yang bertindak atas nama RO) berwenang untuk mengakses resource yang diminta.

Oleh karena itu, persyaratan yang tercantum di sini bersifat spesifik terhadap OAuth atau OIDC dan harus dilakukan setelah validasi token dan sebelum melakukan authorization berdasarkan informasi dari token tersebut.

#	Deskripsi	Level
10.3.1	Verifikasi bahwa resource server hanya menerima access token yang dimaksudkan untuk digunakan dengan layanan tersebut (audience). Audience dapat disertakan dalam access token yang terstruktur (seperti claim 'aud' pada JWT), atau dapat diperiksa menggunakan token introspection endpoint.	2
10.3.2	Verifikasi bahwa resource server menegakkan keputusan authorization berdasarkan claims dari access token yang mendefinisikan delegated authorization. Jika claims seperti 'sub', 'scope', dan 'authorization_details' ada, claims tersebut harus menjadi bagian dari keputusan tersebut.	2
10.3.3	Verifikasi bahwa jika sebuah keputusan access control mensyaratkan identifikasi pengguna unik dari sebuah access token (JWT atau token introspection response terkait), resource server mengidentifikasi pengguna dari claims yang tidak dapat dipindahtangankan (reassigned) ke pengguna lain. Umumnya, hal ini berarti menggunakan kombinasi claims 'iss' dan 'sub'.	2
10.3.4	Verifikasi bahwa, jika resource server mensyaratkan kekuatan, metode, atau kebaruan (recentness) autentikasi tertentu, resource server tersebut memverifikasi bahwa access token yang dipresentasikan memenuhi batasan-batasan tersebut. Misalnya, jika ada, dengan menggunakan claims OIDC 'acr', 'amr', dan 'auth_time' secara berurutan.	2
10.3.5	Verifikasi bahwa resource server mencegah penggunaan access token yang dicuri atau replay access token (dari pihak yang tidak berwenang) dengan mewajibkan sender-constrained access token, baik Mutual TLS untuk OAuth 2 maupun OAuth 2 Demonstration of Proof of Possession (DPoP).	3

V10.4 OAuth Authorization Server

Persyaratan ini merinci tanggung jawab untuk OAuth authorization server, termasuk OpenID Provider.

Untuk client authentication, metode 'self_signed_tls_client_auth' diizinkan dengan prasyarat yang disyaratkan oleh bagian 2.2 dari RFC 8705.

#	Deskripsi	Level
10.4.1	Verifikasi bahwa authorization server memvalidasi redirect URI berdasarkan sebuah allowlist yang spesifik terhadap client dari URI yang telah didaftarkan sebelumnya (pre-registered) menggunakan exact string comparison.	1
10.4.2	Verifikasi bahwa, jika authorization server mengembalikan authorization code pada authorization response, code tersebut hanya dapat digunakan sekali untuk sebuah token request. Untuk request kedua yang valid dengan sebuah authorization code yang telah digunakan sebelumnya untuk menerbitkan access token, authorization server harus menolak token request tersebut dan mencabut (revoke) semua token yang diterbitkan terkait dengan authorization code tersebut.	1
10.4.3	Verifikasi bahwa authorization code memiliki masa berlaku yang singkat (short-lived). Masa berlaku maksimum dapat mencapai 10 menit untuk aplikasi L1 dan L2 serta hingga 1 menit untuk aplikasi L3.	1
10.4.4	Verifikasi bahwa untuk client tertentu, authorization server hanya mengizinkan penggunaan grant yang memang perlu digunakan oleh client tersebut. Perlu dicatat bahwa grant 'token' (Implicit flow) dan 'password' (Resource Owner Password Credentials flow) sudah tidak boleh digunakan lagi.	1
10.4.5	Verifikasi bahwa authorization server memitigasi serangan refresh token replay untuk public client, sebaiknya menggunakan sender-constrained refresh token, yaitu Demonstrating Proof of Possession (DPoP) atau Certificate-Bound Access Token menggunakan mutual TLS (mTLS). Untuk aplikasi L1 dan L2, refresh token rotation dapat digunakan. Jika refresh token rotation digunakan, authorization server harus melakukan invalidasi terhadap refresh token setelah digunakan, dan mencabut semua refresh token untuk authorization tersebut jika sebuah refresh token yang sudah digunakan dan tidak valid diberikan kembali.	1

#	Deskripsi	Level
10.4.6	Verifikasi bahwa, jika code grant digunakan, authorization server memitigasi serangan authorization code interception dengan mewajibkan proof key for code exchange (PKCE). Untuk authorization request, authorization server harus mewajibkan nilai 'code_challenge' yang valid dan tidak boleh menerima nilai 'code_challenge_method*plain'. Untuk token request, authorization server harus mewajibkan validasi parameter 'code_verifier'.	2
10.4.7	Verifikasi bahwa jika authorization server mendukung unauthenticated dynamic client registration, authorization server tersebut memitigasi risiko aplikasi client yang berbahaya. Authorization server harus memvalidasi metadata client seperti URI mana pun yang telah didaftarkan, memastikan persetujuan pengguna, dan memberikan peringatan kepada pengguna sebelum memproses sebuah authorization request dengan aplikasi client yang tidak tepercaya.	2
10.4.8	Verifikasi bahwa refresh token memiliki masa kedaluwarsa mutlak (absolute expiration), termasuk jika sliding refresh token expiration diterapkan.	2
10.4.9	Verifikasi bahwa refresh token dan reference access token dapat dicabut (revoked) oleh pengguna yang berwenang melalui user interface authorization server, guna memitigasi risiko client yang berbahaya atau token yang dicuri.	2
10.4.10	Verifikasi bahwa confidential client diautentikasi untuk client-to-authorized server backchannel requests seperti token requests, pushed authorization requests (PAR), dan token revocation requests.	2
10.4.11	Verifikasi bahwa konfigurasi authorization server hanya memberikan scopes yang diperlukan kepada OAuth client.	2
10.4.12	Verifikasi bahwa untuk client tertentu, authorization server hanya mengizinkan nilai 'response_mode' yang memang perlu digunakan oleh client tersebut. Misalnya, dengan membuat authorization server memvalidasi nilai ini terhadap nilai yang diharapkan atau dengan menggunakan pushed authorization request (PAR) atau JWT-secured Authorization Request (JAR).	3
10.4.13	Verifikasi bahwa grant type 'code' selalu digunakan bersama dengan pushed authorization requests (PAR).	3
10.4.14	Verifikasi bahwa authorization server hanya menerbitkan sender-constrained (Proof-of-Possession) access token, baik menggunakan certificate-bound access token dengan mutual TLS (mTLS) maupun DPoP-bound access token (Demonstration of Proof of Possession).	3

#	Deskripsi	Level
10.4.15	Verifikasi bahwa, untuk sebuah server-side client (yang tidak dieksekusi pada perangkat pengguna akhir), authorization server memastikan bahwa nilai parameter 'authorization_details' berasal dari backend client dan bahwa pengguna tidak telah memanipulasinya (tampered with). Misalnya, dengan mewajibkan penggunaan pushed authorization request (PAR) atau JWT-secured Authorization Request (JAR).	3
10.4.16	Verifikasi bahwa client bersifat confidential dan authorization server mewajibkan penggunaan metode client authentication yang kuat (berbasis kriptografi kunci publik dan tahan terhadap serangan replay), seperti mutual TLS ('tls_client_auth', 'self_signed_tls_client_auth') atau private key JWT ('private_key_jwt').	3

V10.5 OIDC Client

Karena OIDC relying party bertindak sebagai sebuah OAuth client, persyaratan dari bagian "OAuth Client" juga berlaku.

Perlu dicatat bahwa bagian "Authentication with an Identity Provider" pada bab "Authentication" juga memuat persyaratan umum yang relevan.

#	Deskripsi	Level
10.5.1	Verifikasi bahwa client (sebagai relying party) memitigasi serangan ID Token replay. Misalnya, dengan memastikan bahwa claim 'nonce' pada ID Token sesuai dengan nilai 'nonce' yang dikirimkan pada authentication request ke OpenID Provider (dalam OAuth2 disebut sebagai authorization request yang dikirimkan ke authorization server).	2
10.5.2	Verifikasi bahwa client mengidentifikasi pengguna secara unik dari claims pada ID Token, biasanya claim 'sub', yang tidak dapat dipindahtangankan (reassigned) ke pengguna lain (dalam scope sebuah identity provider).	2
10.5.3	Verifikasi bahwa client menolak upaya authorization server yang berbahaya untuk menyamar sebagai authorization server lain melalui metadata authorization server. Client harus menolak metadata authorization server jika URL issuer pada metadata authorization server tersebut tidak sama persis dengan URL issuer yang telah dikonfigurasi sebelumnya (pre-configured) dan diharapkan oleh client.	2

#	Deskripsi	Level
10.5.4	Verifikasi bahwa client memvalidasi bahwa ID Token memang dimaksudkan untuk digunakan oleh client tersebut (audience) dengan memeriksa bahwa claim 'aud' dari token sesuai dengan nilai 'client_id' untuk client tersebut.	2
10.5.5	Verifikasi bahwa, saat menggunakan OIDC back-channel logout, relying party memitigasi denial of service melalui forced logout dan cross-JWT confusion pada alur logout. Client harus memverifikasi bahwa logout token memiliki tipe yang benar dengan nilai 'logout+jwt', mengandung claim 'event' dengan nama member yang benar, dan tidak mengandung claim 'nonce'. Perlu dicatat bahwa juga direkomendasikan untuk memiliki masa kedaluwarsa yang singkat (misalnya, 2 menit).	2

V10.6 OpenID Provider

Karena OpenID Provider bertindak sebagai OAuth authorization server, persyaratan dari bagian "OAuth Authorization Server" juga berlaku.

Perlu dicatat bahwa jika menggunakan ID Token flow (bukan code flow), tidak ada access token yang diterbitkan, dan banyak persyaratan untuk OAuth AS tidak berlaku.

#	Deskripsi	Level
10.6.1	Verifikasi bahwa OpenID Provider hanya mengizinkan nilai 'code', 'code_verifier', 'id_token', atau 'id_token code' untuk response mode. Perlu dicatat bahwa 'code' lebih diutamakan daripada 'id_token code' (OIDC Hybrid flow), dan 'token' (Implicit flow apa pun) tidak boleh digunakan.	2
10.6.2	Verifikasi bahwa OpenID Provider memitigasi denial of service melalui forced logout. Dengan memperoleh konfirmasi eksplisit dari pengguna akhir atau, jika ada, memvalidasi parameter pada logout request (yang diinisiasi oleh relying party), seperti 'id_token_hint'.	2

V10.7 Manajemen Consent

Persyaratan ini membahas verifikasi consent pengguna oleh authorization server. Tanpa verifikasi consent pengguna yang tepat, pelaku jahat dapat memperoleh permission atas nama pengguna melalui spoofing atau social-engineering.

#	Deskripsi	Level
10.7.1	Verifikasi bahwa authorization server memastikan pengguna menyetujui (consent) setiap authorization request. Jika identitas client tidak dapat dipastikan, authorization server harus selalu secara eksplisit meminta consent kepada pengguna.	2
10.7.2	Verifikasi bahwa saat authorization server meminta consent pengguna, authorization server tersebut menyajikan informasi yang cukup dan jelas mengenai apa yang sedang disetujui. Jika berlaku, hal ini harus mencakup sifat dari authorization yang diminta (biasanya berdasarkan scope, resource server, detail authorization Rich Authorization Requests (RAR)), identitas aplikasi yang diberi wewenang, dan masa berlaku dari authorization tersebut.	2
10.7.3	Verifikasi bahwa pengguna dapat meninjau, mengubah, dan mencabut consent yang telah diberikan pengguna melalui authorization server.	2

Referensi

Untuk informasi lebih lanjut mengenai OAuth, silakan lihat:

- oauth.net
- OWASP OAuth 2.0 Protocol Cheat Sheet

Untuk persyaratan terkait OAuth pada ASVS, RFC berikut yang telah dipublikasikan maupun yang masih berstatus draft digunakan:

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens
- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

Untuk informasi lebih lanjut mengenai OpenID Connect, silakan lihat:

- OpenID Connect Core 1.0
- FAPI 2.0 Security Profile

V11 Cryptography

Tujuan Kontrol

Tujuan bab ini adalah untuk mendefinisikan praktik terbaik untuk penggunaan cryptography secara umum, serta menanamkan pemahaman mendasar mengenai prinsip-prinsip kriptografi dan mendorong pergeseran menuju pendekatan yang lebih tangguh dan modern. Bab ini mendorong hal-hal berikut:

- Menerapkan sistem kriptografi yang tangguh, yang gagal secara aman (fail securely), beradaptasi terhadap ancaman yang terus berkembang, dan tahan terhadap perkembangan di masa depan (future-proof).
- Memanfaatkan mekanisme kriptografi yang aman sekaligus selaras dengan praktik terbaik industri.
- Mempertahankan sistem manajemen kunci kriptografi yang aman dengan kontrol akses dan audit yang sesuai.
- Secara berkala mengevaluasi lanskap kriptografi untuk menilai risiko baru dan menyesuaikan algoritma yang digunakan.
- Menemukan dan mengelola kasus penggunaan kriptografi di sepanjang siklus hidup aplikasi guna memastikan bahwa semua aset kriptografi terdata dan terlindungi.

Selain menguraikan prinsip umum dan praktik terbaik, dokumen ini juga menyediakan informasi teknis yang lebih mendalam mengenai persyaratan pada Appendix C - Cryptography Standards. Hal ini mencakup algoritma dan mode yang dianggap “disetujui”(approved) untuk tujuan persyaratan pada bab ini.

Persyaratan yang menggunakan cryptography untuk menyelesaikan masalah yang berbeda, seperti secrets management atau communications security, akan berada pada bagian standar yang berbeda.

V11.1 Inventaris dan Dokumentasi Kriptografi

Aplikasi perlu dirancang dengan arsitektur kriptografi yang kuat untuk melindungi aset data sesuai dengan klasifikasinya. Mengenkripsi segalanya adalah tindakan yang boros; tidak mengenkripsi apa pun adalah kelalaian secara hukum. Sebuah keseimbangan harus dicapai, biasanya selama tahap desain arsitektur atau desain tingkat tinggi, design sprint, atau architectural spike. Merancang cryptography secara “on the fly” atau menambahkannya belakangan (retrofitting) pasti akan memakan

biaya jauh lebih besar untuk diimplementasikan secara aman dibandingkan langsung membangunnya sejak awal.

Penting untuk memastikan bahwa semua aset kriptografi secara rutin ditemukan, diinventarisasi, dan dinilai. Silakan lihat appendix untuk informasi lebih lanjut mengenai bagaimana hal ini dapat dilakukan.

Kebutuhan untuk membuat sistem kriptografi tahan di masa depan (future-proof) terhadap kemunculan quantum computing juga sangat krusial. Post-Quantum Cryptography (PQC) merujuk pada algoritma kriptografi yang dirancang untuk tetap aman terhadap serangan oleh quantum computer, yang diperkirakan akan dapat memecahkan algoritma yang banyak digunakan seperti RSA dan elliptic curve cryptography (ECC).

Silakan lihat appendix untuk panduan terkini mengenai primitif dan standar PQC yang telah diverifikasi (vetted).

#	Deskripsi	Level
11.1.1	Verifikasi bahwa terdapat kebijakan terdokumentasi untuk manajemen kunci kriptografi dan sebuah siklus hidup kunci kriptografi yang mengikuti standar manajemen kunci seperti NIST SP 800-57. Hal ini harus mencakup memastikan bahwa kunci tidak dibagikan secara berlebihan (misalnya, dengan lebih dari dua entitas untuk shared secrets dan lebih dari satu entitas untuk private keys).	2
11.1.2	Verifikasi bahwa sebuah inventaris kriptografi dilakukan, dipelihara, diperbarui secara berkala, dan mencakup semua kunci kriptografi, algoritma, serta sertifikat yang digunakan oleh aplikasi. Inventaris tersebut juga harus mendokumentasikan di mana kunci dapat dan tidak dapat digunakan dalam sistem, serta jenis data yang dapat dan tidak dapat dilindungi menggunakan kunci tersebut.	2
11.1.3	Verifikasi bahwa mekanisme cryptographic discovery diterapkan untuk mengidentifikasi seluruh penggunaan cryptography dalam sistem, termasuk operasi enkripsi, hashing, dan signing.	3
11.1.4	Verifikasi bahwa sebuah inventaris kriptografi dipelihara. Hal ini harus mencakup sebuah rencana terdokumentasi yang menguraikan jalur migrasi menuju standar kriptografi baru, seperti post-quantum cryptography, guna mengantisipasi ancaman di masa depan.	3

V11.2 Implementasi Cryptography yang Aman

Bagian ini mendefinisikan persyaratan untuk pemilihan, implementasi, dan pengelolaan berkelanjutan terhadap algoritma kriptografi inti untuk sebuah aplikasi. Tujuannya adalah untuk

memastikan bahwa hanya primitif kriptografi yang tangguh dan diterima secara industri yang digunakan, selaras dengan standar terkini (misalnya, NIST, ISO/IEC) dan praktik terbaik. Organisasi harus memastikan bahwa setiap komponen kriptografi dipilih berdasarkan bukti yang telah ditinjau sejawat (peer-reviewed) dan pengujian keamanan praktis.

#	Deskripsi	Level
11.2.1	Verifikasi bahwa implementasi yang telah tervalidasi secara industri (termasuk pustaka dan implementasi yang dipercepat oleh perangkat keras) digunakan untuk operasi kriptografi.	2
11.2.2	Verifikasi bahwa aplikasi dirancang dengan crypto agility sedemikian rupa sehingga algoritma random number, authenticated encryption, MAC, atau hashing, panjang kunci, rounds, cipher, dan mode dapat dikonfigurasi ulang, ditingkatkan (upgraded), atau diganti kapan saja, guna melindungi dari pembobolan (breaks) kriptografi. Demikian pula, harus dimungkinkan juga untuk mengganti kunci dan password serta melakukan re-enkripsi data. Hal ini akan memungkinkan peningkatan (upgrade) yang mulus menuju post-quantum cryptography (PQC), setelah implementasi dengan jaminan tinggi (high-assurance) dari skema atau standar PQC yang disetujui tersedia secara luas.	2
11.2.3	Verifikasi bahwa semua primitif kriptografi menggunakan tingkat keamanan minimal 128-bit berdasarkan algoritma, ukuran kunci, dan konfigurasi. Misalnya, sebuah kunci ECC 256-bit memberikan tingkat keamanan kira-kira 128 bit, sedangkan RSA memerlukan kunci 3072-bit untuk mencapai tingkat keamanan 128 bit.	2
11.2.4	Verifikasi bahwa semua operasi kriptografi bersifat constant-time, tanpa operasi 'short-circuit' pada perbandingan, kalkulasi, atau return, guna menghindari kebocoran informasi.	3
11.2.5	Verifikasi bahwa semua modul kriptografi gagal secara aman (fail securely), dan error ditangani dengan cara yang tidak memungkinkan terjadinya kerentanan, seperti serangan Padding Oracle.	3

V11.3 Algoritma Enkripsi

Algoritma authenticated encryption yang dibangun di atas AES dan CHACHA20 menjadi tulang punggung praktik kriptografi modern.

#	Deskripsi	Level
11.3.1	Verifikasi bahwa block mode yang tidak aman (misalnya, ECB) dan skema padding yang lemah (misalnya, PKCS#1 v1.5) tidak digunakan.	1
11.3.2	Verifikasi bahwa hanya cipher dan mode yang disetujui, seperti AES dengan GCM, yang digunakan.	1
11.3.3	Verifikasi bahwa data yang terenkripsi dilindungi dari modifikasi yang tidak sah, sebaiknya dengan menggunakan metode authenticated encryption yang disetujui atau dengan mengombinasikan metode enkripsi yang disetujui dengan algoritma MAC yang disetujui.	2
11.3.4	Verifikasi bahwa nonces, initialization vectors, dan angka sekali pakai (single-use numbers) lainnya tidak digunakan untuk lebih dari satu pasangan kunci enkripsi dan elemen data. Metode pembuatannya harus sesuai untuk algoritma yang digunakan.	3
11.3.5	Verifikasi bahwa kombinasi apa pun antara algoritma enkripsi dan algoritma MAC beroperasi dalam mode encrypt-then-MAC.	3

V11.4 Hashing dan Fungsi Berbasis Hash

Cryptographic hash digunakan dalam berbagai macam protokol kriptografi, seperti digital signatures, HMAC, key derivation functions (KDF), pembuatan bit acak (random bit generation), dan penyimpanan password. Keamanan sistem kriptografi hanya sekuat fungsi hash yang mendasarinya. Bagian ini menguraikan persyaratan untuk menggunakan fungsi hash yang aman dalam operasi kriptografi.

Untuk penyimpanan password, serta appendix kriptografi, OWASP Password Storage Cheat Sheet juga akan memberikan konteks dan panduan yang berguna.

#	Deskripsi	Level
11.4.1	Verifikasi bahwa hanya fungsi hash yang disetujui yang digunakan untuk kasus penggunaan kriptografi secara umum, termasuk digital signatures, HMAC, KDF, dan random bit generation. Fungsi hash yang tidak diizinkan, seperti MD5, tidak boleh digunakan untuk tujuan kriptografi apa pun.	1

#	Deskripsi	Level
11.4.2	Verifikasi bahwa password disimpan menggunakan sebuah key derivation function yang disetujui dan membutuhkan komputasi intensif (juga dikenal sebagai “fungsi hashing password”), dengan pengaturan parameter yang dikonfigurasi berdasarkan panduan terkini. Pengaturan tersebut harus menyeimbangkan antara keamanan dan performa agar serangan brute-force cukup sulit dilakukan sesuai tingkat keamanan yang dibutuhkan.	2
11.4.3	Verifikasi bahwa fungsi hash yang digunakan pada digital signatures, sebagai bagian dari autentikasi data atau integritas data, tahan terhadap collision (collision resistant) dan memiliki panjang bit yang sesuai. Jika ketahanan terhadap collision dibutuhkan, panjang output harus minimal 256 bit. Jika hanya ketahanan terhadap serangan second pre-image yang dibutuhkan, panjang output harus minimal 128 bit.	2
11.4.4	Verifikasi bahwa aplikasi menggunakan key derivation function yang disetujui dengan parameter key stretching saat menurunkan (deriving) secret key dari password. Parameter yang digunakan harus menyeimbangkan antara keamanan dan performa untuk mencegah serangan brute-force membobol kunci kriptografi yang dihasilkan.	2

V11.5 Nilai Acak (Random Values)

Cryptographically secure Pseudo-random Number Generation (CSPRNG) sangat sulit untuk dilakukan dengan benar. Umumnya, sumber entropi yang baik dalam sebuah sistem akan cepat habis jika digunakan secara berlebihan, namun sumber dengan tingkat keacakan yang lebih rendah dapat mengakibatkan kunci dan secret yang dapat diprediksi.

#	Deskripsi	Level
11.5.1	Verifikasi bahwa semua angka dan string acak yang dimaksudkan untuk tidak dapat ditebak (non-guessable) harus dihasilkan menggunakan cryptographically secure pseudo-random number generator (CSPRNG) dan memiliki entropi minimal 128 bit. Perlu dicatat bahwa UUID tidak memenuhi kondisi ini.	2
11.5.2	Verifikasi bahwa mekanisme random number generation yang digunakan dirancang untuk bekerja secara aman, bahkan di bawah permintaan yang tinggi (heavy demand).	3

V11.6 Public Key Cryptography

Public Key Cryptography digunakan pada kondisi di mana tidak memungkinkan atau tidak diinginkan untuk membagikan sebuah secret key antar beberapa pihak.

Sebagai bagian dari hal ini, terdapat kebutuhan akan mekanisme key exchange yang disetujui, seperti Diffie-Hellman dan Elliptic Curve Diffie-Hellman (ECDH), guna memastikan bahwa sistem kriptografi tetap aman terhadap ancaman modern. Bab “Secure Communication” menyediakan persyaratan untuk TLS, sehingga persyaratan pada bagian ini dimaksudkan untuk situasi di mana Public Key Cryptography digunakan pada kasus penggunaan selain TLS.

#	Deskripsi	Level
11.6.1	Verifikasi bahwa hanya algoritma kriptografi dan mode operasi yang disetujui yang digunakan untuk pembuatan dan seeding kunci, serta pembuatan dan verifikasi digital signature. Algoritma pembuatan kunci tidak boleh menghasilkan kunci yang tidak aman dan rentan terhadap serangan yang telah diketahui, misalnya, kunci RSA yang rentan terhadap Fermat factorization.	2
11.6.2	Verifikasi bahwa algoritma kriptografi yang disetujui digunakan untuk key exchange (seperti Diffie-Hellman) dengan fokus untuk memastikan bahwa mekanisme key exchange menggunakan parameter yang aman. Hal ini akan mencegah serangan terhadap proses key establishment yang dapat mengakibatkan serangan adversary-in-the-middle atau pembobolan kriptografi.	3

V11.7 Kriptografi Data yang Sedang Digunakan (In-Use)

Melindungi data selagi sedang diproses sangatlah penting. Teknik seperti full memory encryption, enkripsi data yang sedang ditransmisikan (in transit), dan memastikan data dienkripsi secepat mungkin setelah digunakan sangat direkomendasikan.

#	Deskripsi	Level
11.7.1	Verifikasi bahwa full memory encryption digunakan untuk melindungi data sensitif selagi sedang digunakan, mencegah akses oleh pengguna atau proses yang tidak berwenang.	3
11.7.2	Verifikasi bahwa data minimization memastikan hanya jumlah data minimal yang terekspos selama pemrosesan, dan pastikan data dienkripsi segera setelah digunakan atau sesegera mungkin.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet
- FIPS 140-3
- NIST SP 800-57

V12 Secure Communication

Tujuan Kontrol

Bab ini mencakup persyaratan yang terkait dengan mekanisme spesifik yang harus diterapkan untuk melindungi data yang sedang ditransmisikan (in transit), baik antara client pengguna akhir dan backend service, maupun antara layanan internal dan backend service.

Konsep umum yang didorong oleh bab ini meliputi:

- Memastikan bahwa komunikasi dienkripsi secara eksternal, dan idealnya juga secara internal.
- Mengonfigurasi mekanisme enkripsi menggunakan panduan terkini, termasuk algoritma dan cipher yang direkomendasikan.
- Memastikan bahwa komunikasi tidak diintersep oleh pihak yang tidak berwenang melalui penggunaan sertifikat yang ditandatangani (signed certificates).

Selain menguraikan prinsip umum dan praktik terbaik, ASVS juga menyediakan informasi teknis yang lebih mendalam mengenai kekuatan kriptografi pada Appendix C - Cryptography Standards.

V12.1 Panduan Umum Keamanan TLS

Bagian ini menyediakan panduan awal mengenai cara mengamankan komunikasi TLS. Alat (tools) yang selalu diperbarui harus digunakan untuk meninjau konfigurasi TLS secara berkelanjutan.

Meskipun penggunaan sertifikat TLS wildcard tidak secara inheren tidak aman, kompromi terhadap sebuah sertifikat yang digunakan di seluruh environment yang dimiliki (misalnya, produksi, staging, development, dan test) dapat mengakibatkan kompromi terhadap postur keamanan aplikasi yang menggunakannya. Perlindungan, pengelolaan yang tepat, serta penggunaan sertifikat TLS terpisah pada environment yang berbeda-beda harus diterapkan jika memungkinkan.

#	Deskripsi	Level
12.1.1	Verifikasi bahwa hanya versi terbaru protokol TLS yang direkomendasikan yang diaktifkan, seperti TLS 1.2 dan TLS 1.3. Versi terbaru dari protokol TLS harus menjadi opsi yang diutamakan (preferred).	1
12.1.2	Verifikasi bahwa hanya cipher suite yang direkomendasikan yang diaktifkan, dengan cipher suite terkuat diatur sebagai yang diutamakan (preferred). Aplikasi L3 hanya boleh mendukung cipher suite yang menyediakan forward secrecy.	2
12.1.3	Verifikasi bahwa aplikasi memvalidasi bahwa sertifikat client mTLS tepercaya sebelum menggunakan identitas sertifikat tersebut untuk authentication atau authorization.	2
12.1.4	Verifikasi bahwa mekanisme pencabutan sertifikat (certificate revocation) yang tepat, seperti Online Certificate Status Protocol (OCSP) Stapling, diaktifkan dan dikonfigurasi.	3
12.1.5	Verifikasi bahwa Encrypted Client Hello (ECH) diaktifkan pada pengaturan TLS aplikasi guna mencegah tereksponnya metadata sensitif, seperti Server Name Indication (SNI), selama proses TLS handshake.	3

V12.2 Komunikasi HTTPS dengan Layanan yang Menghadap Eksternal

Pastikan seluruh lalu lintas HTTP menuju layanan yang menghadap eksternal (external-facing) yang diekspos oleh aplikasi dikirimkan secara terenkripsi, dengan sertifikat yang tepercaya secara publik.

#	Deskripsi	Level
12.2.1	Verifikasi bahwa TLS digunakan untuk seluruh konektivitas antara client dan layanan berbasis HTTP yang menghadap eksternal, dan tidak berpindah kembali (fall back) ke komunikasi yang tidak aman atau tidak terenkripsi.	1
12.2.2	Verifikasi bahwa layanan yang menghadap eksternal menggunakan sertifikat TLS yang tepercaya secara publik.	1

V12.3 Keamanan Komunikasi Umum Antar Layanan (Service to Service)

Komunikasi server (baik internal maupun eksternal) melibatkan lebih dari sekadar HTTP. Koneksi menuju dan dari sistem lain juga harus aman, idealnya menggunakan TLS.

#	Deskripsi	Level
12.3.1	Verifikasi bahwa sebuah protokol terenkripsi seperti TLS digunakan untuk seluruh koneksi masuk (inbound) dan keluar (outbound) menuju dan dari aplikasi, termasuk sistem monitoring, alat manajemen, remote access dan SSH, middleware, database, mainframe, sistem partner, atau API eksternal. Server tidak boleh berpindah kembali (fall back) ke protokol yang tidak aman atau tidak terenkripsi.	2
12.3.2	Verifikasi bahwa client TLS memvalidasi sertifikat yang diterima sebelum berkomunikasi dengan server TLS.	2
12.3.3	Verifikasi bahwa TLS atau mekanisme transport encryption lain yang sesuai digunakan untuk seluruh konektivitas antara layanan internal berbasis HTTP dalam aplikasi, dan tidak berpindah kembali (fall back) ke komunikasi yang tidak aman atau tidak terenkripsi.	2
12.3.4	Verifikasi bahwa koneksi TLS antar layanan internal menggunakan sertifikat yang tepercaya. Ketika sertifikat yang dihasilkan secara internal atau self-signed digunakan, layanan yang mengonsumsi (consuming service) harus dikonfigurasi untuk hanya mempercayai CA internal tertentu dan sertifikat self-signed tertentu.	2
12.3.5	Verifikasi bahwa layanan yang berkomunikasi secara internal dalam suatu sistem (intra-service communications) menggunakan strong authentication untuk memastikan bahwa setiap endpoint terverifikasi. Metode strong authentication, seperti TLS client authentication, harus diterapkan untuk memastikan identitas, menggunakan public-key infrastructure dan mekanisme yang tahan terhadap serangan replay. Untuk arsitektur microservice, pertimbangkan penggunaan sebuah service mesh untuk menyederhanakan pengelolaan sertifikat dan meningkatkan keamanan.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP - Transport Layer Security Cheat Sheet
- Panduan konfigurasi Server Side TLS dari Mozilla
- Alat dari Mozilla untuk menghasilkan konfigurasi TLS yang telah dikenal baik.
- O-Saft - Proyek OWASP untuk memvalidasi konfigurasi TLS

V13 Configuration

Tujuan Kontrol

Konfigurasi default aplikasi harus aman untuk digunakan di Internet.

Bab ini menyediakan panduan mengenai berbagai konfigurasi yang diperlukan untuk mencapai hal tersebut, termasuk konfigurasi yang diterapkan selama development, build, dan deployment.

Topik yang dibahas mencakup pencegahan kebocoran data, pengelolaan komunikasi antar komponen secara aman, dan perlindungan secrets.

V13.1 Dokumentasi Konfigurasi

Bagian ini menguraikan persyaratan dokumentasi mengenai bagaimana aplikasi berkomunikasi dengan layanan internal dan eksternal, serta teknik untuk mencegah hilangnya ketersediaan (availability) akibat layanan yang tidak dapat diakses. Bagian ini juga membahas dokumentasi yang terkait dengan secrets.

#	Deskripsi	Level
13.1.1	Verifikasi bahwa semua kebutuhan komunikasi untuk aplikasi terdokumentasi. Hal ini harus mencakup layanan eksternal yang diandalkan oleh aplikasi dan kasus-kasus di mana pengguna akhir mungkin dapat memberikan sebuah lokasi eksternal yang kemudian akan dihubungi oleh aplikasi.	2
13.1.2	Verifikasi bahwa untuk setiap layanan yang digunakan aplikasi, dokumentasi mendefinisikan jumlah maksimum koneksi paralel (concurrent) (misalnya, batas connection pool) dan bagaimana perilaku aplikasi ketika batas tersebut tercapai, termasuk mekanisme fallback atau recovery apa pun, guna mencegah kondisi denial of service.	3
13.1.3	Verifikasi bahwa dokumentasi aplikasi mendefinisikan strategi manajemen resource untuk setiap sistem atau layanan eksternal yang digunakannya (misalnya, database, file handles, thread, koneksi HTTP). Hal ini harus mencakup prosedur pelepasan resource (resource-release), pengaturan timeout, penanganan kegagalan, dan jika logika retry diterapkan, penentuan batas retry, delay, dan algoritma back-off. Untuk operasi request-response HTTP yang bersifat sinkron, dokumentasi harus mewajibkan timeout yang singkat dan menonaktifkan retry atau membatasi retry secara ketat guna mencegah keterlambatan yang berantai (cascading delays) dan habisnya resource.	3

#	Deskripsi	Level
13.1.4	Verifikasi bahwa dokumentasi aplikasi mendefinisikan secrets yang krusial bagi keamanan aplikasi serta jadwal rotasinya, berdasarkan model ancaman (threat model) organisasi dan kebutuhan bisnis.	3

V13.2 Konfigurasi Komunikasi Backend

Aplikasi berinteraksi dengan berbagai layanan, termasuk API, database, atau komponen lainnya. Komponen-komponen ini dapat dianggap internal bagi aplikasi namun tidak termasuk dalam mekanisme access control standar aplikasi, atau dapat sepenuhnya bersifat eksternal. Dalam kedua kasus tersebut, aplikasi perlu dikonfigurasi untuk berinteraksi secara aman dengan komponen-komponen ini dan, jika diperlukan, melindungi konfigurasi tersebut.

Catatan: Bab “Secure Communication” menyediakan panduan untuk enkripsi data yang sedang ditransmisikan (in transit).

#	Deskripsi	Level
13.2.1	Verifikasi bahwa komunikasi antar komponen aplikasi backend yang tidak mendukung mekanisme session pengguna standar aplikasi, termasuk API, middleware, dan data layer, telah diautentikasi. Autentikasi harus menggunakan akun layanan (service accounts) individual, token jangka pendek, atau autentikasi berbasis sertifikat, dan bukan kredensial yang tidak berubah seperti password, API key, atau akun bersama (shared accounts) dengan akses istimewa (privileged access).	2
13.2.2	Verifikasi bahwa komunikasi antar komponen aplikasi backend, termasuk layanan lokal atau sistem operasi, API, middleware, dan data layer, dilakukan dengan akun yang diberikan hak akses (privilege) minimal yang diperlukan.	2
13.2.3	Verifikasi bahwa jika sebuah kredensial harus digunakan untuk autentikasi layanan, kredensial yang digunakan oleh consumer bukan merupakan kredensial default (misalnya, root/root atau admin/admin).	2
13.2.4	Verifikasi bahwa sebuah allowlist digunakan untuk mendefinisikan resource atau sistem eksternal yang diizinkan untuk dihubungi oleh aplikasi (misalnya, untuk outbound requests, pemuatan data, atau akses file). Allowlist ini dapat diterapkan pada application layer, web server, firewall, atau kombinasi dari beberapa lapisan yang berbeda.	2
13.2.5	Verifikasi bahwa web server atau application server dikonfigurasi dengan sebuah allowlist resource atau sistem yang dapat dikirim request atau tempat pemuatan data atau file oleh server tersebut.	2

#	Deskripsi	Level
13.2.6	Verifikasi bahwa ketika aplikasi terhubung ke layanan terpisah, aplikasi tersebut mengikuti konfigurasi terdokumentasi untuk setiap koneksi, seperti jumlah maksimum koneksi paralel, perilaku ketika jumlah maksimum koneksi yang diizinkan tercapai, timeout koneksi, dan strategi retry.	3

V13.3 Manajemen Secret

Manajemen secret merupakan tugas konfigurasi yang esensial untuk memastikan perlindungan data yang digunakan dalam aplikasi. Persyaratan spesifik untuk cryptography dapat ditemukan pada bab “Cryptography”, namun bagian ini berfokus pada aspek pengelolaan dan penanganan secrets.

#	Deskripsi	Level
13.3.1	Verifikasi bahwa sebuah solusi secrets management, seperti sebuah key vault, digunakan untuk membuat, menyimpan, mengontrol akses ke, dan menghapus secrets backend secara aman. Hal ini dapat mencakup password, key material, integrasi dengan database dan sistem pihak ketiga, key dan seed untuk token berbasis waktu, secrets internal lainnya, dan API key. Secrets tidak boleh disertakan dalam kode sumber aplikasi atau disertakan dalam build artifacts. Untuk aplikasi L3, hal ini harus melibatkan sebuah solusi yang didukung oleh perangkat keras (hardware-backed) seperti HSM.	2
13.3.2	Verifikasi bahwa akses terhadap aset secret mematuhi prinsip least privilege.	2
13.3.3	Verifikasi bahwa semua operasi kriptografi dilakukan menggunakan sebuah modul keamanan yang terisolasi (seperti sebuah vault atau hardware security module) guna mengelola dan melindungi key material secara aman dari paparan di luar modul keamanan tersebut.	3
13.3.4	Verifikasi bahwa secrets dikonfigurasi untuk kedaluwarsa dan dirotasi berdasarkan dokumentasi aplikasi.	3

V13.4 Kebocoran Informasi yang Tidak Diinginkan

Konfigurasi produksi harus diperkuat (hardened) guna menghindari terungkapnya data yang tidak diperlukan. Banyak dari masalah ini jarang dinilai sebagai risiko signifikan, namun sering kali dirangkai (chained) dengan kerentanan lainnya. Jika masalah-masalah ini tidak muncul secara default, hal tersebut akan meningkatkan tingkat kesulitan (raises the bar) untuk menyerang sebuah aplikasi.

Misalnya, menyembunyikan versi komponen server-side tidak menghilangkan kebutuhan untuk melakukan patch pada semua komponen, dan menonaktifkan folder listing tidak menghilangkan kebutuhan untuk menggunakan kontrol authorization atau menjauhkan file dari folder publik, namun hal tersebut meningkatkan tingkat kesulitan bagi penyerang.

#	Deskripsi	Level
13.4.1	Verifikasi bahwa aplikasi di-deploy tanpa metadata source control apa pun, termasuk folder .git atau .svn, atau dengan cara sedemikian rupa sehingga folder-folder ini tidak dapat diakses baik secara eksternal maupun oleh aplikasi itu sendiri.	1
13.4.2	Verifikasi bahwa mode debug dinonaktifkan untuk semua komponen pada environment produksi guna mencegah terekspornya fitur debugging dan kebocoran informasi.	2
13.4.3	Verifikasi bahwa web server tidak mengeksport directory listing kepada client kecuali memang secara eksplisit dimaksudkan demikian.	2
13.4.4	Verifikasi bahwa penggunaan metode HTTP TRACE tidak didukung pada environment produksi, guna menghindari potensi kebocoran informasi.	2
13.4.5	Verifikasi bahwa dokumentasi (seperti untuk API internal) dan endpoint monitoring tidak terekspos kecuali memang secara eksplisit dimaksudkan demikian.	2
13.4.6	Verifikasi bahwa aplikasi tidak mengeksport informasi versi terperinci dari komponen backend.	3
13.4.7	Verifikasi bahwa web tier dikonfigurasi untuk hanya menyajikan file dengan ekstensi file tertentu guna mencegah kebocoran informasi, konfigurasi, dan kode sumber yang tidak disengaja.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 Data Protection

Tujuan Kontrol

Aplikasi tidak dapat memperhitungkan seluruh pola penggunaan dan perilaku pengguna, sehingga perlu menerapkan kontrol untuk membatasi akses tidak sah terhadap data sensitif pada perangkat

client.

Bab ini mencakup persyaratan yang terkait dengan mendefinisikan data apa yang perlu dilindungi, bagaimana data tersebut harus dilindungi, serta mekanisme spesifik yang perlu diterapkan atau jebakan (pitfalls) yang perlu dihindari.

Pertimbangan lain untuk perlindungan data adalah ekstraksi massal (bulk extraction), modifikasi, atau penggunaan yang berlebihan. Persyaratan setiap sistem kemungkinan akan sangat berbeda-beda, sehingga penentuan apa yang dianggap “tidak wajar” (abnormal) harus mempertimbangkan model ancaman (threat model) dan risiko bisnis. Dari sudut pandang ASVS, pendeteksian masalah-masalah ini ditangani pada bab “Security Logging and Error Handling”, dan penentuan batasan ditangani pada bab “Validation and Business Logic”.

V14.1 Dokumentasi Perlindungan Data

Prasyarat utama untuk dapat melindungi data adalah mengategorikan data apa yang harus dianggap sensitif. Kemungkinan akan terdapat beberapa tingkat sensitivitas yang berbeda, dan untuk setiap tingkat, kontrol yang diperlukan untuk melindungi data pada tingkat tersebut akan berbeda-beda.

Terdapat berbagai regulasi dan undang-undang privasi yang memengaruhi bagaimana aplikasi harus melakukan pendekatan terhadap penyimpanan, penggunaan, dan transmisi informasi pribadi yang sensitif. Bagian ini tidak lagi berupaya menduplikasi jenis-jenis undang-undang perlindungan data atau privasi tersebut, melainkan berfokus pada pertimbangan teknis utama untuk melindungi data sensitif. Silakan berkonsultasi dengan hukum dan regulasi setempat, serta berkonsultasi dengan spesialis privasi atau pengacara yang berkualifikasi sesuai kebutuhan.

#	Deskripsi	Level
14.1.1	Verifikasi bahwa semua data sensitif yang dibuat dan diproses oleh aplikasi telah diidentifikasi dan diklasifikasikan ke dalam tingkat perlindungan (protection levels). Hal ini mencakup data yang hanya di-encode dan karenanya mudah di-decode, seperti string Base64 atau plaintext payload di dalam sebuah JWT. Tingkat perlindungan perlu mempertimbangkan regulasi dan standar perlindungan data serta privasi yang wajib dipatuhi oleh aplikasi.	2
14.1.2	Verifikasi bahwa semua tingkat perlindungan data sensitif memiliki serangkaian persyaratan perlindungan yang terdokumentasi. Hal ini harus mencakup (namun tidak terbatas pada) persyaratan yang terkait dengan enkripsi secara umum, verifikasi integritas, retensi, bagaimana data tersebut dicatat (logged), kontrol akses terhadap data sensitif dalam log, enkripsi pada level database, privasi dan teknologi peningkatan privasi (privacy-enhancing technologies) yang akan digunakan, serta persyaratan kerahasiaan lainnya.	2

V14.2 Perlindungan Data Secara Umum

Bagian ini berisi berbagai persyaratan praktis yang terkait dengan perlindungan data. Sebagian besar bersifat spesifik terhadap masalah tertentu seperti kebocoran data yang tidak disengaja, namun juga terdapat persyaratan umum untuk menerapkan kontrol perlindungan berdasarkan tingkat perlindungan yang dibutuhkan untuk setiap item data.

#	Deskripsi	Level
14.2.1	Verifikasi bahwa data sensitif hanya dikirimkan ke server melalui HTTP message body atau header fields, dan bahwa URL serta query string tidak mengandung informasi sensitif, seperti sebuah API key atau session token.	1
14.2.2	Verifikasi bahwa aplikasi mencegah data sensitif di-cache pada komponen server, seperti load balancer dan application cache, atau memastikan bahwa data tersebut dihapus (purged) secara aman setelah digunakan.	2
14.2.3	Verifikasi bahwa data sensitif yang telah didefinisikan tidak dikirimkan ke pihak yang tidak tepercaya (misalnya, user trackers) guna mencegah pengumpulan data yang tidak diinginkan di luar kendali aplikasi.	2
14.2.4	Verifikasi bahwa kontrol terhadap data sensitif yang terkait dengan enkripsi, verifikasi integritas, retensi, bagaimana data tersebut dicatat (logged), kontrol akses terhadap data sensitif dalam log, serta privasi dan teknologi peningkatan privasi, diterapkan sebagaimana didefinisikan dalam dokumentasi untuk tingkat perlindungan data tertentu.	2
14.2.5	Verifikasi bahwa mekanisme caching dikonfigurasi untuk hanya menyimpan cache response yang memiliki content type yang sesuai untuk resource tersebut dan tidak mengandung konten dinamis yang sensitif. Web server harus mengembalikan response 404 atau 302 ketika sebuah file yang tidak ada diakses, alih-alih mengembalikan file lain yang valid. Hal ini seharusnya mencegah serangan Web Cache Deception.	3
14.2.6	Verifikasi bahwa aplikasi hanya mengembalikan data sensitif minimal yang diperlukan untuk fungsionalitas aplikasi. Misalnya, hanya mengembalikan sebagian digit dari sebuah nomor kartu kredit, bukan nomor lengkapnya. Jika data lengkap memang diperlukan, data tersebut harus disamarkan (masked) pada user interface kecuali pengguna secara khusus melihatnya.	3
14.2.7	Verifikasi bahwa informasi sensitif tunduk pada klasifikasi retensi data, memastikan bahwa data yang sudah usang atau tidak diperlukan dihapus secara otomatis, sesuai jadwal yang telah ditentukan, atau sesuai kebutuhan situasi.	3

#	Deskripsi	Level
14.2.8	Verifikasi bahwa informasi sensitif dihapus dari metadata file yang dikirimkan oleh pengguna, kecuali penyimpanan telah disetujui (consented) oleh pengguna tersebut.	3

V14.3 Perlindungan Data di Sisi Client

Bagian ini berisi persyaratan untuk mencegah kebocoran data dengan cara tertentu pada sisi client atau user agent dari sebuah aplikasi.

#	Deskripsi	Level
14.3.1	Verifikasi bahwa data yang terautentikasi dihapus dari penyimpanan client, seperti browser DOM, setelah client atau sesi dihentikan. HTTP response header field 'Clear-Site-Data' dapat membantu hal ini, namun sisi client juga harus mampu membersihkan data tersebut jika koneksi ke server tidak tersedia saat sesi dihentikan.	1
14.3.2	Verifikasi bahwa aplikasi mengatur HTTP response header fields anti-caching yang memadai (yaitu, Cache-Control: no-store) sehingga data sensitif tidak di-cache pada browser.	2
14.3.3	Verifikasi bahwa data yang disimpan pada browser storage (seperti localStorage, sessionStorage, IndexedDB, atau cookies) tidak mengandung data sensitif, dengan pengecualian untuk session token.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- Pertimbangkan menggunakan situs Security Headers untuk memeriksa header fields keamanan dan anti-caching
- Dokumentasi mengenai header anti-caching oleh Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- Ikhtisar European Union General Data Protection Regulation (GDPR)
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Informasi mengenai header "Clear-Site-Data"
- White paper mengenai Web Cache Deception

V15 Secure Coding dan Architecture

Tujuan Kontrol

Banyak persyaratan ASVS yang terkait dengan area keamanan tertentu, seperti authentication atau authorization, atau berkaitan dengan jenis fungsionalitas aplikasi tertentu, seperti logging atau file handling.

Bab ini menyediakan persyaratan keamanan umum yang perlu dipertimbangkan saat merancang dan mengembangkan aplikasi. Persyaratan ini tidak hanya berfokus pada arsitektur yang bersih (clean architecture) dan kualitas kode, tetapi juga pada praktik arsitektur dan coding tertentu yang diperlukan untuk keamanan aplikasi.

V15.1 Dokumentasi Secure Coding dan Arsitektur

Banyak persyaratan untuk membangun arsitektur yang aman dan dapat dipertahankan (defensible) bergantung pada dokumentasi yang jelas mengenai keputusan yang dibuat terkait implementasi kontrol keamanan tertentu dan komponen yang digunakan dalam aplikasi.

Bagian ini menguraikan persyaratan dokumentasi, termasuk mengidentifikasi komponen yang dianggap mengandung “dangerous functionality” atau merupakan “risky component”.

Sebuah komponen dengan “dangerous functionality” dapat berupa komponen yang dikembangkan secara internal atau komponen pihak ketiga yang melakukan operasi seperti deserialization terhadap data tidak tepercaya, parsing raw file atau data biner, dynamic code execution, atau manipulasi memory secara langsung. Kerentanan pada jenis operasi ini menimbulkan risiko tinggi terhadap kompromi aplikasi dan berpotensi mengekspos infrastruktur yang mendasarinya.

Sebuah “risky component” adalah sebuah pustaka pihak ketiga (yaitu, tidak dikembangkan secara internal) yang tidak memiliki atau memiliki kontrol keamanan yang buruk seputar proses pengembangan atau fungsionalitasnya. Contohnya mencakup komponen yang kurang terpelihara (poorly maintained), tidak lagi didukung (unsupported), berada pada tahap end-of-life, atau memiliki riwayat kerentanan yang signifikan.

Bagian ini juga menekankan pentingnya mendefinisikan jangka waktu yang sesuai untuk mengatasi kerentanan pada komponen pihak ketiga.

#	Deskripsi	Level
15.1.1	Verifikasi bahwa dokumentasi aplikasi mendefinisikan jangka waktu remediasi berbasis risiko untuk versi komponen pihak ketiga yang memiliki kerentanan dan untuk pembaruan pustaka secara umum, guna meminimalkan risiko dari komponen-komponen tersebut.	1

#	Deskripsi	Level
15.1.2	Verifikasi bahwa sebuah katalog inventaris, seperti software bill of materials (SBOM), dipelihara untuk semua pustaka pihak ketiga yang digunakan, termasuk memverifikasi bahwa komponen-komponen tersebut berasal dari repository yang telah ditentukan sebelumnya, tepercaya, dan terus terpelihara.	2
15.1.3	Verifikasi bahwa dokumentasi aplikasi mengidentifikasi fungsionalitas yang memakan waktu lama atau memerlukan resource besar. Hal ini harus mencakup cara mencegah hilangnya ketersediaan (availability) akibat penggunaan berlebihan terhadap fungsionalitas ini dan cara menghindari situasi di mana pembuatan sebuah response memakan waktu lebih lama daripada timeout consumer. Pertahanan yang mungkin dapat mencakup pemrosesan asinkron, penggunaan queue, dan pembatasan proses paralel per pengguna dan per aplikasi.	2
15.1.4	Verifikasi bahwa dokumentasi aplikasi menyoroti pustaka pihak ketiga yang dianggap sebagai “risky component”.	3
15.1.5	Verifikasi bahwa dokumentasi aplikasi menyoroti bagian-bagian aplikasi di mana “dangerous functionality” digunakan.	3

V15.2 Arsitektur Keamanan dan Dependensi

Bagian ini mencakup persyaratan untuk menangani dependensi dan komponen yang berisiko, usang, atau tidak aman melalui dependency management.

Bagian ini juga mencakup penggunaan teknik pada level arsitektur seperti sandboxing, encapsulation, containerization, dan network isolation untuk mengurangi dampak penggunaan “dangerous operations” atau “risky component” (sebagaimana didefinisikan pada bagian sebelumnya) serta mencegah hilangnya ketersediaan (availability) akibat penggunaan berlebihan terhadap fungsionalitas yang memerlukan resource besar.

#	Deskripsi	Level
15.2.1	Verifikasi bahwa aplikasi hanya mengandung komponen yang belum melanggar jangka waktu pembaruan dan remediasi yang terdokumentasi.	1
15.2.2	Verifikasi bahwa aplikasi telah menerapkan pertahanan terhadap hilangnya ketersediaan (availability) akibat fungsionalitas yang memakan waktu lama atau memerlukan resource besar, berdasarkan keputusan keamanan dan strategi terdokumentasi untuk hal ini.	2

#	Deskripsi	Level
15.2.3	Verifikasi bahwa environment produksi hanya mencakup fungsionalitas yang diperlukan agar aplikasi dapat berfungsi, dan tidak mengekspos fungsionalitas yang tidak diperlukan seperti kode test, contoh cuplikan kode (sample snippets), dan fungsionalitas development.	2
15.2.4	Verifikasi bahwa komponen pihak ketiga beserta seluruh dependensi transitifnya disertakan dari repository yang diharapkan, baik yang dimiliki secara internal maupun sumber eksternal, dan bahwa tidak ada risiko serangan dependency confusion.	3
15.2.5	Verifikasi bahwa aplikasi menerapkan perlindungan tambahan pada bagian-bagian aplikasi yang terdokumentasi mengandung “dangerous functionality” atau menggunakan pustaka pihak ketiga yang dianggap sebagai “risky component”. Hal ini dapat mencakup teknik seperti sandboxing, encapsulation, containerization, atau network level isolation untuk menghambat dan mencegah penyerang yang telah mengompromikan satu bagian aplikasi agar tidak dapat berpindah (pivoting) ke bagian lain dalam aplikasi.	3

V15.3 Defensive Coding

Bagian ini membahas jenis-jenis kerentanan, termasuk type juggling, prototype pollution, dan lainnya, yang muncul akibat penggunaan pola coding yang tidak aman pada bahasa tertentu. Beberapa mungkin tidak relevan untuk semua bahasa, sementara yang lain memiliki perbaikan yang spesifik terhadap bahasa tertentu atau terkait dengan bagaimana suatu bahasa atau framework tertentu menangani sebuah fitur seperti HTTP parameter. Bagian ini juga mempertimbangkan risiko tidak melakukan validasi kriptografis terhadap pembaruan aplikasi.

Bagian ini juga mempertimbangkan risiko yang terkait dengan penggunaan objek untuk merepresentasikan item data serta menerima dan mengembalikan data tersebut melalui API eksternal. Dalam hal ini, aplikasi harus memastikan bahwa field data yang seharusnya tidak dapat ditulis (writable) tidak dimodifikasi oleh input pengguna (mass assignment) dan bahwa API tersebut selektif mengenai field data mana yang dikembalikan. Jika akses field bergantung pada permission pengguna, hal ini harus dipertimbangkan dalam konteks persyaratan field-level access control pada bab Authorization.

#	Deskripsi	Level
15.3.1	Verifikasi bahwa aplikasi hanya mengembalikan subset field yang diperlukan dari sebuah objek data. Misalnya, aplikasi tidak boleh mengembalikan seluruh objek data, karena beberapa field individual seharusnya tidak dapat diakses oleh pengguna.	1
15.3.2	Verifikasi bahwa ketika backend aplikasi melakukan pemanggilan ke URL eksternal, aplikasi tersebut dikonfigurasi untuk tidak mengikuti redirect kecuali hal tersebut memang merupakan fungsionalitas yang dimaksudkan.	2
15.3.3	Verifikasi bahwa aplikasi memiliki penanggulangan (countermeasures) untuk melindungi dari serangan mass assignment dengan membatasi field yang diizinkan per controller dan action, misalnya, tidak memungkinkan untuk menyisipkan atau memperbarui nilai sebuah field ketika hal tersebut tidak dimaksudkan menjadi bagian dari action tersebut.	2
15.3.4	Verifikasi bahwa semua komponen proxying dan middleware meneruskan alamat IP asli pengguna secara benar menggunakan field data tepercaya yang tidak dapat dimanipulasi oleh pengguna akhir, dan aplikasi serta web server menggunakan nilai yang benar ini untuk logging dan keputusan keamanan seperti rate limiting, dengan mempertimbangkan bahwa bahkan alamat IP asli mungkin tidak dapat diandalkan akibat IP dinamis, VPN, atau firewall perusahaan.	2
15.3.5	Verifikasi bahwa aplikasi secara eksplisit memastikan bahwa variabel memiliki tipe yang benar dan melakukan operasi strict equality dan comparator. Hal ini untuk menghindari kerentanan type juggling atau type confusion yang disebabkan oleh kode aplikasi yang membuat asumsi mengenai tipe sebuah variabel.	2
15.3.6	Verifikasi bahwa kode JavaScript ditulis dengan cara yang mencegah prototype pollution, misalnya, dengan menggunakan Set() atau Map() alih-alih object literal.	2
15.3.7	Verifikasi bahwa aplikasi memiliki pertahanan terhadap serangan HTTP parameter pollution, terutama jika application framework tidak membedakan sumber dari request parameter (query string, body parameters, cookies, atau header fields).	2

V15.4 Concurrency yang Aman

Masalah concurrency seperti race conditions, kerentanan time-of-check to time-of-use (TOCTOU), deadlock, livelock, thread starvation, dan sinkronisasi yang tidak tepat dapat mengakibatkan per-

ilaku yang tidak dapat diprediksi dan risiko keamanan. Bagian ini mencakup berbagai teknik dan strategi untuk membantu memitigasi risiko-risiko tersebut.

#	Deskripsi	Level
15.4.1	Verifikasi bahwa objek bersama (shared objects) dalam kode multi-threaded (seperti cache, file, atau objek in-memory yang diakses oleh beberapa thread) diakses secara aman dengan menggunakan tipe yang thread-safe dan mekanisme sinkronisasi seperti locks atau semaphores guna menghindari race conditions dan kerusakan data.	3
15.4.2	Verifikasi bahwa pemeriksaan terhadap state sebuah resource, seperti keberadaannya atau permission-nya, dan tindakan yang bergantung padanya dilakukan sebagai satu operasi atomik guna mencegah race conditions time-of-check to time-of-use (TOCTOU). Misalnya, memeriksa apakah sebuah file ada sebelum membukanya, atau memverifikasi akses pengguna sebelum memberikannya.	3
15.4.3	Verifikasi bahwa locks digunakan secara konsisten guna menghindari thread yang terjebak (stuck), baik karena saling menunggu satu sama lain maupun melakukan retry tanpa henti, dan bahwa logika locking tetap berada dalam kode yang bertanggung jawab mengelola resource tersebut guna memastikan lock tidak dapat dimodifikasi secara tidak sengaja atau secara jahat oleh class atau kode eksternal.	3
15.4.4	Verifikasi bahwa kebijakan alokasi resource mencegah thread starvation dengan memastikan akses yang adil terhadap resource, seperti dengan memanfaatkan thread pool, yang memungkinkan thread dengan prioritas lebih rendah untuk tetap berjalan dalam jangka waktu yang wajar.	3

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 Security Logging dan Error Handling

Tujuan Kontrol

Security log berbeda dari log error atau performa dan digunakan untuk mencatat kejadian (events) yang relevan dengan keamanan, seperti keputusan otentikasi, keputusan kontrol akses, serta upaya untuk melompati (bypass) kontrol keamanan seperti validasi input atau validasi logika bisnis. Tujuannya adalah untuk mendukung deteksi, respons, dan investigasi dengan menyediakan data terstruktur yang ber-signal tinggi (high-signal) untuk alat analisis seperti SIEM.

Log tidak boleh mencakup data pribadi yang sensitif kecuali diwajibkan secara hukum, dan setiap data yang dicatat harus dilindungi sebagai aset bernilai tinggi. Logging tidak boleh mengompromikan privasi atau keamanan sistem. Aplikasi juga harus mengalami kegagalan secara aman (fail securely), menghindari pengungkapan informasi atau gangguan yang tidak perlu.

Untuk panduan implementasi terperinci, lihat OWASP Cheat Sheets di bagian referensi.

V16.1 Dokumentasi Security Logging

Bagian ini memastikan inventarisasi logging yang jelas dan lengkap di seluruh stack aplikasi. Hal ini sangat penting untuk pemantauan keamanan yang efektif, respons insiden, dan kepatuhan (compliance).

#	Deskripsi	Level
16.1.1	Verifikasi bahwa terdapat inventaris yang mendokumentasikan logging yang dilakukan pada setiap lapisan technology stack aplikasi, event apa saja yang dicatat, format log, lokasi penyimpanan log tersebut, bagaimana log digunakan, bagaimana akses ke log dikontrol, dan berapa lama log disimpan.	2

V16.2 General Logging

Bagian ini memberikan persyaratan untuk memastikan bahwa security log terstruktur secara konsisten dan berisi metadata yang diharapkan. Tujuannya adalah membuat log dapat dibaca oleh mesin (machine-readable) dan dapat dianalisis di seluruh sistem dan tools terdistribusi.

Secara alami, event keamanan sering kali melibatkan data sensitif. Jika data tersebut dicatat tanpa pertimbangan, log itu sendiri akan menjadi terklasifikasi dan oleh karena itu tunduk pada persyaratan enkripsi, kebijakan retensi yang lebih ketat, serta potensi pengungkapan selama audit.

Oleh karena itu, sangat penting untuk mencatat hanya apa yang diperlukan dan memperlakukan data log dengan tingkat kepedulian yang sama seperti aset sensitif lainnya.

Persyaratan di bawah ini menetapkan fondasi dasar untuk metadata logging, sinkronisasi, format, dan kontrol.

#	Deskripsi	Level
16.2.1	Verifikasi bahwa setiap entry log mencakup metadata yang diperlukan (seperti kapan, di mana, siapa, apa) yang memungkinkan investigasi mendalam terhadap garis waktu (timeline) saat terjadi suatu event.	2
16.2.2	Verifikasi bahwa sumber waktu untuk semua komponen logging tersinkronisasi, dan timestamp pada metadata event keamanan menggunakan UTC atau menyertakan offset zona waktu yang eksplisit. UTC direkomendasikan untuk memastikan konsistensi di seluruh sistem terdistribusi dan mencegah kebingungan selama transisi daylight saving time.	2
16.2.3	Verifikasi bahwa aplikasi hanya menyimpan atau menyiarkan (broadcast) log ke file dan layanan yang terdokumentasi dalam inventaris log.	2
16.2.4	Verifikasi bahwa log dapat dibaca dan dikorelasikan oleh pemroses log (log processor) yang digunakan, sebaiknya dengan menggunakan format logging umum.	2
16.2.5	Verifikasi bahwa saat mencatat data sensitif, aplikasi menerapkan logging berdasarkan tingkat perlindungan data tersebut. Sebagai contoh, data tertentu seperti kredensial atau rincian pembayaran mungkin tidak diizinkan untuk dicatat. Data lain seperti token sesi hanya boleh dicatat dengan di-hash atau di-masking, baik secara penuh maupun sebagian.	2

V16.3 Security Events

Bagian ini mendefinisikan persyaratan untuk mencatat event yang relevan dengan keamanan di dalam aplikasi. Menangkap event ini sangat penting untuk mendeteksi perilaku mencurigakan, mendukung investigasi, dan memenuhi kewajiban kepatuhan.

Bagian ini menguraikan jenis event yang harus dicatat tetapi tidak mencoba memberikan rincian yang menyeluruh. Setiap aplikasi memiliki faktor risiko dan konteks operasional yang unik.

Harap dicatat bahwa meskipun ASVS mencakup pencatatan event keamanan dalam cakupannya, pengalihan peringatan (*alerting*) dan korelasi (misalnya aturan SIEM atau infrastruktur pemantauan) dianggap di luar cakupan dan ditangani oleh sistem operasional dan pemantauan.

#	Deskripsi	Level
16.3.1	Verifikasi bahwa semua operasi otentikasi dicatat, termasuk percobaan yang berhasil dan yang gagal. Metadata tambahan, seperti jenis otentikasi atau faktor yang digunakan, juga harus dikumpulkan.	2
16.3.2	Verifikasi bahwa percobaan otorisasi yang gagal dicatat. Untuk L3, ini harus mencakup pencatatan semua keputusan otorisasi, termasuk mencatat saat data sensitif diakses (tanpa mencatat data sensitif itu sendiri).	2
16.3.3	Verifikasi bahwa aplikasi mencatat event keamanan yang didefinisikan dalam dokumentasi dan juga mencatat upaya untuk melompati (<i>bypass</i>) kontrol keamanan, seperti validasi input, logika bisnis, dan anti-otomatisasi.	2
16.3.4	Verifikasi bahwa aplikasi mencatat error yang tidak terduga dan kegagalan kontrol keamanan seperti kegagalan backend TLS.	2

V16.4 Log Protection

Log adalah artefak forensik yang berharga dan harus dilindungi. Jika log dapat diubah atau dihapus dengan mudah, log tersebut kehilangan integritasnya dan menjadi tidak dapat diandalkan untuk investigasi insiden atau proses hukum. Log dapat mengekspos perilaku internal aplikasi atau metadata sensitif, menjadikannya target yang menarik bagi penyerang.

Bagian ini mendefinisikan persyaratan untuk memastikan bahwa log dilindungi dari akses yang tidak sah, manipulasi, dan pengungkapan, serta ditransmisikan dan disimpan secara aman di dalam sistem yang aman dan terisolasi.

#	Deskripsi	Level
16.4.1	Verifikasi bahwa semua komponen logging melakukan encode data dengan tepat untuk mencegah log injection.	2
16.4.2	Verifikasi bahwa log dilindungi dari akses yang tidak sah dan tidak dapat diubah.	2
16.4.3	Verifikasi bahwa log ditransmisikan secara aman ke sistem terpisah secara logis untuk analisis, deteksi, alerting, dan eskalasi. Tujuannya adalah untuk memastikan bahwa jika aplikasi dibobol, log tidak ikut terkompromikan.	2

V16.5 Error Handling

Bagian ini mendefinisikan persyaratan untuk memastikan bahwa aplikasi mengalami kegagalan secara baik (*graceful*) dan aman tanpa mengungkapkan rincian internal yang sensitif.

#	Deskripsi	Level
16.5.1	Verifikasi bahwa pesan generik dikembalikan ke pengguna/konsumen saat terjadi error yang tidak terduga atau sensitif terhadap keamanan, memastikan tidak ada paparan data sistem internal yang sensitif seperti stack trace, query, secret key, dan token.	2
16.5.2	Verifikasi bahwa aplikasi tetap beroperasi secara aman ketika akses sumber daya eksternal gagal, misalnya dengan menggunakan pola seperti circuit breaker atau pemulihan secara bertahap (<i>graceful degradation</i>).	2
16.5.3	Verifikasi bahwa aplikasi mengalami kegagalan secara <i>graceful</i> dan aman, termasuk saat terjadi pengecualian (<i>exception</i>), mencegah kondisi <i>fail-open</i> seperti memproses transaksi meskipun terjadi error akibat logika validasi.	2
16.5.4	Verifikasi bahwa error handler “last resort”(upaya terakhir) telah ditentukan untuk menangkap semua pengecualian (<i>unhandled exceptions</i>). Ini bertujuan untuk menghindari kehilangan rincian error yang harus masuk ke file log dan memastikan bahwa error tidak menjatuhkan seluruh proses aplikasi, yang mengakibatkan hilangnya ketersediaan (<i>availability</i>).	3

Catatan: Bahasa tertentu, (termasuk Swift, Go, dan melalui praktik desain umum, banyak bahasa fungsional,) tidak mendukung *exceptions* atau *last-resort event handlers*. Dalam hal ini, arsitek dan pengembang harus menggunakan pola, bahasa, atau cara yang ramah *framework* untuk memastikan bahwa aplikasi dapat menangani event luar biasa, tidak terduga, atau terkait keamanan secara aman.

Referensi

Untuk informasi lebih lanjut, lihat juga:

- OWASP Web Security Testing Guide: Testing for Error Handling
- OWASP Authentication Cheat Sheet section about error messages
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

Tujuan Kontrol

Web Real-Time Communication (WebRTC) memungkinkan pertukaran suara, video, dan data secara *real-time* dalam aplikasi modern. Seiring dengan peningkatan adopsinya, mengamankan

infrastruktur WebRTC menjadi sangat krusial. Bagian ini menyediakan persyaratan keamanan bagi para pemangku kepentingan yang mengembangkan, meng-host, atau mengintegrasikan sistem WebRTC.

Pasar WebRTC secara garis besar dapat dikategorikan menjadi tiga segmen:

1. **Product Developers:** Vendor *proprietary* dan *open-source* yang membuat dan menyuplai produk serta solusi WebRTC. Fokus mereka adalah mengembangkan teknologi WebRTC yang tangguh dan aman yang dapat digunakan oleh pihak lain.
2. **Communication Platforms as a Service (CPaaS):** Penyedia yang menawarkan API, SDK, serta infrastruktur atau platform yang diperlukan untuk mengaktifkan fungsionalitas WebRTC. Penyedia CPaaS dapat menggunakan produk dari kategori pertama atau mengembangkan perangkat lunak WebRTC mereka sendiri untuk menawarkan layanan ini.
3. **Service Providers:** Organisasi yang memanfaatkan produk dari pengembang produk atau penyedia CPaaS, atau mengembangkan solusi WebRTC mereka sendiri. Mereka membuat dan mengimplementasikan aplikasi untuk konferensi *online*, layanan kesehatan, *e-learning*, dan domain lainnya di mana komunikasi *real-time* sangat penting.

Persyaratan keamanan yang diuraikan di sini utamanya difokuskan pada Pengembang Produk, CPaaS, dan Penyedia Layanan yang:

- Memanfaatkan solusi *open-source* untuk membangun aplikasi WebRTC mereka.
- Menggunakan produk WebRTC komersial sebagai bagian dari infrastruktur mereka.
- Menggunakan solusi WebRTC yang dikembangkan secara internal atau mengintegrasikan berbagai komponen menjadi satu penawaran layanan yang kohesif.

Penting untuk dicatat bahwa persyaratan keamanan ini tidak berlaku bagi pengembang yang secara eksklusif menggunakan SDK dan API yang disediakan oleh vendor CPaaS. Bagi pengembang tersebut, penyedia CPaaS biasanya bertanggung jawab atas sebagian besar masalah keamanan yang mendasari platform mereka, dan standar keamanan generik seperti ASVS mungkin tidak sepenuhnya memenuhi kebutuhan mereka.

V17.1 TURN Server

Bagian ini mendefinisikan persyaratan keamanan untuk sistem yang mengoperasikan server TURN (Traversal Using Relays around NAT) milik mereka sendiri. Server TURN membantu meneruskan (*relay*) media dalam lingkungan jaringan yang terbatas, tetapi dapat menimbulkan risiko jika salah dikonfigurasi. Kontrol ini berfokus pada penyaringan alamat yang aman dan perlindungan terhadap kehabisan sumber daya (*resource exhaustion*).

#	Deskripsi	Level
17.1.1	Verifikasi bahwa layanan Traversal Using Relays around NAT (TURN) hanya mengizinkan akses ke alamat IP yang tidak dipesan untuk tujuan khusus (misalnya, jaringan internal, <i>broadcast</i> , <i>loopback</i>). Perhatikan bahwa ini berlaku untuk alamat IPv4 dan IPv6.	2
17.1.2	Verifikasi bahwa layanan Traversal Using Relays around NAT (TURN) tidak rentan terhadap kehabisan sumber daya (<i>resource exhaustion</i>) ketika pengguna yang sah mencoba membuka sejumlah besar <i>port</i> pada server TURN.	3

V17.2 Media

Persyaratan ini hanya berlaku untuk sistem yang meng-host server media WebRTC milik mereka sendiri, seperti Selective Forwarding Units (SFUs), Multipoint Control Units (MCUs), server perekaman, atau server gateway. Server media menangani dan mendistribusikan *stream* media, menjadikan keamanannya sangat krusial untuk melindungi komunikasi antar *peer*. Mengamankan *stream* media sangat penting dalam aplikasi WebRTC untuk mencegah penyadapan (*eavesdropping*), manipulasi data (*tampering*), dan serangan *Denial-of-Service* (DoS) yang dapat mengompromikan privasi pengguna dan kualitas komunikasi.

Secara khusus, sangat penting untuk mengimplementasikan perlindungan terhadap *flood attack* seperti *rate limiting*, memvalidasi *timestamp*, menggunakan jam yang tersinkronisasi untuk mencocokkan interval *real-time*, dan mengelola *buffer* untuk mencegah *overflow* serta menjaga pewaktuan yang tepat. Jika paket untuk sesi media tertentu tiba terlalu cepat, paket berlebih harus dibuang (*dropped*). Penting juga untuk melindungi sistem dari paket yang rusak (*malformed*) dengan menerapkan validasi input, menangani *integer overflow* secara aman, mencegah *buffer overflow*, dan menggunakan teknik penanganan error yang tangguh lainnya.

Sistem yang hanya mengandalkan komunikasi media *peer-to-peer* antar peramban web, tanpa keterlibatan server media perantara, dikecualikan dari persyaratan keamanan terkait media khusus ini.

Bagian ini mengacu pada penggunaan Datagram Transport Layer Security (DTLS) dalam konteks WebRTC. Persyaratan yang berkaitan dengan kepemilikan kebijakan terdokumentasi untuk pengelolaan kunci kriptografi dapat ditemukan pada bab “Kriptografi”. Informasi tentang metode kriptografi yang disetujui dapat ditemukan dalam Lampiran Kriptografi ASVS atau dalam dokumen seperti NIST SP 800-52 Rev. 2 atau BSI TR-02102-2 (Versi 2025-01).

#	Deskripsi	Level
17.2.1	Verifikasi bahwa kunci untuk sertifikat Datagram Transport Layer Security (DTLS) dikelola dan dilindungi berdasarkan kebijakan terdokumentasi untuk pengelolaan kunci kriptografi.	2
17.2.2	Verifikasi bahwa server media dikonfigurasi untuk menggunakan dan mendukung <i>cipher suites</i> Datagram Transport Layer Security (DTLS) yang disetujui serta <i>protection profile</i> yang aman untuk DTLS Extension dalam menetapkan kunci bagi Secure Real-time Transport Protocol (DTLS-SRTP).	2
17.2.3	Verifikasi bahwa otentikasi Secure Real-time Transport Protocol (SRTP) diperiksa pada server media untuk mencegah serangan injeksi Real-time Transport Protocol (RTP) yang dapat menyebabkan kondisi <i>Denial of Service</i> atau penyisipan media audio atau video ke dalam <i>stream</i> media.	2
17.2.4	Verifikasi bahwa server media mampu melanjutkan pemrosesan lalu lintas media yang masuk ketika menemui paket Secure Real-time Transport Protocol (SRTP) yang cacat (<i>malformed</i>).	2
17.2.5	Verifikasi bahwa server media mampu melanjutkan pemrosesan lalu lintas media yang masuk selama terjadinya lonjakan paket (<i>flood</i>) Secure Real-time Transport Protocol (SRTP) dari pengguna yang sah.	3
17.2.6	Verifikasi bahwa server media tidak rentan terhadap kerentanan <i>Race Condition</i> “ClientHello” pada Datagram Transport Layer Security (DTLS) dengan memeriksa apakah server media secara publik diketahui rentan atau dengan melakukan pengujian <i>race condition</i> .	3
17.2.7	Verifikasi bahwa setiap mekanisme perekaman audio atau video yang terkait dengan server media mampu melanjutkan pemrosesan lalu lintas media yang masuk selama terjadinya lonjakan paket (<i>flood</i>) Secure Real-time Transport Protocol (SRTP) dari pengguna yang sah.	3
17.2.8	Verifikasi bahwa sertifikat Datagram Transport Layer Security (DTLS) diperiksa terhadap atribut <i>fingerprint</i> Session Description Protocol (SDP), dan menghentikan <i>stream</i> media jika pemeriksaan gagal, untuk memastikan keaslian (<i>authenticity</i>) dari <i>stream</i> media.	3

V17.3 Signaling

Bagian ini mendefinisikan persyaratan untuk sistem yang mengoperasikan server *signaling* WebRTC milik mereka sendiri. *Signaling* mengoordinasikan komunikasi *peer-to-peer* dan harus tangguh terhadap serangan yang dapat mengganggu pembuatan atau kontrol sesi.

Untuk memastikan *signaling* yang aman, sistem harus menangani input yang cacat (*malformed*) den-

gan baik (*gracefully*) dan tetap tersedia di bawah beban kerja yang tinggi.

#	Deskripsi	Level
17.3.1	Verifikasi bahwa server <i>signaling</i> mampu melanjutkan pemrosesan pesan <i>signaling</i> masuk yang sah selama serangan <i>flood</i> . Hal ini harus dicapai dengan mengimplementasikan <i>rate limiting</i> pada tingkat <i>signaling</i> .	2
17.3.2	Verifikasi bahwa server <i>signaling</i> mampu melanjutkan pemrosesan pesan <i>signaling</i> yang sah ketika menemui pesan <i>signaling</i> yang cacat (<i>malformed</i>) yang dapat menyebabkan kondisi <i>Denial of Service</i> . Ini dapat mencakup pengimplementasian validasi input, penanganan <i>integer overflow</i> secara aman, pencegahan <i>buffer overflow</i> , dan penggunaan teknik penanganan error yang tangguh lainnya.	2

Referensi

Untuk informasi lebih lanjut, lihat juga:

- WebRTC DTLS ClientHello DoS didokumentasikan paling baik pada postingan blog Enable Security yang ditujukan untuk profesional keamanan dan white paper terkait yang ditujukan untuk pengembang WebRTC
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)
- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

Lampiran A: Glosarium

- **Absolute Maximum Session Lifetime** –Juga disebut sebagai “Overall Timeout” oleh NIST, ini adalah jumlah waktu maksimal suatu sesi dapat tetap aktif setelah otentikasi tanpa memandang interaksi pengguna. Ini merupakan komponen dari kedaluwarsa sesi (*session expiration*).
- **Allowlist** –Daftar data atau operasi yang diizinkan, misalnya daftar karakter yang diizinkan untuk melakukan validasi input.
- **Anti-forgery token** –Sebuah mekanisme di mana satu atau lebih token diteruskan dalam sebuah *request* dan divalidasi oleh server aplikasi untuk memastikan bahwa *request* tersebut berasal dari *endpoint* yang diharapkan.

- **Application Security** –Keamanan tingkat aplikasi berfokus pada analisis komponen-komponen yang membentuk lapisan aplikasi pada Open Systems Interconnection Reference Model (Model OSI), alih-alih berfokus pada sistem operasi yang mendasarinya atau jaringan yang terhubung.
- **Application Security Verification** –Penilaian teknis terhadap suatu aplikasi berdasarkan OWASP ASVS.
- **Application Security Verification Report** –Laporan yang mendokumentasikan hasil keseluruhan dan analisis pendukung yang dihasilkan oleh verifikator untuk aplikasi tertentu.
- **Authentication** –Verifikasi atas klaim identitas dari seorang pengguna aplikasi.
- **Automated Verification** –Penggunaan *tools* otomatis (baik alat analisis dinamis, alat analisis statis, atau keduanya) yang menggunakan tanda tangan kerentanan (*vulnerability signatures*) untuk menemukan masalah.
- **Black box testing** –Metode pengujian perangkat lunak yang memeriksa fungsionalitas aplikasi tanpa melihat ke dalam struktur atau kerja internalnya.
- **Common Weakness Enumeration (CWE)** –Daftar kelemahan keamanan perangkat lunak umum yang dikembangkan oleh komunitas. Ini berfungsi sebagai bahasa umum, tolok ukur untuk *tools* keamanan perangkat lunak, dan garis dasar untuk upaya identifikasi, mitigasi, serta pencegahan kelemahan.
- **Component** –Unit kode mandiri (*self-contained*), dengan antarmuka disk dan jaringan terkait yang berkomunikasi dengan komponen lain.
- **Credential Service Provider (CSP)** –Juga disebut sebagai Identity Provider (IdP). Sumber data pengguna yang dapat digunakan sebagai sumber otentikasi oleh aplikasi lain.
- **Cross-Site Script Inclusion (XSSI)** – Varian dari serangan Cross-Site Scripting (XSS) di mana aplikasi web mengambil kode berbahaya dari sumber daya eksternal dan menyertakan kode tersebut sebagai bagian dari kontennya sendiri.
- **Cross-Site Scripting (XSS)** –Kerentanan keamanan yang biasanya ditemukan dalam aplikasi web yang memungkinkan injeksi skrip di sisi klien (*client-side scripts*) ke dalam konten.
- **Cryptographic module** –Perangkat keras, perangkat lunak, dan/atau *firmware* yang mengimplementasikan algoritma kriptografi dan/atau menghasilkan kunci kriptografi.
- **Cryptographically secure pseudo-random number generator (CSPRNG)** – Pembangkit angka acak semu dengan sifat-sifat yang membuatnya cocok untuk digunakan dalam kriptografi, juga disebut sebagai *cryptographic random number generator (CRNG)*.
- **Datagram Transport Layer Security (DTLS)** –Protokol kriptografi yang menyediakan keamanan komunikasi melalui koneksi jaringan. Didasarkan pada protokol TLS tetapi disesuaikan untuk melindungi protokol berorientasi datagram (biasanya melalui UDP). Didefinisikan dalam RFC 9147 untuk DTLS 1.3.
- **Datagram Transport Layer Security Extension to Establish Keys for the Secure Real-time Transport Protocol (DTLS-SRTP)** –Mekanisme penggunaan *handshake* DTLS untuk menetapkan materi kunci (*key material*) bagi sesi SRTP. Didefinisikan dalam RFC 5764.
- **Design Verification** –Penilaian teknis terhadap arsitektur keamanan suatu aplikasi.
- **Dynamic Application Security Testing (DAST)** –Teknologi yang dirancang untuk mendeteksi

kondisi yang mengindikasikan kerentanan keamanan dalam aplikasi yang sedang berjalan (*running state*).

- **Dynamic Verification** –Penggunaan *tools* otomatis yang memanfaatkan *vulnerability signatures* untuk menemukan masalah selama eksekusi aplikasi.
- **Fast IDentity Online (FIDO)** –Serangkaian standar otentikasi yang memungkinkan berbagai metode otentikasi berbeda untuk digunakan termasuk biometrik, Trusted Platform Modules (TPMs), token keamanan USB, dll.
- **Hardware Security Module (HSM)** –Komponen perangkat keras yang menyimpan kunci kriptografi dan rahasia lainnya dengan cara yang terlindungi.
- **Hibernate Query Language (HQL)** –Bahasa kueri yang tampilannya mirip dengan SQL yang digunakan oleh pustaka Hibernate ORM.
- **HTTP Strict Transport Security (HSTS)** –Kebijakan yang menginstruksikan peramban untuk hanya terhubung ke domain yang mengembalikan *header* via TLS dan saat sertifikat yang valid ditunjukkan. Diaktifkan menggunakan bidang respons header Strict-Transport-Security.
- **HyperText Transfer Protocol (HTTP)** –Protokol aplikasi untuk sistem informasi hipermedia yang terdistribusi dan kolaboratif. Merupakan fondasi komunikasi data untuk World Wide Web.
- **HyperText Transfer Protocol over SSL/TLS (HTTPS)** –Metode mengamankan komunikasi HTTP dengan mengenskripsikannya menggunakan Transport Layer Security (TLS).
- **Identity Provider (IdP)** –Juga disebut sebagai Credential Service Provider (CSP) dalam referensi NIST. Entitas yang menyediakan sumber otentikasi untuk aplikasi lain.
- **Inactivity Timeout** –Jangka waktu di mana suatu sesi dapat tetap aktif tanpa adanya interaksi pengguna dengan aplikasi. Ini merupakan komponen dari kedaluwarsa sesi (*session expiration*).
- **Input Validation** –Kanonisasi dan validasi dari input pengguna yang tidak terpercaya (*untrusted user input*).
- **JSON Web Token (JWT)** –RFC 7519 mendefinisikan standar untuk objek data JSON yang terdiri dari bagian *header* (menjelaskan cara memvalidasi objek), bagian *body* (berisi kumpulan klaim), dan bagian *signature* (berisi tanda tangan digital yang digunakan untuk memvalidasi isi bagian *body*). Merupakan jenis *self-contained token*.
- **Local File Inclusion (LFI)** –Serangan yang mengeksploitasi prosedur pemuatan file (*file inclusion*) yang rentan pada aplikasi, yang mengarah pada pemuatan file lokal yang sudah ada di server.
- **Malicious Code** –Kode yang dimasukkan ke dalam aplikasi selama pengembangannya tanpa sepengetahuan pemilik aplikasi, yang mengabaikan kebijakan keamanan yang dimaksudkan dari aplikasi. Tidak sama dengan *malware* seperti virus atau *worm*!
- **Malware** –Kode yang dapat dieksekusi yang dimasukkan ke dalam aplikasi selama *runtime* tanpa sepengetahuan pengguna atau administrator aplikasi.
- **Message authentication code (MAC)** –Checksum kriptografi pada data, yang dihitung dengan algoritma pembuatan MAC, yang digunakan untuk memberikan jaminan atas integritas dan keasliannya.
- **Multi-factor authentication (MFA)** –Otentikasi yang mencakup dua atau lebih faktor tunggal

(*single factors*).

- **Mutual TLS** (mTLS) –Lihat *TLS client authentication*.
- **Object-relational Mapping** (ORM) –Sistem yang digunakan untuk memungkinkan basis data relasional/berbasis tabel dirujuk dan dikueri di dalam program aplikasi menggunakan model objek yang kompatibel dengan aplikasi.
- **One-time Password** (OTP) –Kata sandi yang dihasilkan secara unik untuk digunakan pada satu kesempatan saja.
- **Open Worldwide Application Security Project** (OWASP) –Open Worldwide Application Security Project (OWASP) adalah komunitas bebas dan terbuka di seluruh dunia yang berfokus pada peningkatan keamanan perangkat lunak aplikasi. Misi kami adalah membuat keamanan aplikasi “terlihat,” sehingga individu dan organisasi dapat membuat keputusan yang tepat tentang risiko keamanan aplikasi. Lihat: <https://www.owasp.org/>.
- **Password-Based Key Derivation Function 2** (PBKDF2) –Algoritma satu arah khusus yang digunakan untuk membuat kunci kriptografi yang kuat dari teks input (seperti kata sandi) dan nilai acak tambahan (*salt*), sehingga dapat digunakan untuk mempersulit pembongkaran kata sandi secara luring (*offline*) jika nilai hasilnya disimpan sebagai pengganti kata sandi asli.
- **Public Key Infrastructure** (PKI) –Pengaturan yang mengikat kunci publik dengan identitas entitas masing-masing. Pengikatan ini dibentuk melalui proses registrasi dan penerbitan sertifikat pada dan oleh otoritas sertifikat (*Certificate Authority/CA*).
- **Public Switched Telephone Network** (PSTN) –Jaringan telepon tradisional yang mencakup telepon kabel (*fixed-line*) dan telepon seluler.
- **Real-time Transport Protocol** (RTP) dan **Real-time Transport Control Protocol** (RTCP) –Dua protokol yang digunakan secara berpasangan untuk mentransportasikan *stream* multimedia. Digunakan oleh *stack* WebRTC. Didefinisikan dalam RFC 3550.
- **Reference Token** –Jenis token yang bertindak sebagai petunjuk (*pointer*) atau pengenalan ke *state* atau metadata yang disimpan di server, terkadang disebut sebagai *random tokens* atau *opaque tokens*. Berbeda dengan *self-contained tokens* yang menyematkan beberapa data relevannya di dalam token itu sendiri, *reference token* tidak berisi informasi intrinsik, melainkan mengandalkan server untuk konteksnya. *Reference token* akan berupa atau berisi pengenalan sesi (*session ID*).
- **Relying Party** (RP) –Umumnya aplikasi yang mengandalkan pengguna yang telah terotentikasi pada penyedia otentikasi terpisah. Aplikasi mengandalkan sejenis token atau kumpulan asersi bertanda tangan yang disediakan oleh penyedia otentikasi tersebut untuk mempercayai bahwa pengguna adalah siapa yang mereka klaim.
- **Remote File Inclusion** (RFI) - Serangan yang mengeksploitasi prosedur pemuatan (*inclusion*) yang rentan pada aplikasi, yang mengakibatkan dimasukkannya file dari jarak jauh (*remote files*).
- **Scalable Vector Graphics** (SVG) –Bahasa markup berbasis XML untuk menggambarkan grafik vektor dua dimensi.
- **Secure Real-time Transport Protocol** (SRTP) dan **Secure Real-time Transport Control Protocol** (SRTCP) –Profil dari protokol RTP dan RTCP yang menyediakan dukungan untuk enkripsi

pesan, otentikasi, dan perlindungan integritas. Didefinisikan dalam RFC 3711.

- **Security Architecture** –Abstraksi dari desain aplikasi yang mengidentifikasi dan menjelaskan di mana serta bagaimana kontrol keamanan digunakan, dan juga mengidentifikasi serta menjelaskan lokasi dan sensitivitas data pengguna maupun aplikasi.
- **Security Assertion Markup Language (SAML)** –Standar terbuka untuk otentikasi *single sign-on* berbasis penerusan asersi bertanda tangan (biasanya objek XML) antara *identity provider* dan *relying party*.
- **Security Configuration** –Konfigurasi *runtime* dari suatu aplikasi yang memengaruhi cara kontrol keamanan digunakan.
- **Security Control** –Fungsi atau komponen yang melakukan pemeriksaan keamanan (misalnya pemeriksaan otorisasi) atau saat dipanggil menghasilkan efek keamanan (misalnya menghasilkan catatan audit).
- **Security information and event management (SIEM)** - Sistem untuk deteksi ancaman, kepatuhan, dan manajemen insiden keamanan melalui pengumpulan dan analisis data terkait keamanan dari berbagai sumber di dalam infrastruktur IT organisasi.
- **Self-Contained Token** –Token yang mengapsulkan satu atau lebih atribut yang tidak mengandalkan *state* di sisi server atau penyimpanan eksternal lainnya. Token ini memastikan keaslian dan integritas atribut yang terkandung di dalamnya, memungkinkan pertukaran informasi “stateless” yang aman di seluruh sistem. *Self-contained token* umumnya diamankan menggunakan teknik kriptografi, seperti tanda tangan digital atau *message authentication codes* (MAC), untuk memastikan keaslian, integritas, dan dalam beberapa kasus kerahasiaan datanya. Contoh umumnya meliputi Asersi SAML dan JWT.
- **Server-side Request Forgery (SSRF)** –Serangan yang menyalahgunakan fungsionalitas pada server untuk membaca atau memperbarui sumber daya internal. Penyerang menyuplai atau memodifikasi URL, yang akan dibaca atau dikirimkan datanya oleh kode yang berjalan di server.
- **Session Description Protocol (SDP)** –Format pesan untuk mengatur sesi multimedia (digunakan misalnya dalam WebRTC). Didefinisikan dalam RFC 4566.
- **Session Identifier** atau **Session ID** –Kunci yang mengidentifikasi sesi *stateful* yang disimpan di *back-end*. Akan ditransfer ke dan dari klien baik sebagai atau di dalam *Reference Token*.
- **Session Token** –Frasa “cakup-semua” (*catch-all*) yang digunakan dalam standar ini untuk merujuk pada token atau nilai yang digunakan dalam mekanisme sesi *stateless* (yang menggunakan *self-contained token*) atau mekanisme sesi *stateful* (yang menggunakan *reference token*).
- **Session Traversal Utilities for NAT (STUN)** –Protokol yang digunakan untuk membantu penjelajahan NAT (*NAT traversal*) guna membangun komunikasi *peer-to-peer*. Didefinisikan dalam RFC 3489.
- **Single-factor authenticator** –Mekanisme untuk memeriksa bahwa pengguna telah terotentikasi. Harus berupa sesuatu yang Anda ketahui (*memorized secrets*, kata sandi, frasa sandi, PIN), sesuatu yang merupakan diri Anda (biometrik, sidik jari, pemindaian wajah), atau sesuatu yang Anda miliki (token OTP, perangkat kriptografi seperti *smart card*).
- **Single Sign-on Authentication (SSO)** –Terjadi ketika pengguna masuk (*login*) ke satu aplikasi

dan kemudian secara otomatis masuk ke aplikasi lain tanpa harus melakukan otentikasi ulang. Misalnya, saat masuk ke Google, pengguna akan secara otomatis masuk ke layanan Google lainnya seperti YouTube, Google Docs, dan Gmail.

- **Software bill of materials (SBOM)** - Daftar terstruktur dan komprehensif dari semua komponen, modul, pustaka, kerangka kerja, dan sumber daya lain yang diperlukan untuk membangun atau merakit aplikasi perangkat lunak.
- **Software Composition Analysis (SCA)** -Serangkaian teknologi yang dirancang untuk menganalisis komposisi aplikasi, dependensi, pustaka, dan paket untuk kerentanan keamanan dari versi komponen spesifik yang digunakan. Ini tidak boleh disamakan dengan analisis *source-code* yang sekarang biasa disebut sebagai SAST.
- **Software development lifecycle (SDLC)** -Proses langkah-demi-langkah di mana perangkat lunak dikembangkan dari persyaratan awal hingga penggelaran (*deployment*) dan pemeliharaan.
- **SQL Injection (SQLi)** -Teknik injeksi kode yang digunakan untuk menyerang aplikasi berbasis data, di mana pernyataan SQL berbahaya dimasukkan ke dalam titik masuk (*entry point*).
- **Stateful Session Mechanism** -Dalam mekanisme sesi *stateful*, aplikasi mempertahankan *session state* pada *backend* yang biasanya sesuai dengan *session token*, yang dihasilkan menggunakan *cryptographically secure pseudo-random number generator (CSPRNG)*, yang diterbitkan kepada pengguna akhir.
- **Stateless Session Mechanism** -Mekanisme sesi *stateless* akan menggunakan *self-contained token* yang diteruskan ke klien, dan berisi informasi sesi yang tidak harus disimpan di dalam layanan yang kemudian menerima dan memvalidasi token tersebut. Pada kenyataannya, layanan perlu memiliki akses ke beberapa informasi sesi (seperti daftar pencabutan JWT) agar dapat menegakkan kontrol keamanan yang diperlukan.
- **Static application security testing (SAST)** -Serangkaian teknologi yang dirancang untuk menganalisis kode sumber aplikasi (*source code*), *byte code*, dan biner untuk kondisi pengodean dan desain yang mengindikasikan kerentanan keamanan. Solusi SAST menganalisis aplikasi dari “dalam ke luar” dalam kondisi tidak berjalan (*nonrunning state*).
- **Threat Modeling** -Teknik yang terdiri dari pengembangan arsitektur keamanan yang semakin diperhalus untuk mengidentifikasi *threat agents*, *security zones*, *security controls*, serta aset teknis dan bisnis yang penting.
- **Time-of-check to time-of-use (TOCTOU)** -Situasi di mana aplikasi memeriksa kondisi (*state*) suatu sumber daya sebelum menggunakan sumber daya tersebut, tetapi kondisi sumber daya tersebut dapat berubah antara pemeriksaan dan penggunaan. Hal ini dapat membatalkan hasil pemeriksaan dan menyebabkan situasi di mana aplikasi melakukan tindakan yang tidak valid karena ketidakcocokan kondisi tersebut.
- **Time based One-time Passwords (TOTPs)** - Metode menghasilkan OTP di mana waktu saat ini bertindak sebagai bagian dari algoritma untuk menghasilkan kata sandi.
- **TLS client authentication**, juga disebut **Mutual TLS (mTLS)** -Dalam koneksi TLS standar, klien dapat menggunakan sertifikat yang disediakan oleh server untuk memvalidasi identitas server. Jika otentikasi klien TLS digunakan, klien juga menggunakan kunci privat dan sertifikatnya sendiri untuk memungkinkan server memvalidasi identitas klien tersebut.

- **Transport Layer Security (TLS)** –Protokol kriptografi yang menyediakan keamanan komunikasi melalui koneksi jaringan.
- **Traversal Using Relays around NAT (TURN)** –Ekstensi dari protokol STUN yang menggunakan server TURN sebagai *relay* ketika koneksi *peer-to-peer* langsung tidak dapat dibangun. Didefinisikan dalam RFC 8656.
- **Trusted execution environment (TEE)** - Lingkungan pemrosesan terisolasi di mana aplikasi dapat dieksekusi secara aman terlepas dari sisa sistem lainnya.
- **Trusted Platform Module (TPM)** –Jenis HSM yang biasanya terpasang pada komponen perangkat keras yang lebih besar seperti *motherboard* dan bertindak sebagai “root of trust” untuk sistem tersebut.
- **Trusted Service Layer** –Setiap titik penegakan kontrol yang terpercaya, seperti *microservice*, *serverless API*, *server-side*, API terpercaya pada perangkat klien yang memiliki *secure boot*, API mitra atau eksternal, dan sebagainya. *Trusted* (terpercaya) berarti tidak ada kekhawatiran bahwa pengguna yang tidak terpercaya akan dapat melompati (*bypass*) atau mengabaikan lapisan atau kontrol yang diimplementasikan pada lapisan tersebut.
- **Uniform Resource Identifier (URI)** - String karakter unik yang mengidentifikasi sumber daya, seperti halaman web, alamat email, tempat.
- **Uniform Resource Locator (URL)** –String yang menentukan lokasi sumber daya di Internet.
- **Universally Unique Identifier (UUID)** –Nomor referensi unik yang digunakan sebagai pengenalan dalam perangkat lunak.
- **Verifier** –Orang atau tim yang meninjau aplikasi terhadap persyaratan OWASP ASVS.
- **Web Real-Time Communication (WebRTC)** –*Stack* protokol dan API web terkait yang digunakan untuk transportasi *stream* multimedia dalam aplikasi web, biasanya dalam konteks telekonferensi. Berdasarkan SRTP, SRTCP, DTLS, SDP, dan STUN/TURN.
- **WebSocket over TLS (WSS)** –Praktik mengamankan komunikasi WebSocket dengan melapiskan WebSocket di atas protokol TLS.
- **What You See Is What You Get (WYSIWYG)** –Jenis editor konten kaya (*rich content editor*) yang menunjukkan bagaimana konten sebenarnya akan terlihat saat dirender alih-alih menampilkan pengodean yang digunakan untuk mengatur rendering tersebut.
- **X.509 Certificate** –Sertifikat digital yang menggunakan standar infrastruktur kunci publik (PKI) X.509 yang diterima secara luas secara internasional untuk memverifikasi bahwa kunci publik milik pengguna, komputer, atau identitas layanan terkandung dalam sertifikat tersebut.
- **XML eXternal Entity (XXE)** –Jenis entitas XML yang dapat mengakses konten lokal atau jarak jauh melalui pengenalan sistem yang dideklarasikan. Ini dapat menyebabkan berbagai serangan injeksi.

Lampiran B: Referensi

Proyek-proyek OWASP berikut ini kemungkinan besar akan bermanfaat bagi pengguna/pengadopsi standar ini:

Proyek Inti OWASP

1. OWASP Top 10 Project: <https://owasp.org/www-project-top-ten/>
2. OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
3. OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
4. OWASP Software Assurance Maturity Model (SAMM): <https://owasp.org/www-project-samm/>
5. OWASP Secure Headers Project: <https://owasp.org/www-project-secure-headers/>

Proyek Cheat Sheet OWASP

Proyek ini memiliki beberapa cheat sheet yang relevan dengan berbagai topik dalam ASVS.

Terdapat pemetaan ke ASVS yang dapat ditemukan di sini: <https://cheatsheetseries.owasp.org/IndexASVS.html>

Proyek-proyek Terkait Keamanan Mobile

1. OWASP Mobile Security Project: <https://owasp.org/www-project-mobile-security/>
2. OWASP Mobile Top 10 Risks: <https://owasp.org/www-project-mobile-top-10/>
3. OWASP Mobile Security Testing Guide and Mobile Application Security Verification Standard: <https://owasp.org/www-project-mobile-security-testing-guide/>

Proyek-proyek terkait Internet of Things OWASP

1. OWASP Internet of Things Project: <https://owasp.org/www-project-internet-of-things/>

Proyek OWASP Serverless

1. OWASP Serverless Project: <https://owasp.org/www-project-serverless-top-10/>

Yang lain

Demikian pula, situs web berikut ini kemungkinan besar akan bermanfaat bagi pengguna/pengadopsi standar ini.

1. SecLists Github: <https://github.com/danielmiessler/SecLists>
2. MITRE Common Weakness Enumeration: <https://cwe.mitre.org/>
3. PCI Security Standards Council: <https://www.pcisecuritystandards.org/>
4. PCI Data Security Standard (DSS) v3.2.1 Requirements and Security Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf

5. PCI Software Security Framework - Secure Software Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-Software-Standard-v1_0.pdf
6. PCI Secure Software Lifecycle (Secure SLC) Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
7. OWASP ASVS 4.0 Testing Guide <https://github.com/BlazingWind/OWASP-ASVS-4.0-testing-guide>

Lampiran C: Standar Kriptografi

Bab “Kriptografi” bergerak lebih jauh dari sekadar mendefinisikan *best practices*. Bab ini bertujuan untuk meningkatkan pemahaman tentang prinsip-prinsip kriptografi dan mendorong adopsi metode keamanan yang lebih tangguh dan modern. Lampiran ini menyediakan informasi teknis terperinci mengenai setiap persyaratan, melengkapi standar menyeluruh yang diuraikan dalam bab “Kriptografi”.

Lampiran ini mendefinisikan tingkat persetujuan untuk berbagai mekanisme kriptografi:

- Mekanisme Approved/Disetujui (A) dapat digunakan dalam aplikasi.
- Mekanisme Legacy/Warisan (L) tidak boleh digunakan dalam aplikasi baru, tetapi mungkin masih digunakan hanya untuk kompatibilitas dengan aplikasi atau kode *legacy* yang ada. Meskipun penggunaan mekanisme ini saat ini tidak dianggap sebagai kerentanan dengan sendirinya, mekanisme tersebut harus digantikan oleh mekanisme yang lebih aman dan *future-proof* sesegera mungkin.
- Mekanisme Disallowed/Dilarang (D) tidak boleh digunakan karena saat ini dianggap rusak atau tidak memberikan keamanan yang memadai.

Daftar ini dapat digantikan (*overridden*) dalam konteks aplikasi tertentu karena berbagai alasan termasuk:

- Perkembangan baru di bidang kriptografi;
- Kepatuhan terhadap regulasi (*compliance*).

Inventarisasi dan Dokumentasi Kriptografi

Bagian ini menyediakan informasi tambahan untuk V11.1 Inventarisasi dan Dokumentasi Kriptografi.

Penting untuk memastikan bahwa semua aset kriptografi, seperti algoritma, kunci, dan sertifikat, secara teratur ditemukan, diinventarisasi, dan dievaluasi. Untuk Level 3, ini harus mencakup penggunaan pemindaian statis dan dinamis untuk menemukan penggunaan kriptografi dalam aplikasi. *Tools* seperti SAST dan DAST dapat membantu dalam hal ini, tetapi mungkin diperlukan *tools* khusus untuk mendapatkan cakupan yang lebih komprehensif. Contoh *tools* gratis (*freeware*) meliputi:

- CryptoMon - Network Cryptography Monitor - using eBPF, written in python

- Cryptobom Forge Tool: Generating Comprehensive CBOMs from CodeQL Outputs

Kekuatan Setara dari Parameter Kriptografi

Kekuatan keamanan relatif untuk berbagai sistem kriptografi ada dalam tabel ini (dari NIST SP 800-57 Part 1, hal. 71):

Security Strength	Symmetric Key Algorithms	Finite Field	Integer Factorisation	Elliptic Curve
<= 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

Contoh aplikasi:

- Finite Field Cryptography: DSA, FFDH, MQV
- Integer Factorisation Cryptography: RSA
- Elliptic Curve Cryptography: ECDSA, EdDSA, ECDH, MQV

Catatan: Bagian ini mengasumsikan bahwa belum ada komputer kuantum; jika komputer tersebut ada, estimasi untuk 3 kolom terakhir tidak lagi berlaku.

Nilai Acak (Random Values)

Bagian ini menyediakan informasi tambahan untuk V11.5 Nilai Acak (*Random Values*).

Nama	Versi/Referensi	Catatan	Status
/dev/urandom	Linux 4.8+ (Okt 2016), juga ditemukan pada iOS, Android, dan sistem operasi POSIX berbasis Linux lainnya. Berdasarkan RFC7539	Menggunakan stream ChaCha20. Ditemukan di iOS SecRandomCopyBytes dan Android Secure Random dengan pengaturan yang benar yang diberikan pada masing-masing OS.	A
/dev/random	File khusus kernel Linux untuk menyediakan data acak	Menyediakan sumber entropi berkualitas tinggi dari acak perangkat keras (<i>hardware randomness</i>)	A

Nama	Versi/Referensi	Catatan	Status
AES-NIST SP800-90A		Seperti yang digunakan dalam implementasi umum, seperti Windows CNG API BCryptGenRandom yang diatur oleh BCrypt_RNG_ALGORITHM.	A
HMA	NIST SP800-90A		A
HashNIST SP800-90A			A
get	OpenBSD, tersedia di Linux glibc 2.25+ dan macOS 10.12+	Menyediakan byte acak yang aman langsung dari sumber entropi kernel dengan API yang sederhana dan minimalis. Metode ini lebih modern dan menghindari masalah yang terkait dengan API lama.	A

Fungsi hash mendasar yang digunakan dengan HMAC-DRBG atau Hash-DRBG harus disetujui untuk penggunaan ini.

Algoritma Cipher

Bagian ini menyediakan informasi tambahan untuk V11.3 Algoritma Enkripsi.

Algoritma cipher yang disetujui (*Approved*) terdaftar berdasarkan urutan preferensi.

Algoritma Kunci		Status
Simetris	Referensi	
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D

Algoritma Kunci		
Simetris	Referensi	Status
Blowfish		D
ARC4		D
DES		D

Mode Cipher AES

Block cipher, seperti AES, dapat digunakan dengan mode operasi yang berbeda. Banyak mode operasi, seperti Electronic Codebook (ECB), tidak aman dan tidak boleh digunakan. Mode operasi Galois/Counter Mode (GCM) dan Counter with cipher block chaining message authentication code (CCM) menyediakan enkripsi terotentikasi (*authenticated encryption*) dan harus digunakan dalam aplikasi modern.

Mode yang disetujui (*Approved*) terdaftar berdasarkan urutan preferensi.

Mode	Terotentikasi	Referensi	Status	Pembatasan
GCM	Ya	NIST SP 800-38D	A	
CCM	Ya	NIST SP 800-38C	A	
CBC	Tidak	NIST SP 800-38A	L	
CCM-8	Ya		D	
ECB	Tidak		D	
CFB	Tidak		D	
OFB	Tidak		D	
CTR	Tidak		D	

Catatan:

- Semua pesan yang dienkripsi harus diotentikasi. Untuk SETIAP penggunaan mode CBC, HARUS ada algoritma MAC hashing terkait untuk memvalidasi pesan. Secara umum, ini HARUS diterapkan dalam metode Encrypt-Then-Hash (tetapi TLS 1.2 menggunakan Hash-Then-Encrypt sebagai gantinya). Jika hal ini tidak dapat dijamin, maka CBC TIDAK BOLEH digunakan. Satu-satunya aplikasi di mana enkripsi tanpa algoritma MAC diizinkan adalah enkripsi disk.
- Jika CBC digunakan, harus dijamin bahwa verifikasi *padding* dilakukan dalam waktu konstan (*constant time*).

- Saat menggunakan CCM-8, tag MAC hanya memiliki keamanan 64 bit. Ini tidak sesuai dengan persyaratan 6.2.9 yang memerlukan keamanan setidaknya 128 bit.
- Enkripsi disk dianggap di luar cakupan ASVS. Oleh karena itu, lampiran ini tidak mencantumkan metode yang disetujui untuk enkripsi disk. Untuk penggunaan ini, enkripsi tanpa otentikasi biasanya diterima dan mode XTS, XEX, serta LRW biasanya digunakan.

Key Wrapping

Cryptographic key wrap (dan unwrap kunci terkait) adalah metode untuk melindungi kunci yang ada dengan mengapsulkan (misalnya, membungkus/wrapping) kunci tersebut menggunakan mekanisme enkripsi tambahan sehingga kunci asli tidak terekspos secara jelas, misalnya selama transfer. Kunci tambahan yang digunakan untuk melindungi kunci asli ini disebut sebagai *wrap key*.

Operasi ini dapat dilakukan ketika ingin melindungi kunci di tempat-tempat yang dianggap tidak terpercaya, atau untuk mengirimkan kunci sensitif melalui jaringan yang tidak terpercaya atau di dalam aplikasi. Namun, pertimbangan serius harus diberikan untuk memahami sifat (seperti identitas dan tujuan) dari kunci asli sebelum melakukan prosedur wrap/unwrap karena hal ini dapat berimplikasi pada sistem/aplikasi sumber dan target dalam hal keamanan dan terutama kepatuhan yang mungkin mencakup jejak audit (*audit trails*) dari fungsi kunci (seperti penandatanganan) serta penyimpanan kunci yang tepat.

Secara khusus, AES-256 HARUS digunakan untuk *key wrapping*, mengikuti NIST SP 800-38F dan mempertimbangkan ketentuan jangka panjang terhadap ancaman kuantum. Mode cipher yang menggunakan AES adalah sebagai berikut, berdasarkan urutan preferensi:

Key Wrapping	Referensi	Status
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

AES-192 dan AES-128 BOLEH digunakan jika kasus penggunaan menghendakinya, tetapi alasannya HARUS didokumentasikan dalam inventarisasi kriptografi entitas.

Enkripsi Terotentikasi (Authenticated Encryption)

Dengan pengecualian enkripsi disk, data terenkripsi harus dilindungi dari modifikasi tanpa izin menggunakan beberapa bentuk skema Authenticated Encryption (AE), biasanya menggunakan skema Authenticated Encryption with Associated Data (AEAD).

Aplikasi sebaiknya menggunakan skema AEAD yang disetujui. Secara alternatif, aplikasi dapat menggabungkan skema cipher yang disetujui dan algoritma MAC yang disetujui dengan konstruksi Encrypt-then-MAC.

MAC-then-encrypt masih diizinkan untuk kompatibilitas dengan aplikasi *legacy*. Ini digunakan dalam TLS v1.2 dengan *cipher suites* lama.

Mekanisme		Status
AEAD	Referensi	
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Fungsi Hash

Bagian ini menyediakan informasi tambahan untuk V11.4 Hashing dan Fungsi Berbasis Hash.

Fungsi Hash untuk Kasus Penggunaan Umum

Tabel berikut mencantumkan fungsi hash yang disetujui dalam kasus penggunaan kriptografi umum seperti tanda tangan digital:

- Fungsi hash yang disetujui memberikan resistensi kolisi (*collision resistance*) yang kuat dan cocok untuk aplikasi keamanan tinggi.
- Beberapa dari algoritma ini menawarkan resistensi yang kuat terhadap serangan jika digunakan dengan manajemen kunci kriptografi yang tepat, sehingga secara tambahan disetujui untuk fungsi HMAC, KDF, dan RBG.
- Fungsi hash dengan output kurang dari 254 bit memiliki resistensi kolisi yang tidak memadai dan tidak boleh digunakan untuk tanda tangan digital atau aplikasi lain yang memerlukan resistensi kolisi. Untuk penggunaan lain, fungsi tersebut mungkin digunakan untuk kompati-

bilitas dan verifikasi HANYA dengan sistem *legacy*, tetapi tidak boleh digunakan dalam desain baru.

Fungsi		Status	
Hash	Referensi	Pembatasan	
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	
SHA-224	FIPS 180-4	L	Tidak cocok untuk HMAC, KDF, RBG, tanda tangan digital
SHA-512/224	FIPS 180-4	L	Tidak cocok untuk HMAC, KDF, RBG, tanda tangan digital
SHA3-224	FIPS 202	L	Tidak cocok untuk HMAC, KDF, RBG, tanda tangan digital
SHA-1	RFC 3174 & RFC 6194	L	Tidak cocok untuk HMAC, KDF, RBG, tanda tangan digital
CRC (panjang apa pun)		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

Fungsi Hash untuk Penyimpanan Kata Sandi

Untuk hashing kata sandi yang aman, fungsi hash khusus harus digunakan. Algoritma *slow-hashing* ini memitigasi serangan *brute-force* dan *dictionary attack* dengan meningkatkan kesulitan komputasi dalam pembongkaran kata sandi (*password cracking*).

KDF	Referensi	Parameter yang Diperlukan	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1	A
		t = 2: m ≥ 19456 (19 MiB), p = 1	A
		t ≥ 3: m ≥ 12288 (12 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N ≥ 2 ¹⁷ (128 MiB), r = 8	A
		p = 2: N ≥ 2 ¹⁶ (64 MiB), r = 8	A
		p ≥ 3: N ≥ 2 ¹⁵ (32 MiB), r = 8	A
bcrypt	A Future-Adaptable Password Scheme	cost ≥ 10	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterations ≥ 210,000	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterations ≥ 600,000	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterations ≥ 1,300,000	L

Fungsi derivasi kunci berbasis kata sandi yang disetujui dapat digunakan untuk penyimpanan kata sandi.

Key Derivation Functions (KDFs)

Key Derivation Functions Umum

KDF	Referensi	Status
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
MD5-based KDFs	RFC 1321	D
SHA-1-based KDFs	RFC 3174 & RFC 6194	D

Key Derivation Functions Berbasis Kata Sandi

KDF	Referensi	Parameter yang Diperlukan	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1	A
		t = 2: m ≥ 19456 (19 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N ≥ 2 ¹⁷ (128 MiB), r = 8	A
		p = 2: N ≥ 2 ¹⁶ (64 MiB), r = 8	A
		p ≥ 3: N ≥ 2 ¹⁵ (32 MiB), r = 8	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iterations ≥ 210,000	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iterations ≥ 600,000	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iterations ≥ 1,300,000	L

Mekanisme Pertukaran Kunci (Key Exchange)

Bagian ini menyediakan informasi tambahan untuk V11.6 Kriptografi Kunci Publik.

Skema KEX

Kekuatan keamanan 112 bit atau lebih tinggi HARUS dipastikan untuk semua skema Pertukaran Kunci (*Key Exchange*), dan implementasinya HARUS mengikuti pilihan parameter dalam tabel berikut.

Skema	Parameter Domain	Forward Secrecy	Status
Finite Field Diffie-Hellman (FFDH)	$L \geq 3072$ & $N \geq 256$	Ya	A
Elliptic Curve Diffie-Hellman (ECDH)	$f \geq 256-383$	Ya	A
Encrypted key transport dengan RSA-PKCS#1 v1.5		Tidak	D

Di mana parameter berikut adalah:

- k adalah ukuran kunci untuk kunci RSA.
- L adalah ukuran kunci publik dan N adalah ukuran kunci privat untuk kriptografi finite field.
- f adalah rentang ukuran kunci untuk ECC.

Setiap implementasi baru TIDAK BOLEH menggunakan skema apa pun yang TIDAK patuh terhadap NIST SP 800-56A & B dan NIST SP 800-77. Secara khusus, IKEv1 TIDAK BOLEH digunakan dalam lingkungan produksi.

Grup Diffie-Hellman

Grup berikut disetujui untuk implementasi pertukaran kunci Diffie-Hellman. Kekuatan keamanan didokumentasikan dalam NIST SP 800-56A, Lampiran D, dan NIST SP 800-57 Part 1 Rev.5.

Grup	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A

Grup	Status
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

Message Authentication Codes (MAC)

Message Authentication Codes (MAC) adalah konstruksi kriptografi yang digunakan untuk memverifikasi integritas dan keaslian (*authenticity*) suatu pesan. MAC menerima pesan dan kunci rahasia sebagai input dan menghasilkan tag berukuran tetap (nilai MAC). MAC banyak digunakan dalam protokol komunikasi aman (misalnya TLS/SSL) untuk memastikan bahwa pesan yang dipertukarkan antar pihak adalah otentik dan utuh.

Algoritma	Referensi	Status
MAC		
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A

Algoritma	Referensi	Status
MAC		
KMAC128	NIST SP 800-185	A
KMAC256	NIST SP 800-185	A
BLAKE3 (keyed_hash mode)	BLAKE3 one function, fast everywhere	A
AES-CMAC	RFC 4493 & NIST SP 800-38B	A
AES-GMAC	NIST SP 800-38D	A
Poly1305- AES	The Poly1305-AES message-authentication code	A
HMAC-SHA- 1	RFC 2104 & FIPS 198-1	L
HMAC-MD5	RFC 1321	D

Tanda Tangan Digital (Digital Signatures)

Skema tanda tangan HARUS menggunakan ukuran kunci dan parameter yang disetujui sesuai NIST SP 800-57 Part 1.

Algoritma Tanda Tangan	Referensi	Status
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA (ukuran kunci apa pun)	FIPS 186-4	D

Standar Enkripsi Pasca-Kuantum (Post-Quantum Encryption)

Implementasi PQC harus sejalan dengan FIPS-203/204/205 karena sejauh ini masih minim kode yang diperkuat (*hardened code*) maupun referensi implementasi. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

Metode *key agreement* TLS hibrida pasca-kuantum yang diusulkan mlkem768x25519 telah didukung oleh peramban-peramban utama seperti Firefox release 132 dan Chrome release 131. Metode ini dapat digunakan dalam lingkungan pengujian kriptografi atau ketika tersedia dalam pustaka (*libraries*) yang disetujui oleh industri atau pemerintah.

Lampiran D: Rekomendasi

Pendahuluan

Saat mempersiapkan versi 5.0 dari Standar Verifikasi Keamanan Aplikasi (ASVS), menjadi jelas bahwa ada sejumlah item yang sudah ada dan yang baru diusulkan yang seharusnya tidak dimasukkan sebagai persyaratan dalam versi 5.0. Hal ini mungkin karena item-item tersebut tidak termasuk dalam ruang lingkup ASVS sesuai dengan definisi untuk versi 5.0, atau alternatifnya, dirasa bahwa meskipun merupakan ide yang bagus, item-item tersebut tidak dapat dijadikan wajib.

Karena tidak ingin kehilangan semua item ini sepenuhnya, beberapa di antaranya telah disertakan dalam lampiran ini.

Mekanisme yang direkomendasikan dan termasuk dalam cakupan

Berikut ini adalah hal-hal yang termasuk dalam ruang lingkup ASVS. Hal-hal ini tidak boleh diwajibkan, tetapi sangat disarankan untuk mempertimbangkannya sebagai bagian dari aplikasi yang aman.

- Sebaiknya disediakan alat pengukur kekuatan kata sandi untuk membantu pengguna membuat kata sandi yang lebih kuat.
- Buat file `security.txt` yang dapat diakses publik di direktori root atau `.well-known` dari aplikasi yang secara jelas mendefinisikan tautan atau alamat email bagi orang-orang untuk menghubungi pemilik tentang masalah keamanan.
- Validasi input sisi klien harus diterapkan sebagai tambahan validasi pada lapisan layanan tepercaya karena ini memberikan peluang bagus untuk menemukan kapan seseorang telah melewati kontrol sisi klien dalam upaya menyerang aplikasi.
- Cegah halaman yang tidak sengaja dapat diakses dan berisi informasi sensitif agar tidak muncul di mesin pencari dengan menggunakan file `robots.txt`, header respons `X-Robots-Tag`, atau tag meta `robots.html`.
- Saat menggunakan GraphQL, terapkan logika otorisasi pada lapisan logika bisnis, bukan pada lapisan GraphQL atau resolver, untuk menghindari keharusan menangani otorisasi pada setiap antarmuka secara terpisah.

Referensi:

- Informasi selengkapnya tentang `security.txt` termasuk tautan ke RFC

Prinsip-prinsip Keamanan Perangkat Lunak

Berikut ini adalah beberapa poin yang sebelumnya ada di ASVS tetapi sebenarnya bukan persyaratan. Lebih tepatnya, ini adalah prinsip-prinsip yang perlu dipertimbangkan saat menerapkan kontrol keamanan yang, jika diikuti, akan menghasilkan kontrol yang lebih kuat. Prinsip-prinsip tersebut meliputi:

- Kontrol keamanan harus terpusat, sederhana (efisiensi desain), terbukti aman, dan dapat digunakan kembali. Hal ini harus menghindari kontrol yang duplikat, hilang, atau tidak efektif.
- Sebisa mungkin, gunakan implementasi kontrol keamanan yang telah ditulis dan diuji dengan baik sebelumnya, daripada mengandalkan implementasi kontrol dari awal.
- Idealnya, mekanisme kontrol akses tunggal harus digunakan untuk mengakses data dan sumber daya yang dilindungi. Semua permintaan harus melewati mekanisme tunggal ini untuk menghindari penyalinan dan penempelan atau jalur alternatif yang tidak aman.
- Kontrol akses berbasis atribut atau fitur adalah pola yang direkomendasikan di mana kode memeriksa otorisasi pengguna untuk suatu fitur atau item data, bukan hanya peran mereka. Izin tetap harus dialokasikan menggunakan peran.

Proses Keamanan Perangkat Lunak

Terdapat sejumlah proses keamanan yang dihapus dari ASVS 5.0 tetapi masih merupakan ide yang bagus. Proyek OWASP SAMM dapat menjadi sumber yang baik untuk mengetahui cara mengimplementasikan proses-proses ini secara efektif. Item-item yang sebelumnya ada di ASVS meliputi:

- Verifikasi penggunaan siklus pengembangan perangkat lunak yang aman yang menangani keamanan di semua tahapan pengembangan.
- Verifikasi penggunaan pemodelan ancaman untuk setiap perubahan desain atau perencanaan sprint guna mengidentifikasi ancaman, merencanakan tindakan penanggulangan, memfasilitasi respons risiko yang tepat, dan memandu pengujian keamanan.
- Pastikan bahwa semua user story dan fitur mengandung batasan keamanan fungsional, seperti “Sebagai pengguna, saya harus dapat melihat dan mengedit profil saya. Saya tidak boleh dapat melihat atau mengedit profil orang lain.”
- Pastikan ketersediaan daftar periksa pengkodean yang aman, persyaratan keamanan, pedoman, atau kebijakan bagi semua pengembang dan penguji.
- Verifikasi bahwa terdapat proses berkelanjutan untuk memastikan bahwa kode sumber aplikasi bebas dari pintu belakang (backdoor), kode berbahaya (misalnya, serangan salami, bom logika, bom waktu), dan fitur yang tidak terdokumentasi atau tersembunyi (misalnya, Easter egg, alat debugging yang tidak aman). Mematuhi bagian ini tidak mungkin dilakukan tanpa akses penuh ke kode sumber, termasuk pustaka pihak ketiga, dan oleh karena itu mungkin hanya cocok untuk aplikasi yang membutuhkan tingkat keamanan tertinggi.
- Pastikan mekanisme telah tersedia untuk mendeteksi dan menanggapi penyimpangan konfigurasi di lingkungan yang telah diimplementasikan. Hal ini dapat mencakup penggunaan

infrastruktur yang tidak dapat diubah, penerapan ulang otomatis dari basis data yang aman, atau alat deteksi penyimpangan yang membandingkan kondisi saat ini dengan konfigurasi yang telah disetujui.

- Pastikan bahwa pengerasan konfigurasi dilakukan pada semua produk, pustaka, kerangka kerja, dan layanan pihak ketiga sesuai dengan rekomendasi masing-masing.

Referensi:

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

Lampiran E - Kontributor

Kami dengan penuh rasa terima kasih mengakui kontribusi dari orang-orang berikut yang telah memberikan komentar atau membuka permintaan pull request sejak dirilisnya ASVS 4.0.0.

Jika Anda mengetahui adanya kesalahan atau ingin nama Anda ditampilkan secara berbeda, mohon beri tahu kami.

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii IZUKA Dovy- Masahiro d- (maizuka) kov (olek-si-idov)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshi Andrew van der Stock (van-deraj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Raphael Gargiulo (mgargiulo)

Cesar Kohl (cesarkohl)	inaz0	Joerg David Bruer Deatherner age (Jo- (se- erg- cu- Bruer rity- ner) dave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	MatteoSebastien Pace gio- (M4tteo12) (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig- Riccardo shony Sirigu (ric- sirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael mjang- Green cobalt (Rafael- Green1)
clallier94	Kevin W. Wall (kwwall)	Jorda Ingo Sher- Rauner man (ingo- (jshe rauner) fwdse / delete
Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	PatrickDavid DwyerClarke (coder-(davidclarke- pa- au) tros)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoy leirn Sabur (motc sabur

wet- certitude	timhemel	RL Thomas Thorn-Bandt ton (asp- (thorn-netde) shadow99)
Roel Storms (roel- storms)	Jeroen Willemsen (commjoen)	anony Kamran 31 Sai- ful- lah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Marx314 Her- rera (hansphp)
CarlosAller	Yonah Russ (yruss972)	Sandk Luboš Mai- Bretschnei- jers der (sann (bretik) NL)
Eva Sarafi- anou (esarafi- anou)	ataseren	Steve Dominique Thomas RIGHETTO (Sc00bz) righet- tod)
Steven van der Baan (svdb- ncc)	Michael Vacarella (Aif4thah)	Tonin Stefan Kisas Stre- ondi ichs- (tk- bier isa- (stre- son) ichs- baer)
hi-unc1e	sb3k (starbuck3000)	mario-Devdatta platt Akhawe (devd)

Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Jonny Wichers (JonnySchnittger)
Silvia Väli (silvi-avali)	jackgates73	lsongb - (timurozkul)
Gareth Heyes (hackver-tor)	appills	suvik chaals (chaals)
DanielPharow (At-lasHack-ert)	will Farrell (willfarrell)	Alina Paul Vasil- Mc-jeva Cann (avasil(is-jeva) misep-aul)
Sage (Sajjad-Pourali)	rbsec	Bene James Baue Jar- (mas- dine tache (james-jar-dine)
Mark Burnett (m8burnett)	dschwarz91	Cyber-Tib3rius AppSec (Cyber-AppSec)
BitnessWise (bitness-wise)	damienbod (damienbod)	Jared Stefan Meit Seel- (jmei mann fwdse (sseel-mann)

Brendan O'Connor (ussjoin)	Andrei Titov (andrettv)	Hans- Petter Fjeld (atlux-ity)
Neil Madden (Neil-Madden)	Michael Geramb (mgeramb)	Osam mackowski El-nagar (ossie git)
Ravi Balla (rav-iballa)	Hazana (hazanasec)	David Alexander MeansStein (dmeans2) hch4)
BaeSenseii (baesen-seii)	Vincent De Schutter (VincentDS)	S Mitsuaki Bani Akiyama (sban (mak1yama)
Christopher Loessl (hashier)	victorxm	MichalVeeresh Rada De- (michalveeresh radacz)(drveresh)
MaknaSEO	darkzero2022	Liam Frank (Liam De- laere) nis (jedisct1)
Otto Sulin (otosulin)	carllaw6885	AndersRichard Jo- Fritsch han (rfricz) Holme-fjord (aholmis)
mesutgung	Scott Helme (ScottHelme)	Carlo Suyash Reg- Sri-giani vas- (car- tava loreg- (suyash5053) giani)

Mark Potter (markon-web)	Arjan Lamers (alamers)	Gøran flo-Breivik (go-b-rtg)
Guillaume Déflache (guillaume-d)	Toufik Airane (toufik-airane)	Keith Sinner Hood (SoftwareSinner) (se-wareSinner) ingde
iloving	Jeroen Beckers (TheDauntless)	Joubinyu Jab-fu-bari jioka (joubin-fu-jiokayu)
execjosh (execjosh)	Alicja Kario (tomato42)	Sidne Gabriel Ribeir Marquet (Gby56) (srj-quet soft-ware)
Drew Schulz (drschulz)	bedirhan	murali Ronnie Flathers (ronnop)
Philippe De Ryck (philipped-eryck)	Malte (mal33)	Maze Andreasson Falk (andi-falk)
Javi (javi-ixeneize)	Daniel Hahn (averell23)	borislav Robin Wood (dig-in-inja)
miro2ns	Jan Dockx (jandockx)	vipin priyansh shukuma

Nat Sakimura (sakimura)	Benjamin Häublein (BenjaminHae)	unknown- user- Ra- from mazan TAŞDE- LEN (ali- tas- dln)
Pedro Escaleira (oEscal)	Josh (josh-hemphill)	Tim Avid Würte (avi- (SEC- douglen) tim)
SheHacksPurple (shehack- spurple)	fullo-cycubix	HectorIrene Eryx Mich- Pare- lin des (irene221b) Ca- ma- cho (heryxpc)
Jonah Y-M (TG- Techie)	Dhiraj Bahroos (bahroos)	Jef IzmaDoesItbeta Mei- jvis (jefm jvis)
Abdessamad TEMMAR (Tmmm- mmR)	sectroyer	Soh regoravalaz Satoh (sohsatoh)
james-t (james- bitherder)	Aram Hovsepyan (aramhovsepyan)	Jaime ValdiviaGarciaSan
iwatachan (ishowta)	Vinod Anandan (VinodAnandan)	Kevin paul- Kien williamson- (KevinKien)
endergizr	Radhwan Alshamamri (Rado0z)	Grant Cure53 Onge: (cure53) (rewt

AliR2Linux	Ads Dawson (GangGreenTemperTatum)	Williargabe Reyor (gcrow) (Bill- Reyor)
mascotter	luissaiz	Suren Piotr Manu Gliźniewicz (vx- (pglizniewicz) sec)
Tadeusz Wa- chowski (tadeuszwa- chowski)	Nasir aka Nate (andesecc)	settantLarste Haulin (LarsH)
Terence Eden (edent)	JasmineScholz	Arun Yusuf Sivad. GÜR (tea- (yusuf- vanis fgur)
Troy Marshall (troymar- shall)	Tanner Prynn (tprynn)	Nick raoul361 K. (nickific)
Azeem Ilyas (TheAxZim)	Evo Stamatov (avioli)	Tim Gavin Pot- Ray ter (Gav- (tim- in- pot- Ray97) ter87)
monis (demideus)	Marcin Hoppe (MarcinHoppe)	GrambJufdan (rams-Pike hazar) (com- put- er- sare- bad)
Jason Rogers (jason- invision)	Ben Hall (benbhall)	James WhiteCuckLabs (jame (white- hack- labs)

Alex Gaynor (alex)	Filip van Laenen (filipvanlaenen)	jeurgen	GraoMelo
Andreas Kurtz (ay-kay)	Tom Tervoort (TomTervoort)	old man (de- v- eras)	Marco Schnüriger (mar- cortw)
stiiin	infoseclearn (teaminfoseclearn)	hljupki	Noe (nmarher)
Lyz (lyz-code)	Martin Riedel (mrtnrdl)	KIM Jae- suck (tcaes	Barbara Schachner (bschach)
René Reuter (AresSec)	carhackpils	Tyler (tyler2014- sousa)	Hugo
Wouter Bloeyaert (Some- niak)	Mark de Rijk (markderijkinfosec)	Rami (pi- co- hub)	Philip D. Turner (philipdturner)
Will Chatham (willc)			